
Structured Abstractions for Learning and Adaptation in Open-World Environments

A dissertation submitted by

Shivam Goel

in partial fulfillment of the requirements
for the degree of

Doctor of Philosophy

in

Computer Science

TUFTS UNIVERSITY

May 2026

©Copyright 2026 by Shivam Goel

Dissertation Committee:

Prof. Jivko Sinapov (Chair)
Tufts University

Prof. Matthias Scheutz (Chair)
Tufts University

Prof. Reuth Mirsky
Tufts University

Prof. Matthew E. Taylor
University of Alberta

Prof. Chris Rogers
Tufts University

Abstract

Despite significant advances in artificial intelligence and robotics, and recent progress toward more flexible and open-world capable systems, many approaches still exhibit brittle behavior when confronted with changes beyond their training distributions, reflecting a continued reliance on closed-world assumptions. In real-world environments, new objects, altered dynamics, and unforeseen task variations arise naturally, and even small changes can render existing knowledge inadequate, exposing a limitation in how current systems represent and reason about their interaction with the world.

This dissertation advances the study of open-world agent and environment interaction by developing the formal, algorithmic, and representational foundations required for agents to operate under novelty. We formalize novelty as a mismatch between an agent’s internal models and the environment and introduce environment models, transformations, and evaluation methodologies for its systematic study. Building on this foundation, the dissertation shows how abstractions shape learning and behavior in open-world settings. Symbolic abstractions over actions and tasks expose the structure of execution, enabling failures to be localized and converted into targeted learning problems for rapid adaptation. Object-centric force-space abstractions represent interaction in terms of object dynamics rather than robot-specific control, enabling generalization in robotic manipulation across objects, embodiments, and physical conditions.

Viewed as a whole, this dissertation introduces a formal framework for open-world agent–environment interaction and develops the tools to study and address novelty within this setting. It further highlights how abstractions enable adaptation and generalization under changing conditions, providing a step toward artificial intelligence and robotic systems that can operate more robustly in real-world environments.

Acknowledgments

I would like to thank...

Contents

Abstract	iii
Acknowledgments	iv
1 Introduction	2
1.1 Thesis Contributions	3
1.2 Thesis Outline	5
2 Foundations of Open-World Agent–Environment Interaction	8
2.1 Open-World Environments	9
2.1.1 The Environment	9
2.1.2 The Agent	9
2.1.3 Closed-World and Open-World Assumptions	10
2.1.4 A Running Example: The Crafting Agent	11
2.1.5 Environment States and Agent Representations	12
2.2 Novelty	13
2.2.1 An Agent-Relative Definition of Novelty	13
2.2.2 A Taxonomy of Novelty by Task Impact	14
2.2.3 Novelty by Domain Element	16
2.3 A Framework for Open-World Agents	18
2.3.1 Symbolic Planning	19
2.3.2 Reinforcement Learning	21
2.3.3 Hybrid Planning and Learning	23

2.3.4	Novelty in Hybrid Systems	27
2.4	Structured Abstractions for Open-World Agents	28
2.4.1	Abstraction in Agent Systems	28
2.4.2	Action Abstractions and Adaptation	30
2.4.3	Interaction Abstractions and Generalization	31
2.4.4	The Unifying Perspective	32
2.5	Summary	33
3	Related Work and Perspectives on Open-World Autonomy	35
3.1	Open-World Sensing and Perception	35
3.2	Open-World Planning and Learning	37
3.3	Open-World Execution and Recovery	39
3.4	Foundation Models and Open-World Autonomy	41
3.4.1	Vision-Language-Action Models	41
3.4.2	Language Models as Planners	41
3.4.3	What Foundation Models Solve and What They Do Not	42
3.4.4	Complementary Perspectives	44
3.5	Toward Integrated Open-World Agents	44
3.6	Summary	46
4	Benchmark Environments for Open Worlds	48
4.1	Motivation	48
4.2	Related Work	49
4.3	NovelGridworlds	53
4.3.1	Environment Design	53
4.3.2	Novelty Taxonomy	54
4.3.3	Evaluation Metrics	58
4.3.4	Experimental Results	58
4.3.5	Key Insights	60
4.4	NovelGym	60
4.4.1	Environment Formalism	61

4.4.2	Agent Formalism	62
4.4.3	Novelty as Environment Transformation	63
4.4.4	Architecture and Implementation	64
4.4.5	Evaluation Protocol and Metrics	66
4.4.6	Benchmark Task and Novelties	67
4.4.7	Agent Architectures	69
4.4.8	Experimental Results	70
4.4.9	Convergence Criteria	75
4.4.10	Action Space	76
4.4.11	Observation Spaces	77
4.4.12	Novelty Implementations	78
4.4.13	Reward Function	79
4.4.14	Sample Configuration File	83
4.4.15	PDDL Specification	85
4.4.16	Hyperparameters	96
4.4.17	Novelty Implementation Example	97
4.5	Summary and Future Directions	99
5	Adaptive Planning and Learning for Open-World Novelty	102
5.1	Motivation	102
5.2	Related Work	104
5.3	A Neurosymbolic Architecture for Open-World Novelty Handling	108
5.3.1	Novelty Typology for Goal-Oriented Agents	109
5.3.2	Architecture Overview	110
5.3.3	Failure Recovery Strategies	112
5.4	Problem Formulation	113
5.4.1	Executors	114
5.4.2	Integrated Planning Task	114
5.4.3	The Executor Discovery Problem	115
5.5	RAPid-Learn	116

5.5.1	Overview	117
5.5.2	Executor Discovery	119
5.5.3	Knowledge-Guided Exploration	122
5.5.4	Recovery and Executor Reuse	123
5.6	Experimental Evaluation	124
5.6.1	RAPid-Learn Evaluation	125
5.6.2	Domain PDDL	131
5.6.3	Integrated Architecture Evaluation	135
5.7	Summary and Future Directions	137
6	Generalization via Interaction Abstractions	143
6.1	Motivation	143
6.2	Related Work	146
6.3	FLEX: Force-Based Learning for Articulated Object Manipulation	149
6.3.1	Articulated Object Parametrization	150
6.3.2	MDP Formulation	151
6.3.3	Interactive Joint Parameter Estimation	152
6.3.4	Robot Execution	153
6.3.5	Experimental Evaluation	153
6.3.6	Simulation Details	155
6.3.7	Learning Process	156
6.3.8	Execution Details	158
6.3.9	Baselines	158
6.3.10	Real-world setup	159
6.3.11	Additional experiments and results	160
6.3.12	Object details and dataset	160
6.4	Pushing Heavy Objects: Force Policies for Rigid Body Transport	160
6.4.1	Problem Setting	161
6.4.2	Object-Centric Policy Learning	161
6.4.3	Interactive Estimation	163

6.4.4	Robot Execution	165
6.4.5	Experimental Evaluation	165
6.5	Summary and Future Directions	167
6.5.1	Physics Engine and Scene Configuration	170
6.5.2	Object Geometry	170
6.5.3	Reference Path Generation	171
6.5.4	Domain Randomization	171
6.6	MDP Specification	171
6.6.1	State Space	171
6.6.2	Action Space	172
6.6.3	Reward Function	172
6.7	TD3 Hyperparameters	173
6.8	Network Architecture	174
6.8.1	Actor	174
6.8.2	Critic	175
6.9	Simulation Evaluation Protocol	175
6.9.1	In-Distribution Evaluation (Table II in main paper)	175
6.9.2	Path Generalization Evaluation (Table III in main paper)	176
6.9.3	RMSE Computation	176
6.9.4	Ablation: Interaction Descriptor (Table IV in main paper)	176
6.10	Hardware Deployment Details	177
6.10.1	Robot Platform	177
6.10.2	Perception Pipeline	177
6.10.3	Impedance-Based Force Probing	177
6.10.4	Policy Execution	177
6.10.5	Impedance Controller Gains	178
6.10.6	Test Object	178
6.10.7	Reference Path	178
6.11	Software and Reproducibility	178
6.11.1	Software Versions	178

6.11.2 Training Seeds and Reproducibility	179
6.11.3 Compute	179
7 Conclusion	180
7.1 Summary of Contributions	180
7.2 A Unified Perspective	181
7.3 Future Directions	183
7.4 Closing Remarks	185

List of Figures

2.1	The crafting gridworld environment	12
2.2	Two-dimensional novelty taxonomy	18
2.3	Symbolic planning	20
2.4	The reinforcement learning interaction loop	22
2.5	Agent–environment interaction in the hybrid framework	26
2.6	Structured abstractions mediating agent responses to novelty and variation . . .	32
4.1	Screenshot of the NovelGridworld-Bow-v0 environment. The agent (triangle) faces the crafting table (green). Sticks (yellow) and strings (orange) are distributed in the gridworld. Blue spaces represent air blocks through which the agent can navigate, and purple blocks represent walls.	54
4.2	Representative NovelGridworlds novelty scenarios discussed in the evaluation (fence, axe, and fire wall). Each panel shows how the grid layout and entities change under the corresponding novelty.	56
4.3	Venn diagram classifying all twelve novelties implemented in NovelGridworlds by the novelty class they affect. Novelties in overlapping regions span multiple classes simultaneously.	57
4.4	Learning performance of the RL agent on NovelGridworld-Bow-v0. The vertical line at 2 million time steps marks the introduction of novelty. Break increase (orange) shows improved reward but reduced success rate. Fire wall (green) causes complete failure. Action remapping (blue) shows partial recovery.	61

- 4.5 Illustration of the agent’s sensor representations in NovelGym. (Left) A LiDAR-like representation emitting beams at angular increments. (Right) An image-based local view representation using one-hot vector encodings. 63
- 4.6 System design of the NovelGym ecosystem. Blue highlights the environment modules and purple highlights the agent modules. The novelty injector (red) transforms the environment based on specified configurations. 65
- 4.7 Illustration of the NovelGym evaluation protocol and performance over time: the learning agent’s success rate (black) as training progresses, with novelty injection (red) dividing the pre- and post-novelty regimes. At initialization the agent’s policy is effectively random, so the curve begins at the random-action baseline (blue) and learning lifts it above this chance-level reference. Novelty injection collapses performance back to the chance-level baseline; the post-novelty learning curve then restarts from this baseline point and adaptation gradually recovers performance to S_{post} . The annotated regions correspond to the metrics in Section 4.4.5: pre-novelty asymptotic performance (S_{pre}), novelty impact (I_{novelty}), time to adapt (T_{adapt}), and post-novelty asymptotic performance (S_{post}). 67
- 4.8 Screenshot of the NovelGym environment running the pogostick crafting task. The agent (blue arrow) navigates a grid containing oak logs (brown), diamond ore (teal), platinum blocks (silver), a crafting table, and a trader (green face). The right panel displays the current episode, step count, agent facing direction, selected action, total cost, and inventory contents. 68
- 4.9 Illustration of the pre-novelty environment and three novelty scenarios: fire, fence, and chest. Each transformation modifies specific components of the environment formalism. 69

4.10	Time to adapt (T_{adapt} , lower is better) across novelty scenarios, shown on a logarithmic scale; error bars denote one standard deviation over 10 seeds. Hybrid agents (RapidLearn, RapidLearn ⁺) adapt up to an order of magnitude faster than the Transfer RL baselines, most dramatically under the fence novelty. The Chest novelty was detected only by the Transfer RL agents (the hybrid agents' plans never failed, so no adaptation was triggered), and the Fire novelty is the exception where Transfer RL (PPO) adapts quickly. Full numerical values, including the Δt metric, are reported in Table 4.6.	72
5.1	High-level architecture of the neurosymbolic cognitive architecture for novelty handling. The four main components communicate through the shared knowledge repository $\mathcal{K} = (KB, \Theta)$. Symbolic inference (task planner, goal manager) generates and executes plans, detecting novelties through expectation violations. Neural inference (vision model, agent model) detects visual and behavioral anomalies and asserts novelty descriptions into KB . When execution fails, the novelty exploration component applies a cascading sequence of recovery strategies—culminating in the RAPid-Learn executor learner—interacting with the environment through the domain interface, which mediates symbolic state observations, images, and LiDAR sensor data.	111
5.2	Conceptual overview of the integrated planning task (IPT) formalism. Each symbolic operator o_i is mapped to a subsymbolic executor $x_i = \langle I_{x_i}, \pi_{x_i}, \beta_{x_i} \rangle$ via the executor function e . The abstraction function $\alpha : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ maps environment states back to symbolic states for the planner. (a) In a working IPT, all executors realize their associated operators and the agent reaches the goal g . (b) When novelty causes an operator to fail, the IPT becomes a Stretch-IPT: the task remains solvable but the current executor mapping is broken. RAPid-Learn solves the executor discovery problem by constructing a new executor x'_2 through reinforcement learning, restoring solvability.	117
5.3	Overview of the Rapid-Learn Framework	118

- 5.4 Overview of the agent in action. (a) Illustrates the NovelGridworlds domain in the pre-novelty scenario, showing an agent in a walled arena surrounded by *trees*, and facing a *crafting-table*. (b) Shows the scenario when the environment introduces the *rubber-tree-hard* novelty and the agent reaches an impasse. (c) *Executor-Discovery* phase when the agent accommodates the novelty using our approach RAPid-KGE-UCB. (d) Agent successfully resolves the plan execution impasse. (e) Agent reaches the goal state of crafting a *pogo-stick*. 124
- 5.5 Screenshot of the NovelGridworld-Pogostick environment. The agent (triangle) can navigate in the environment and collect items such as trees (shown in yellow). The agent can use the crafting-table to craft complicated items such as tree-tap, sticks etc. Blue spaces represent air blocks through which the agent can navigate, and purple blocks represent walls. 125
- 5.6 Learning curves (success rate vs. training time steps) for four of the five novelty scenarios. All RAPid-Learn variants (KGE-UCB, KGE-UAB, EG) converge one to two orders of magnitude faster than the transfer learning baselines (Actor-Critic Transfer and Policy Reuse). Shaded regions denote one standard deviation over 10 independent trials. Transfer learning baselines did not converge on the RT hard scenario within the evaluation budget and are therefore omitted from that panel. 140
- 5.7 Baseline learning curves on the four novelties. Green curve shows the actor-critic transfer, and the red curve shows policy-reuse. Each plot demonstrates the performance during the novelty adaptation phase. 141
- 5.8 Breakdown of novelty accommodation performance across novelty categories in the Polycraft systemic evaluation. Each bar shows the goal achievement rate for the four agent configurations under each novelty type. The executor learner (Base+EL) provides the most consistent improvement on attribute, action, and concealed novelties, where symbolic recovery strategies are insufficient and learning-based executor discovery is essential. 142

- 6.1 Illustration of the two joint configurations used in FLEX. (a) A prismatic joint allows linear motion along a fixed axis h_p . (b) A revolute joint allows rotational motion around an axis h_r with radius r . The state representation for each joint type captures the joint axis direction and the current displacement or projected position, providing an object-centric description independent of robot kinematics. 150
- 6.2 Architecture of FLEX. Force-based skill learning (left) trains object-centric policies in simulation without any robot. Interactive joint parameter estimation (center) infers the object’s joint type and configuration from physical interaction. Robot execution (right) maps the learned forces to joint torques through an operational space controller, enabling deployment on any robot platform. . . . 152
- 6.3 FLEX learns force-based skills for manipulating articulated objects across various robots by decoupling policy learning from robot dynamics. The UR5e opens a cabinet, Kinova opens a microwave door, Panda opens a dishwasher rack, and a real UR5 opens a drawer—all using the same learned policies without retraining. 155
- 6.4 Architecture of the pushing heavy objects framework. The policy is trained in simulation (top right) using a parameterized MDP conditioned on an interaction descriptor z . At deployment (bottom), the robot estimates z through impedance-based force probing and constructs the object-centric state from proprioceptive sensing, then executes the learned boundary forces through compliant impedance control. 162
- 6.5 Diagrammatic representation of the state space in the parameterized MDP. The object’s pose is represented relative to the reference path through lateral deviation e_t^{lat} , heading error e_t^θ , and velocity components along the path tangent and normal. The normalized contact-resistance descriptor z conditions the policy on the interaction regime. 163
- 6.6 Chronophotographic composites of real-world box transport on the Boston Dynamics Spot. Top row: proposed force-based policy under 8.8 kg (a) and 12.5 kg (c) payloads, showing consistent trajectory tracking. Bottom row: kinematic baseline under the same conditions (b, d), showing frequent deviation and failure to reach the goal. 168

List of Tables

3.1	Coverage of open-world approaches across novelty types	45
4.1	Actions and blocks specific to each NovelGridworlds task. “Default” denotes the actions and blocks common to all environments; each task adds the listed actions and blocks to the default set.	55
4.2	Classification of all novelties based on their class and type. Some novelties span multiple classes (see Figure 4.3).	56
4.3	Performance of the planning agent across four tasks. Each task reports pre-novelty (Default) and post-novelty (Fence, Arrow/Spring, Axe) scenarios.	59
4.4	RL agent success rate on <i>NovelGridworld-Bow-v0</i> and three novelty scenarios ($n = 10$ trials). Values are mean $\pm$ std of success rate, computed at 1.8M timesteps (default) and 4M timesteps (novelty scenarios).	60
4.5	Performance metrics across novelty scenarios. S_{pre} : pre-novelty success rate; I_{nov} : novelty impact (immediate performance drop); S_{post} : post-adaptation success rate. $\uparrow$ indicates higher is better, $\downarrow$ lower is better. $-$ indicates the novelty was not detected by the agent. All values are mean $\pm$ standard deviation over 10 seeds.	71
4.6	Efficiency metrics across novelty scenarios. T_{adapt} : time steps required for adaptation to converge; Δt : difference in task completion time before and after novelty (negative values indicate the agent found a more efficient post-novelty strategy). $\downarrow$ indicates lower is better. All values are mean $\pm$ standard deviation over 10 seeds.	73

4.7	Results of unpaired t -test between the best-performing hybrid agent and each Transfer RL baseline on T_{adapt} (time steps to adapt). The hybrid advantage is statistically significant across Axe, Trader, and Fence novelties.	75
4.8	Novelties currently implemented in the NovelGym environment	80
4.9	PPO hyperparameters.	96
4.10	ICM hyperparameters.	97
4.11	Pre-novelty training routine hyperparameters.	97
4.12	Post-novelty training routine hyperparameters.	97
5.1	Performance and timesteps trained for the best pre-novelty model that was later used in transfer learning the novelty environments	128
5.2	RAPid-Learn results across five novelty scenarios. T_{adapt} : time steps to adaptation convergence. Post-novelty SR: success rate on 100 evaluation episodes. Best performing agent highlighted per scenario. In all scenarios, RAPid-Learn adapts one to two orders of magnitude faster than transfer learning baselines.	129
5.3	Results of unpaired t -test between best performing RAPid-Learn and transfer learning baselines	132
5.4	Overall systemic evaluation results in Polycraft across 288 novelties. FN_{CDT} : average false negative episodes before detection; $\text{CDT}\%$: correctly detected trials; $\text{FP}\%$: false positive rate; $\text{NRP}\%$: novelty reaction performance; $\text{GA}\%$: goal achieved rate. The executor learner improves accommodation ($\text{NRP}\%$, $\text{GA}\%$), while the agent model improves detection ($\text{CDT}\%$, FN_{CDT}).	136
6.1	FLEX results on novel object instances evaluated using the Panda robot. Training time in hours and time steps in millions. FLEX achieves the highest success rates across all object types while requiring substantially less training time than all RL baselines.	153
6.2	Cross-robot transfer results for FLEX. The same learned policies are deployed on three different robotic platforms without retraining. Performance remains consistent across platforms, demonstrating that the force-space representation enables direct embodiment transfer.	154

6.3	Agent Network Struture	157
6.4	Reinforcement Learning Hyperparameters	158
6.5	Baseline Hyperparameters for CIPS	159
6.6	Parameter generalization for pushing heavy objects. Centroid-to-path RMSE (cm) under single-parameter out-of-distribution shifts. The in-distribution baseline achieves 2.9 ± 0.4 cm. Performance remains stable across all OOD conditions.	166
6.7	Path generalization for pushing heavy objects. RMSE (cm) under novel path geometries with in-distribution physical parameters. Performance degrades gracefully with increasing curvature complexity.	166
6.8	Ablation study for the interaction descriptor z . The descriptor provides meaningful conditioning for the learned policy; removing it significantly degrades performance.	167
6.9	MuJoCo physics and contact parameters.	170
6.10	Reward components. $\Delta t = 0.025$ s; $v_{\text{ref}} = 0.3$ m/s.	173
6.11	TD3 training hyperparameters.	174
6.12	Out-of-distribution evaluation ranges. When testing one parameter OOD, all others remain in-distribution.	175
6.13	Impedance probing parameters.	177
6.14	Hardware execution parameters.	178
6.15	Software dependencies.	179

Chapter 1

Introduction

Autonomous agents and robots are increasingly expected to operate in real-world environments such as homes, warehouses, hospitals, and urban spaces. In these settings, agents must interact with diverse objects, perform varied tasks, and respond to changing environmental conditions. Unlike controlled laboratory settings, real-world environments are inherently open: new objects may appear, interaction dynamics may change, and tasks may evolve in ways that were not anticipated during system design or training. Developing systems that can operate robustly under such conditions remains a central challenge in artificial intelligence and robotics.

Most contemporary approaches are developed under closed-world assumptions, where the environment, task structure, and interaction dynamics encountered at deployment closely match those observed during training. These assumptions enable effective learning and planning, but they limit the ability of agents to operate reliably when confronted with novelty. In open-world settings, even small deviations from prior experience can invalidate learned models, cause action execution to fail, and degrade policy performance. As a result, agents must be able to remain effective when their existing knowledge becomes incomplete or incorrect.

Two fundamental challenges arise in this setting. First, when novelty leads to execution failure, an agent must be able to adapt its behavior efficiently, acquiring only the knowledge necessary to restore successful task execution without relearning from scratch. Second, even in the absence of failure, agents must be able to generalize their learned behaviors across variations in objects, environments, and embodiment, so that previously acquired skills remain applicable under new conditions. Addressing both adaptation and generalization is essential for enabling

robust operation in open-world environments.

A key difficulty in achieving these capabilities lies in how agents represent their interaction with the world. When representations are tightly coupled to specific environments or control strategies, failures induced by novelty are difficult to interpret, and learned behaviors do not transfer beyond the conditions in which they were acquired. In contrast, representations that capture meaningful structure in tasks, actions, and interactions can support both targeted adaptation and reuse across variations. In particular, structured abstractions can expose where mismatches occur during execution, restrict learning to the components that must change, and encode invariance that allow behaviors to generalize across different contexts.

This dissertation investigates the role of structured abstractions in enabling agents to operate effectively under novelty. We begin by developing formal frameworks, taxonomies, and benchmark environments that enable systematic study of novelty as a mismatch between an agent’s internal models and the environment. Building on this foundation, we examine how abstractions over actions enable efficient adaptation when execution fails, and how abstractions over interaction enable the learning of skills that generalize across objects, dynamics, and robotic embodiment.

Taken together, these contributions advance a unified perspective: while novelty introduces unavoidable mismatch in open-world environments, the structure of an agent’s representations determines whether this mismatch leads to brittle behavior or can be managed through efficient adaptation and generalizable skill learning. By organizing interaction at appropriate levels of abstraction, agents can localize failure, acquire new behaviors when needed, and reuse existing knowledge across diverse conditions.

1.1 Thesis Contributions

This dissertation develops methods and frameworks for enabling autonomous agents and robots to operate effectively in open-world environments, where previously learned knowledge may become incomplete or invalid. The contributions address three complementary aspects of this problem: (1) how to systematically study agent behavior under novelty, (2) how to enable efficient adaptation when execution failures occur, and (3) how to design interaction representations

that support generalization across diverse objects, tasks, and embodiment.

A unifying theme across these contributions is the role of structured abstractions in shaping how agents remain effective under mismatch between their internal models and the environment. In particular, this work shows how abstractions enable both efficient adaptation in the presence of failure and generalization of learned behaviors across variations in the world.

The main contributions of this dissertation are summarized as follows:

- **Evaluation frameworks for open-world novelty.** This work introduces benchmark environments, formalism, and evaluation methodologies that enable the systematic study of agent behavior under novelty. The proposed environments support controlled injection of diverse classes of novelty, including changes to objects, environment dynamics, and task structure.

In addition to enabling reproducible experimentation, these benchmarks provide metrics that characterize the dynamics of agent behavior under novelty, including how quickly deviations from expected outcomes are detected, how performance degrades following a change, and how effectively the agent recovers. These contributions establish a principled foundation for analyzing adaptation and generalization in open-world settings.

- **Action abstractions for efficient adaptation under novelty.** This work develops hybrid planning–learning methods that enable agents to recover from execution failures caused by novelty. The key idea is to leverage structured action abstractions that associate high-level operators with executable behaviors, allowing failures to be localized to specific components of a plan.

When execution fails, these abstractions enable the agent to convert a global failure into a targeted learning problem, focusing exploration on acquiring only the missing behavior required to restore successful execution. This results in significantly improved sample efficiency compared to unstructured reinforcement learning approaches, while preserving previously valid knowledge. These results demonstrate how structured representations over actions can support efficient adaptation in the presence of unexpected changes.

- **Interaction abstractions for generalizable skill learning in robotics.** This work

introduces object-centric, force-based representations for robotic manipulation that capture interaction at the level of object dynamics rather than robot-specific control. By decoupling learned policies from particular robot embodiments and control strategies, these abstractions enable skills to generalize across objects with similar physical semantics, across variations in object parameters, and across different robotic platforms.

Through experiments on articulated and heavy object manipulation tasks, this work shows that representing interaction in force space reduces unnecessary exploration and enables efficient learning of behaviors that transfer across environments. These results demonstrate how appropriate abstractions over interaction can support generalization without requiring task- or embodiment-specific retraining.

Summary. Taken together, these contributions establish a unified perspective on open-world autonomy: while novelty introduces mismatch between an agent and its environment, structured abstractions determine whether this mismatch can be managed effectively. By enabling systematic study of novelty, supporting efficient adaptation when failures occur, and promoting generalization across variations, the proposed methods provide a step toward agents that can operate robustly in real-world environments.

1.2 Thesis Outline

The remainder of this dissertation is organized as follows.

Chapter 2 presents the foundational concepts and formal framework used in the dissertation. We begin by formalizing the definition of novelty as a mismatch between an agent’s internal models and the environment, followed by the taxonomy of novelties. We further present the preliminaries of the agent framework, which integrates symbolic planning and subsymbolic execution into a unified decision-making architecture. Finally, we discuss the role of structured abstractions in mediating the agent–environment interaction. The goal of this chapter is to establish the formal vocabulary and conceptual claims that the subsequent chapters build upon: benchmark environments for evaluating agents under controlled novelty (Chapter 4), action abstractions that enable efficient adaptation under failure (Chapter 5), and interaction abstrac-

tions that support generalization across objects and embodiments (Chapter 6).

Chapter 3 is a broader literature review that situates the contributions of this dissertation within the landscape of open-world autonomy research. It reviews related work in the related disciplines that the current dissertation touches upon. Specifically, we review articles related to how open world autonomy research has evolved in the areas of Reinforcement Learning, and Hybrid Planning and Learning Systems, with used cases and applications in robotics. We also briefly review the recent advances in the area of foundation models and how they have impacted the design space for autonomous agents. Finally, we discuss how the contributions of this dissertation relate to and complement these broader developments, and identify open problems that remain.

Chapter 4 introduces benchmark environments, formalism, and evaluation methodologies for systematically studying novelty. These environments enable controlled injection of diverse classes of mismatch and provide metrics for analyzing how agent performance degrades and recovers under changing conditions. This chapter establishes an experimental foundation for evaluating adaptation and generalization in open-world settings.

Chapter 5 investigates adaptation under novelty. It presents a neurosymbolic cognitive architecture for open-world novelty handling and hybrid planning–learning methods that enable agents to recover from execution failures by leveraging structured action abstractions. These approaches utilize various counterfactual reasoning based methods to efficiently identify the source of failure and acquire missing behaviors, enabling agents to adapt to novel conditions without relearning entire tasks. By localizing failures to specific components of a plan and enabling targeted learning, these methods allow agents to efficiently acquire missing behaviors without relearning entire tasks.

Chapter 6 shows how abstractions defined at a certain level can lead to generalization in robotic manipulation. It presents object-centric, force-based learning approaches that represent interaction at the level of object dynamics. These representations enable learned skills to generalize across objects with similar semantics, across variations in physical properties, and across different robotic platforms, supporting robust performance under diverse conditions.

In Chapters 4 through 6, we also review related work specific to each technical contribution and discuss how the proposed methods compare to and complement existing approaches.

Chapter 7 concludes the dissertation by summarizing the main findings, articulating a unified perspective on how structured abstractions mediate an agent’s response to novelty, and discussing future directions for research on autonomous agents and robots operating in open-world environments.

Chapter 2

Foundations of Open-World Agent–Environment Interaction¹

This chapter develops the formal and conceptual foundations that support the remainder of the dissertation. We begin by defining the open-world setting: environments in which an agent’s knowledge is necessarily incomplete with respect to the true state of the world (Section 2.1). We then formalize the concept of novelty as a mismatch between an agent’s internal models and the environment, and introduce a taxonomy that classifies novelties by their impact on task performance (Section 2.2). With the setting and the problem defined, we turn to the agent itself: how it represents, reasons about, and acts upon the world, integrating symbolic planning and subsymbolic execution into a unified decision-making framework (Section 2.3). Finally, we examine the role of structured abstractions in mediating the agent–environment interaction, arguing that the choice of abstraction determines whether novelty-induced mismatch can be handled through efficient adaptation and generalizable behavior (Section 2.4).

Together, these foundations establish the formal vocabulary and conceptual claims that the subsequent chapters build upon: benchmark environments for evaluating agents under controlled novelty (Chapter 4), action abstractions that enable efficient adaptation under failure (Chapter 5), and interaction abstractions that support generalization across objects and em-

¹Portions of this chapter are based on: S. Goel, P. Lympieropoulos, R. Thielstrom, E. Krause, P. Feeney, P. Lorang, S. Schneider, Y. Wei, E. Kildebeck, S. Goss, M. C. Hughes, L. Liu, J. Sinapov, and M. Scheutz, “A Neurosymbolic Cognitive Architecture Framework for Handling Novelties in Open Worlds,” *Artificial Intelligence*, vol. 331, p. 104111, 2024.

bodiments (Chapter 6).

2.1 Open-World Environments

Autonomous agents operate within environments that they must perceive, reason about, and act upon in order to achieve their goals. In this section, we formalize the structure of such environments and the agents that inhabit them, and distinguish between the closed-world settings that underlie most current AI research and the open-world settings that motivate this dissertation.

2.1.1 The Environment

We consider a task environment $\mathcal{W} = \langle E, \mathcal{F} \rangle$ consisting of a set of environment states E and a set of dynamics $\mathcal{F}$ that govern how states evolve over time. The environment may contain any number of objects and other agents as part of its state, and its dynamics may be deterministic or stochastic. Environment states are agent-independent in the sense that they specify the underlying configuration of the world whether or not any particular agent observes or represents them correctly.

At any given time t , the environment is in some state $e_t \in E$. The dynamics $\mathcal{F}$ define how this state changes, both due to the natural evolution of the environment and as a consequence of actions taken by agents within it. In general, an agent does not have direct access to the full environment state e_t ; instead, it observes the environment through its sensors and acts upon it through its effectors, as described below.

2.1.2 The Agent

An agent $\mathcal{A}_g = \langle \zeta, \Xi, \mathcal{K}, \vdash \rangle$ is defined by four components: a set of sensors ζ that allow it to perceive the environment, a set of effectors Ξ that allow it to act upon it, a knowledge repository $\mathcal{K}$ that stores what the agent knows about the world, and an inference mechanism $\vdash$ that enables the agent to reason over its knowledge.

The knowledge repository $\mathcal{K} = (KB, \Theta)$ is composed of two complementary parts. The knowledge base KB stores explicit symbolic knowledge—facts, rules, and relational descrip-

tions expressed in a formal language $\mathcal{L}$. The subsymbolic component Θ stores knowledge in distributed representations, such as the weights of neural networks that encode policies, value functions, or perceptual models. The inference mechanism $\vdash$ operates over both components: for KB , it may take the form of logical deduction or planning; for Θ , it corresponds to forward inference through learned models.

At time t , the agent’s sensors generate internal representations of the partially observed environment state. Based on these observations and its knowledge $\mathcal{K}$, the agent selects actions through its inference mechanism and executes them via its effectors, producing changes in the environment state. This perceive–reason–act cycle defines the agent’s ongoing interaction with the world.

2.1.3 Closed-World and Open-World Assumptions

The distinction between closed-world and open-world settings concerns the relationship between the agent’s knowledge and the environment.

Under closed-world assumptions, the agent’s representational vocabulary and task model are treated as fixed and sufficient for the situations the agent may encounter. Specifically, the relevant objects, predicates, observations, actions, rewards, dynamics, and action effects are assumed to fall within the agent’s predefined modeling framework. This assumption has been productive: it underlies classical planning [28, 94], standard reinforcement learning formulations [123], and many supervised learning approaches to perception. Under these assumptions, the primary challenge is computational—finding good plans, learning effective policies, or achieving accurate classification within a fixed representational space.

In open-world environments, this assumption is violated. The environment may contain objects, dynamics, or relationships that the agent’s models do not represent. New objects may appear that do not correspond to any known category. Actions may produce effects that differ from what the agent’s models predict. Task structure may change in ways that invalidate previously successful strategies. Critically, these changes need not be adversarial or extreme: even routine variation in everyday environments can fall outside the scope of models designed under closed-world assumptions.

This gap between the agent’s knowledge and the true state of the world is not merely a

quantitative shortcoming that can be addressed by training on more data or enumerating more states. It is a qualitative property of the agent–environment relationship: the world may contain structure that is not derivable from, or even expressible within, the agent’s current representational framework. This observation motivates the formal treatment of novelty developed in the next section.

2.1.4 A Running Example: The Crafting Agent

To make the framework concrete, we introduce a running example that will be used throughout this chapter and the remainder of the dissertation. Consider an agent operating in a gridworld environment inspired by the crafting mechanics of Minecraft. The environment contains objects such as trees, crafting tables, ores, and storage containers. The agent can perform actions such as navigating to objects, breaking trees to obtain logs, crafting logs into planks and sticks, and assembling complex items using crafting recipes. The agent’s task is to craft a target item—say, a pogostick—which requires collecting and combining resources in a specific sequence.

In this setting, the agent maintains symbolic knowledge of the objects in the environment, their properties, the available crafting recipes, and the operators it can use (e.g., `break_tree`, `craft_planks`, `craft_pogostick`). Each operator is associated with an executor—a policy that carries out the corresponding action in the environment. The agent generates a plan (a sequence of operators) and executes it step by step. Under closed-world conditions, where the environment matches the agent’s models, this process succeeds reliably.

Now suppose the environment changes while the agent is performing its task. A new entity—a supplier—appears and may affect the agent’s ability to obtain resources. Or the properties of a known object change: trees become unbreakable unless the agent first acquires a new tool, an axe. Or a crafting recipe is altered so that a previously unnecessary ingredient is now required. Each of these changes may constitute a novelty for the agent: an aspect of the environment that is not derivable from, or not expressible within, its current knowledge repository. How the agent responds—whether it can detect the change, diagnose its impact, and adapt its behavior—depends on the structure of its representations and reasoning capabilities. The formal treatment of novelty in Section 2.2 and the agent framework in Section 2.3 provide the machinery for analyzing these situations precisely.



Figure 2.1: The crafting gridworld environment used as a running example throughout this dissertation. The agent (center) operates among objects such as trees, crafting tables, and ores. A novel actor—the supplier, highlighted in blue—has been introduced and may affect the agent’s ability to complete its task. Adapted from Goel et al. [43].

2.1.5 Environment States and Agent Representations

An environment state $e \in E$ specifies the complete configuration of the world at a given time, including all objects, their properties, spatial arrangements, and dynamic relationships. The agent, however, does not operate over environment states directly. Instead, it constructs representations of the world using its available representational resources.

At the symbolic level, the agent uses a formal language $\mathcal{L}$, consisting of predicates, variables, constants, and possibly functions, to describe aspects of the environment. A symbolic state description $\tilde{s} \in \tilde{\mathcal{S}}$ is a partial description of an environment state in this language. In planning domains, this description can be instantiated as a set of fluents $\mathcal{F}$ —state variables whose values may change over time—or as a set of grounded predicates that are used by the planner. When unlisted predicates are treated as false, this is a closed-world convention of the planning representation rather than a claim that the agent has complete knowledge of the environment.

At the subsymbolic level, the agent may represent the environment through continuous

state vectors $s \in \mathcal{S}$, which may include raw sensory observations (e.g., images, Virtual LiDAR representations), learned feature representations, or structured numerical descriptions of the environment. These representations are typically used for perception and low-level control, and may be learned from data rather than explicitly defined. The agent’s knowledge repository $\mathcal{K}$ contains the symbolic and subsymbolic representations that it uses to reason about the world and select actions. The inference mechanism $\vdash$ operates over these representations to generate plans, predict outcomes, and select actions. The agent’s sensors and effectors mediate the mapping between the environment and its internal representations, but this mapping is not necessarily complete or accurate.

2.2 Novelty

Novelty is not a property of the environment alone, but a relation between the environment and the agent’s knowledge of it. In this section, we formalize this concept and develop a taxonomy of novelty types based on their impact on an agent’s task performance.

2.2.1 An Agent-Relative Definition of Novelty

We depart from treatments that define novelty as an intrinsic property of objects or events [14], and instead adopt an agent-relative view: an aspect of the environment is novel for an agent if and only if the agent cannot represent or derive it from its current knowledge [43, 99].

Definition 2.2.1 (Novelty). *Let $\mathcal{A}_g$ be an agent with knowledge repository $\mathcal{K} = (KB, \Theta)$ and inference mechanism $\vdash$. A representation ν of an aspect of an environment state $e \in E$ is a novelty for $\mathcal{A}_g$ if $\mathcal{K} \not\vdash \nu$, where ν may be a symbolic expression in $\mathcal{L}$ or a subsymbolic representation.*

This definition captures the intuition that novelty is epistemic: what is novel for one agent may be entirely familiar to another. A mass spectrometer is novel to a patient who has never encountered one, but not to the clinician who uses it daily. In the context of task-performing agents, novelty arises whenever the environment presents aspects that fall outside the agent’s current representational and inferential capacity.

Definition 2.2.1 encompasses several distinct ways in which mismatch can manifest. Novelty may involve entirely new elements—objects, predicates, actions, or states that the agent’s models do not contain—or it may involve changes in how existing elements behave. Formally, a novelty can induce extensions or modifications of the agent’s models such that at least one of the following holds:

$$\mathcal{F}' \setminus \mathcal{F} \neq \emptyset, \quad \mathcal{O}' \setminus \mathcal{O} \neq \emptyset, \quad \mathcal{S}' \not\subseteq \mathcal{S}, \quad \mathcal{A}' \not\subseteq \mathcal{A}, \quad P' \neq P. \quad (2.1)$$

The first four conditions capture the introduction of entirely new elements that the agent’s models do not contain. The fifth condition captures changes in the environment’s dynamics for existing state-action pairs—situations where the agent’s learned transition model no longer predicts the environment’s behavior correctly. This distinction matters because many real-world novelties do not introduce new elements but instead alter how existing elements behave.

2.2.2 A Taxonomy of Novelty by Task Impact

In order to reason about how agents should respond to novelty, it is useful to classify novelties by their impact on the agent’s ability to achieve its goals. Some novelties may entirely prevent the agent from solving a task, while others may only cause minor disruptions or even present new opportunities for improved performance. To capture this variation, we introduce a taxonomy of novelty types based on how they affect the agent’s ability to solve a given planning task $\mathcal{T}$.

The following definitions formalize four categories of novelty based on their relationship to the agent’s task performance [43]. In each case, the novelty type is defined relative to a specific agent $\mathcal{A}_g$ and task $\mathcal{T}$, since the same environmental change may have different implications for different agents or tasks.

Definition 2.2.2 (Prohibitive Novelty). *A novelty ν is prohibitive for agent $\mathcal{A}_g$ and task $\mathcal{T}$ if, for all plans ρ that $\mathcal{A}_g$ can generate from its current knowledge, ρ does not solve $\mathcal{T}$, but there exists a plan ρ_ν that solves $\mathcal{T}$ and can be derived from $KB \cup \{\nu\}$.*

Prohibitive novelties represent aspects of the environment that the agent *must* represent and reason with in order to generate a successful plan. Without accommodating the novelty,

no plan exists within the agent’s current knowledge that achieves the goal. For example, in the running example from Section 2.1.4, the agent relies on a crafting table to combine resources into the target item. If no crafting table is available in the current state description, then the agent’s existing knowledge contains no valid plan for crafting the pogostick. This constitutes a prohibitive novelty when the environment nevertheless contains an unmodeled way to make the task solvable—for instance, a supplier that can provide a crafting table, a storage container that contains one, or a new recipe that no longer requires one. Until such information is discovered and incorporated into the agent’s knowledge base, the agent cannot construct a successful plan.

Definition 2.2.3 (Obstructive Novelty). *A novelty ν is obstructive for agent $\mathcal{A}_g$ and task $\mathcal{T}$ if it causes the executor $e(o)$ for some operator o in a plan for $\mathcal{T}$ to fail—that is, execution completes without $\alpha(s') \models \omega_o$ for the post-execution state s' .*

Obstructive novelties manifest during plan execution: the agent may construct a symbolic plan that is valid according to its current models, but an operator fails when executed in the environment. For example, in the crafting domain, the agent may plan to break a tree to obtain logs, assuming that trees can be broken directly. If trees now require the agent to hold a tool before they can be broken, the `break_tree` operator fails despite being applicable under the agent’s prior model, making the novelty obstructive.

Definition 2.2.4 (Beneficial Novelty). *A novelty ν is beneficial for agent $\mathcal{A}_g$ and task $\mathcal{T}$ if at least one plan derivable from KB already solves $\mathcal{T}$, and there exists a plan ρ_ν derivable from $KB \cup \{\nu\}$ such that ρ_ν solves $\mathcal{T}$ with lower cost than any plan derivable from KB alone that also solves $\mathcal{T}$.*

When all operators have unit cost, lower cost corresponds to shorter plan length. Beneficial novelties provide new capabilities, resources, or interactions that improve task performance without being necessary for task success. For example, a new tool that accelerates resource collection, or a newly introduced supplier who can provide an ingredient more directly than the agent’s existing strategy, would constitute a beneficial novelty. The agent need not accommodate such novelties in order to complete the task, but recognizing them allows it to generate lower-cost plans.

Definition 2.2.5 (Nuisance Novelty). *A novelty ν is a nuisance for agent $\mathcal{A}_g$ and task $\mathcal{T}$ if at least one plan derivable from KB already solves $\mathcal{T}$, and incorporating ν into the agent’s knowledge does not enable any plan that is shorter or less costly than the best plan derivable from KB alone.*

Nuisance novelties are aspects of the environment that are genuinely unknown to the agent but have no bearing on task performance. They may cause minor diversions or require brief investigation, but accommodating them yields no task benefit. A new decorative object in the environment, or a change to the visual appearance of an irrelevant entity, would typically constitute nuisance novelties.

These four categories characterize novelties in terms of their impact on the agent’s ability to solve a task. A novelty may make planning impossible without additional knowledge (prohibitive), cause failure during plan execution (obstructive), enable a lower-cost plan (beneficial), or fail to improve task performance while also not preventing task completion (nuisance). These categories are agent- and task-relative rather than solely intrinsic properties of the environment: the same environmental change may have different effects for different agents, tasks, or plans. They therefore provide a principled vocabulary for reasoning about what an agent must do in response to a detected novelty.

2.2.3 Novelty by Domain Element

Orthogonal to the task-impact taxonomy, novelties can also be classified by the type of domain element they affect. Whereas the task-impact taxonomy characterizes how a novelty affects the agent’s ability to complete a task, the domain-element taxonomy characterizes what aspect of the environment or agent–environment interaction has changed. Following the NovelGridworlds taxonomy [9], we distinguish three broad classes: object, attribute, and action novelty. This classification is still relative to a particular agent and task, rather than being an intrinsic property of the environment alone.

Object novelty. Object novelties occur when a previously unknown object or entity appears in the environment. Such an object does not correspond to any category in the agent’s current model of the world and may introduce unfamiliar properties, affordances, or constraints. For

example, a new fence object may appear around task-relevant resources, requiring the agent to discover how to interact with it before those resources can be accessed.

Attribute novelty. Attribute novelties occur when a known object remains recognizable, but one or more of its properties or effects changes. In this case, the agent can identify the object, but its prior model of how the object behaves is no longer accurate. For example, breaking a tree may produce a different quantity of wood than expected, or a wall may acquire a dangerous property, such as a fire hazard that penalizes the agent for moving near it.

Action novelty. Action novelties occur when the agent’s action space changes, or when the effects of known actions differ from the agent’s expectations. A new action may become available, such as a jump or chop action, or an existing action may be remapped so that it produces a different outcome than before; for example, a move-forward action may unexpectedly move the agent backward. In such cases, the agent must revise its model of what actions are available and what effects they produce.

These two classification schemes—by task impact and by domain element—are complementary. A given novelty can be described both in terms of what changed in the domain and in terms of how that change affects the agent’s task performance. For example, a fence is an object novelty because it introduces a new object into the environment, but it may be prohibitive, obstructive, or merely a nuisance depending on whether it blocks access to task-relevant resources and whether the agent has another way to reach them. Similarly, a new chop action is an action novelty, but it may be beneficial if it allows the agent to collect resources more efficiently. Figure 2.2 illustrates this two-dimensional classification with examples from the crafting scenario introduced in Section 2.1.4.

Returning to the running example, the axe novelty illustrates how a single environmental change can be classified along both dimensions. The introduction of the axe is an object novelty, and the change to tree-breaking dynamics is an action novelty (the `break_tree` action no longer succeeds without holding an axe). At the task-impact level, the novelty is obstructive: the agent can still generate a plan that includes `break_tree`, but the executor $e(\text{break_tree})$ fails during execution because the environment now requires a tool the agent’s prior executor does not use.

Domain Element (what changed)				
Task Impact	Object		Attribute	Action
	Prohibitive	New tool required (axe to break trees)	Recipe changed (new ingredient needed)	Old action removed (cannot break by hand)
	Obstructive	Obstacle blocks path (new wall entity)	Object too heavy (weight increased)	Action effect changed (move distance halved)
	Beneficial	New trade partner (supplier agent)	Resource doubled (tree yields more logs)	New shortcut action (craft multiple at once)
	Nuisance	Decorative object (no task relevance)	Visual change (tree color changed)	Irrelevant action (new emote available)

Figure 2.2: The two-dimensional novelty taxonomy. Columns classify novelties by the type of domain element affected (object, attribute, or action). Rows classify novelties by their impact on the agent’s task performance (prohibitive, obstructive, beneficial, or nuisance). Each cell illustrates the combination with a concrete example from the crafting scenario. Both dimensions are agent-relative: the same environmental change may fall into different categories for different agents depending on their knowledge and current task.

If the agent’s architecture decomposes behavior into operators and executors, it can detect that the `break_tree` executor failed, identify the mismatch between expected and observed effects, and focus its adaptation on acquiring a new executor for this specific operator. If, on the other hand, the agent uses a monolithic representation, the localization of the failure is not clear, with no indication of what went wrong or what must change, and the agent may need to adapt the entire policy to the new environment. This contrast—between structured and unstructured representations—is the central theme of this dissertation.

2.3 A Framework for Open-World Agents

Having defined the environment and the nature of novelty, we now turn to the structure of the agent itself. The agents studied in this dissertation operate by combining symbolic reasoning at the level of task planning with subsymbolic execution at the level of environment interaction. This section formalizes the components of this framework: symbolic planning, reinforcement learning, and their integration into a hybrid architecture. More specifically, it provides the

definitions that will be used throughout the dissertation.

2.3.1 Symbolic Planning

The agent’s high-level decision-making is conducted through symbolic planning, which enables structured reasoning over abstract representations of states and actions. Following the representation introduced in Section 2.1.5, we adopt a STRIPS-style formalism [28] in which symbolic states, preconditions, and effects are defined over grounded literals. This representation is used throughout the dissertation and provides the foundation for the operator–executor consistency conditions developed in Chapter 5.

Symbolic states. Let $\mathcal{L}$ be the agent’s formal language of predicates, variables, constants, and possibly functions. A *grounded literal* is an atom $p(c_1, \dots, c_k)$ or its negation $\neg p(c_1, \dots, c_k)$, where p is a predicate symbol and each c_i is a constant. We write A for the set of all grounded literals over $\mathcal{L}$. A symbolic state $\tilde{s} \in \tilde{\mathcal{S}}$ is a consistent subset $\tilde{s} \subseteq A$ containing no atom together with its negation. Under the closed-world convention (Section 2.1.5), literals not contained in $\tilde{s}$ are treated as false. In propositional planning domains, $\tilde{s}$ is typically represented by the set of positive ground atoms that hold; negated literals are implicit. A conjunction of positive literals Φ is satisfied in $\tilde{s}$, written $\tilde{s} \models \Phi$, if and only if $\Phi \subseteq \tilde{s}$.

Planning domain and task. The agent’s symbolic model is organized into a *planning domain* and a *planning task*. The planning domain $\Sigma = \langle \mathcal{F}, \mathcal{O} \rangle$ specifies the domain vocabulary: $\mathcal{F}$ is the set of fluent symbols (predicate and function symbols whose grounded instances may change over time), and $\mathcal{O}$ is a set of operator schemas. A planning task is a tuple $\mathcal{T} = \langle \Sigma, \tilde{s}_0, g \rangle$, equivalently written $\mathcal{T} = \langle \mathcal{F}, \mathcal{O}, \tilde{s}_0, g \rangle$, where $\tilde{s}_0 \in \tilde{\mathcal{S}}$ is the initial state and g is a goal condition over literals (a conjunction of positive literals, or a disjunctive normal form thereof). Throughout the dissertation, we write $\psi_o = \psi_o$ for the precondition literal set of operator o and $\omega_o = \omega_o$ for its effect literal set.

Operators. Each operator schema defines a parameterized action template. A *grounded operator* $o \in \mathcal{O}$ is specified as

$$o = \langle \psi_o, \omega_o^+, \omega_o^- \rangle,$$

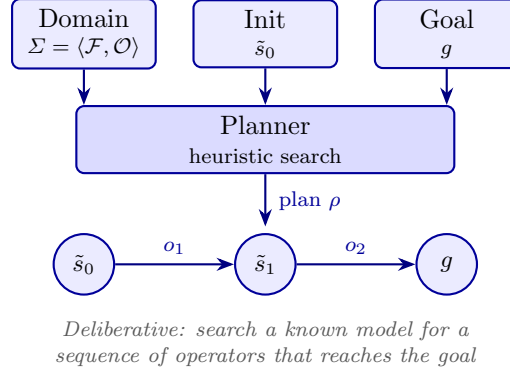


Figure 2.3: Symbolic planning. Given a domain model $\Sigma = \langle \mathcal{F}, \mathcal{O} \rangle$, an initial symbolic state $\tilde{s}_0$, and a goal condition g , a planner searches for a plan $\rho = [o_1, o_2, \dots]$ —a sequence of operators that transforms $\tilde{s}_0$ into a state satisfying g . Reasoning is deliberative and model-based, operating over discrete symbolic states $\tilde{s}_i \in \tilde{\mathcal{S}}$.

where $\psi_o \subseteq \Lambda$ is a set of positive literals whose conjunction must hold for the operator to be applicable, $\omega_o^+ \subseteq \Lambda$ is the *add list* of positive literals made true by the operator, and $\omega_o^- \subseteq \Lambda$ is the *delete list* of positive literals made false. When an operator’s effects are referenced as a single set—for example, in effect satisfaction checks during execution— ω_o denotes the postcondition add list ω_o^+ . Operator schemas are instantiated over domain objects to produce grounded operators; for instance, a schema `break(?x)` grounded with a tree constant yields a grounded operator with concrete preconditions and effects. Domains with numeric fluents extend this representation with assignment and arithmetic effects, as in PDDL [94]; the domains evaluated in this dissertation are predominantly propositional.

An operator o is *applicable* in symbolic state $\tilde{s}$ if $\tilde{s} \models \psi_o$, equivalently $\psi_o \subseteq \tilde{s}$. The operator’s effects are *satisfied* in $\tilde{s}$, written $\tilde{s} \models \omega_o$, if $\omega_o^+ \subseteq \tilde{s}$ and $\omega_o^- \cap \tilde{s} = \emptyset$. A *partial effect failure* occurs when a nonempty strict subset of ω_o^+ is observed after execution; a *total effect failure* occurs when none of ω_o^+ is observed.

State transitions and plans. Applying an operator induces a deterministic state transition. The *application function* $\Gamma : \tilde{\mathcal{S}} \times \mathcal{O} \rightarrow \tilde{\mathcal{S}}$ is defined by

$$\Gamma(\tilde{s}, o) = \begin{cases} (\tilde{s} \setminus \omega_o^-) \cup \omega_o^+ & \text{if } \psi_o \subseteq \tilde{s}, \\ \text{undefined} & \text{otherwise.} \end{cases}$$

A *plan* is a finite sequence of grounded operators $\rho = [o_1, \dots, o_n]$. Executing ρ from $\tilde{s}_0$ produces the sequence of states $\tilde{s}_{i+1} = \Gamma(\tilde{s}_i, o_i)$ for $i = 1, \dots, n$, provided each operator is applicable in its respective state. The plan is *valid* (or a *planning solution*) if every operator is applicable and the final state satisfies the goal: $\tilde{s}_n \models g$.

Operator dependencies. Operators within a plan may depend on one another through shared literals. Operator o_j *depends on* operator o_i , written $o_i \prec o_j$, if at least one effect of o_i is required by the preconditions of o_j :

$$o_i \prec o_j \iff \omega_{o_i} \cap \psi_{o_j} \neq \emptyset.$$

Classical planning assumes deterministic transitions and complete, correct domain models. Planning domains and problems are commonly specified using the Planning Domain Definition Language (PDDL) [94], and solved using domain-independent heuristic search algorithms such as Fast Downward [54] or Metric-FF [56].

In the context of this dissertation, symbolic planning provides the agent with the ability to decompose long-horizon tasks into sequences of abstract actions, as illustrated in Figure 2.3. This decomposition is valuable precisely because it creates structure that can be exploited when novelty occurs: if an operator fails, the agent can identify *which* step of the plan was affected and reason about what must change.

2.3.2 Reinforcement Learning

While symbolic planning reasons over abstract state descriptions, the agent must ultimately execute actions in the environment, where internal state representations may be high-dimensional, observations partial, and dynamics stochastic. Reinforcement learning provides the framework for learning effective behaviors through direct interaction [123], in which the agent repeatedly selects actions and observes the resulting states and rewards (Figure 2.4). At the execution level, $\mathcal{S}$ denotes the agent’s internal state representation—which may consist of sensor readings, learned features, or structured numerical descriptions—rather than the full environment state $e \in E$ (Section 2.1.5).

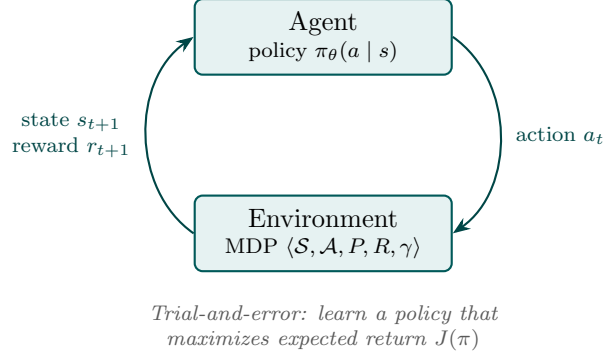


Figure 2.4: The reinforcement learning interaction loop. At each time step the agent observes a state, selects an action a_t according to its policy π_θ , and receives from the environment a next state s_{t+1} and a reward r_{t+1} . Through repeated trial-and-error interaction with the MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, the agent learns a policy that maximizes the expected return $J(\pi)$.

Markov decision processes. The agent's interaction with the environment at the primitive level is modeled as a Markov Decision Process (MDP), defined as $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of primitive actions, $P(s' | s, a)$ defines the probability of transitioning to state s' after taking action a in state s , $R(s, a, s')$ is the immediate reward received upon this transition, and $\gamma \in [0, 1)$ is the discount factor. The transition dynamics and reward satisfy the *Markov property*: the distribution over next states and rewards depends only on the current state and action, not on prior history. When transitions are deterministic, $P(s' | s, a) = 1$ for a unique next state s' and 0 otherwise; deterministic transition functions, as used in gridworld domains (Chapter 4), are a special case of this formulation. At each time step t , the agent observes state $s_t \in \mathcal{S}$, selects action $a_t \in \mathcal{A}$, transitions to s_{t+1} according to $P(\cdot | s_t, a_t)$, and receives reward $r_{t+1} = R(s_t, a_t, s_{t+1})$.

Policies. A policy $\pi(a | s)$ specifies the agent's behavior by defining a probability distribution over actions given a state; deterministic policies are the special case where $\pi(a | s) \in \{0, 1\}$ for each (s, a) pair. The agent's objective is to find a policy that maximizes expected cumulative discounted reward. The *return* from time step t is $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, the performance objective is $J(\pi) = \mathbb{E}_\pi[G_0 | s_0]$, and an *optimal policy* satisfies $\pi^* = \arg \max_\pi J(\pi)$. The state-value function is $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$, and the action-value function is $Q^\pi(s, a) = \mathbb{E}_\pi[G_t |$

$s_t = s, a_t = a]$. These satisfy the Bellman expectation equations:

$$V^\pi(s) = \mathbb{E}_\pi [r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s], \quad (2.2)$$

$$Q^\pi(s, a) = \mathbb{E}_\pi [r_{t+1} + \gamma \mathbb{E}_{a' \sim \pi(\cdot \mid s_{t+1})} Q^\pi(s_{t+1}, a') \mid s_t = s, a_t = a]. \quad (2.3)$$

In discrete settings with finite $\mathcal{S}$ and $\mathcal{A}$, value functions can be represented in tabular form; in continuous settings where $\mathcal{S} \subseteq \mathbb{R}^n$ and $\mathcal{A} \subseteq \mathbb{R}^m$, policies and value functions are approximated using parameterized function approximators $\pi_\theta(a \mid s)$, $V_\phi(s)$, or $Q_\phi(s, a)$, optimized by algorithms such as DQN [96], PPO [117], and SAC [52].

Semi-Markov decision processes and temporal abstraction. Long-horizon tasks require decisions at multiple levels of temporal granularity. Sutton et al. [124] formalize temporal abstraction in the framework introduced in *Between MDPs and Semi-MDPs*: given a base MDP $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, an *option* $\omega = \langle \mathcal{I}_\omega, \pi_\omega, \beta_\omega \rangle$ consists of an initiation set $\mathcal{I}_\omega \subseteq \mathcal{S}$ specifying the states in which the option may be initiated, an intra-option policy $\pi_\omega(a \mid s)$ that selects primitive actions while the option executes, and a termination function $\beta_\omega(s) \in [0, 1]$ giving the probability that the option terminates upon entering state s . When the agent selects an option ω in a state $s \in \mathcal{I}_\omega$, the intra-option policy executes primitive actions; after each transition to s' , the option terminates with probability $\beta_\omega(s')$ or continues. A set of options Ω together with a policy over options $\mu(\omega \mid s)$ defines a *semi-Markov decision process* (SMDP) over $\mathcal{S}$, in which high-level decisions select options and each option induces a semi-Markov transition of random duration [124, 123]. Primitive actions are the degenerate special case of options that are available in every state, follow a single primitive action, and terminate with probability one.

2.3.3 Hybrid Planning and Learning

We now formalize the integration of symbolic planning (Section 2.3.1) with reinforcement learning (Section 2.3.2) following the integrated planning task formulation introduced by Sarathy et al. [112]. The central object is the *integrated planning task* (IPT), which couples a symbolic planning task with subsymbolic execution in a base MDP through a detector and an executor

mapping. In this section we state the closed-world base case: the symbolic model is assumed to be correct, every operator has a corresponding executor, and the detector supplies the symbolic state descriptions used to monitor execution. Later chapters relax this correspondence to characterize novelty-induced executor failures in RAPid-Learn [42].

Integrated planning tasks. An *integrated planning task* (IPT) is a tuple

$$\mathfrak{T} = \langle \mathcal{T}, M, \alpha, e \rangle,$$

where $\mathcal{T}$ is a symbolic planning task, M is the MDP in which the agent acts, α is a detector that maps MDP states to symbolic states, and e maps symbolic operators to executors. Throughout this section, $s \in \mathcal{S}$ denotes an MDP state at the subsymbolic execution level, while $\tilde{s} \in \tilde{\mathcal{S}}$ denotes a symbolic state used by the planner. The detector relates these two levels by producing $\alpha(s) \in \tilde{\mathcal{S}}$ from an MDP state s . In the crafting running example (Section 2.1.4), $\mathcal{T}$ contains the symbolic recipe and operator model for crafting the pogostick, M is the gridworld interaction process, α converts the agent’s current MDP state into literals such as whether it has logs or is facing a crafting table, and e maps operators such as `break_tree` and `craft_planks` to their corresponding executors.

The *planning task* $\mathcal{T} = \langle \Sigma, \tilde{s}_0, g \rangle$ specifies the symbolic problem (Section 2.3.1): $\Sigma = \langle \mathcal{F}, \mathcal{O} \rangle$ defines the fluent vocabulary and operator set, $\tilde{s}_0 \in \tilde{\mathcal{S}}$ is the initial symbolic state, and g is the goal condition. A symbolic planner operates only on $\mathcal{T}$, producing a plan $\rho = [o_1, \dots, o_n]$ when such a sequence reaches a state satisfying g .

The *MDP* $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ models interaction at the execution level (Section 2.3.2). Its states $\mathcal{S}$ need not be symbolic: they may be continuous, high-dimensional, or otherwise subsymbolic. Executors defined over M select primitive actions from $\mathcal{A}$ and terminate according to conditions evaluated over MDP states.

The *detector* $\alpha : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ grounds MDP states in the symbolic vocabulary by mapping each state $s \in \mathcal{S}$ to a symbolic state $\alpha(s) \in \tilde{\mathcal{S}}$. Thus, $\alpha(s)$ is the symbolic description used to test whether operator preconditions hold before dispatching an executor and whether operator effects hold after execution.

The *executor mapping* $e : \mathcal{O} \rightarrow \mathcal{X}$ assigns each grounded symbolic operator $o \in \mathcal{O}$ an executor $e(o) \in \mathcal{X}$ that realizes that operator in M . We write $\mathcal{X}$ for the set of executors available over M . For example, $e(\text{break_tree})$ is the low-level behavior that performs the interaction needed to obtain logs, while $e(\text{craft_planks})$ is the behavior that carries out the corresponding crafting step once the required resources are available.

Figure 2.5 depicts this two-level structure. The top row contains symbolic states $\tilde{s}_i$ and operators o_i in a plan; the bottom row contains MDP states s_i and the corresponding executors $e(o_i)$. The detector α maps bottom-level MDP states back to top-level symbolic descriptions, allowing the agent to check whether execution has achieved the intended symbolic transition.

Executors. Following Sarathy et al. [112], an executor for M is a triple $x = \langle I_x, \pi_x, \beta_x \rangle$. The initiation set $I_x \subseteq \mathcal{S}$ specifies where the executor may be started. The intra-executor policy $\pi_x(a \mid s_0, s)$ gives the probability of selecting primitive action $a \in \mathcal{A}$ when the executor was initiated in state s_0 and is currently in state s . The termination condition $\beta_x(s_0, s) \in \{0, 1\}$ indicates whether the executor terminates in s given initiation state s_0 . Thus, an executor is an option [124] whose policy and termination condition may depend on the state in which it was initiated [112]. Through e , each symbolic operator in a plan is realized by such a temporally extended policy.

Operator–executor consistency. Following the terminology of Sarathy et al. [112], an executor $e(o)$ is *accurate to* an operator $o \in \mathcal{O}$ when its initiation and termination conditions agree with the symbolic specification of o . In the STRIPS-style setting used here, this requires

$$I_{e(o)} \supseteq \{s \in \mathcal{S} : \alpha(s) \models \psi_o\}$$

and

$$\beta_{e(o)}(s_0, s) = \begin{cases} 1 & \text{if } \alpha(s) \models \omega_o, \\ 0 & \text{otherwise.} \end{cases}$$

In other words, if the symbolic planner says that o is applicable because $\alpha(s)$ satisfies ψ_o , then the executor $e(o)$ must be allowed to start from the underlying MDP state s . Similarly,

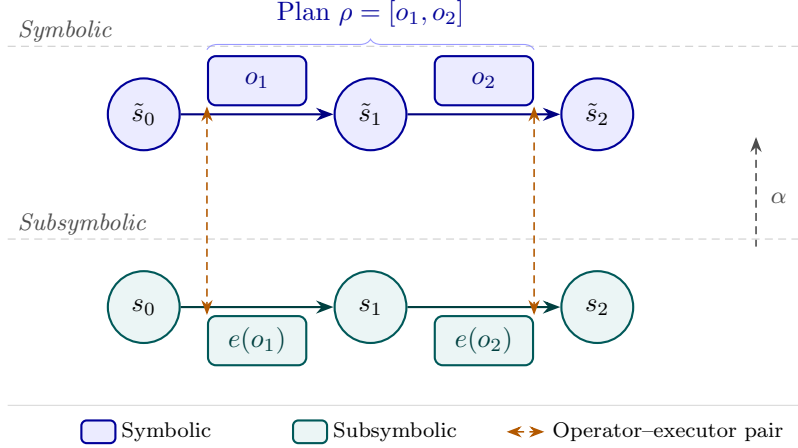


Figure 2.5: The two-level structure of the hybrid planning and learning framework. **Top (symbolic):** The agent generates a plan ρ as a sequence of operators o_i transitioning between symbolic states $\tilde{s}_i \in \tilde{\mathcal{S}}$. **Bottom (subsymbolic):** Execution occurs in MDP states $s_i \in \mathcal{S}$, and transitions are realized by executors $e(o_i)$ —policies that implement the corresponding operators. The abstraction function $\alpha : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ maps MDP states to symbolic descriptions. Each operator-executor pair (dashed orange) links a symbolic action to its realization in the MDP. Novelty manifests when $e(o_i)$ no longer achieves the intended symbolic transition despite a valid plan.

once execution reaches a new MDP state s' , the executor should terminate precisely when the detector reports that the intended symbolic effects now hold, i.e., $\alpha(s') \models \omega_o$. In the crafting example, if o is `break_tree` and its effect is `has_log`, then $e(o)$ may start in any MDP state whose detected symbolic state satisfies the preconditions for breaking the tree, and it terminates when the detector maps the resulting MDP state to a symbolic state containing `has_log`. An IPT is *well-formed* when every mapped executor is accurate to its corresponding operator.

Planning and execution. A *planning solution* to $\mathfrak{T}$ is a valid plan $\rho = [o_1, \dots, o_n]$ for $\mathcal{T}$. The plan induces the executor sequence $[e(o_1), \dots, e(o_n)]$, executed sequentially in M : after $e(o_i)$ terminates, the detector is used to evaluate the symbolic state before dispatching $e(o_{i+1})$. An *execution solution* is such an induced executor sequence that, when run from an MDP state s_0 satisfying $\alpha(s_0) = \tilde{s}_0$, terminates in some state s' with $\alpha(s') \models g$. The IPT is *plannable* if a planning solution exists and *solvable* if an execution solution exists. In a well-formed closed-world IPT whose executors terminate successfully in finite time, plannability implies solvability: the symbolic plan can be grounded into the corresponding sequence of executors. For the pogostick task, a symbolic plan might include steps such as `approach_tree`,

`break_tree`, `approach_crafting_table`, `craft_planks`, `craft_sticks`, and `craft_pogostick`. At execution time, the agent follows the corresponding executor sequence in the bottom row of Figure 2.5, using α after each executor terminates to confirm that the expected symbolic transition has occurred before continuing.

2.3.4 Novelty in Hybrid Systems

The IPT formalism makes explicit where novelty can manifest and how it propagates through the agent’s decision-making. Because planning and execution are coupled through the detector α and the executor mapping e , novelty can break the IPT at two distinct points.

At the *planning level*, novelty may render the symbolic model incomplete: new fluents, new operators, or changed preconditions and effects may mean that no valid plan exists within the agent’s current model. The IPT is then not plannable. This corresponds to prohibitive novelty (Definition 2.2.2) and requires symbolic model revision—discovering new operators, fluents, or task structure—before execution can proceed.

At the *execution level*, novelty may cause an executor to fail while the symbolic plan remains valid: the environment dynamics have changed such that $e(o)$ no longer achieves ω_o , even though ψ_o is satisfied under α . The IPT remains plannable but is no longer solvable, yielding the kind of problem later formalized as a stretch-IPT (Definition 5.4.3). This corresponds to obstructive novelty (Definition 2.2.3) and manifests as a discrepancy between expected and observed symbolic states after an operator is dispatched for execution.

This distinction is operationally significant. Prohibitive novelties demand changes to Σ or $\mathcal{O}$; obstructive novelties demand changes to the executor mapping e —learning or replacing executors for operators whose symbolic specifications remain valid. Chapter 5 develops a neurosymbolic architecture that addresses both failure modes through a cascading recovery pipeline, and formalizes obstructive failures as an *executor discovery problem* in which the agent learns new executors for failed operators while preserving the existing symbolic plan structure.

In either case, the hybrid decomposition localizes failure. When an executor fails, the agent knows *which* operator was being executed and can compare ω_o against $\alpha(s')$ for the post-execution state s' . This converts what would otherwise be a global performance degradation into a targeted adaptation problem. Without this structure, a failure in the environment would

be an undifferentiated signal with no indication of what specifically went wrong.

This observation motivates the central claim of the dissertation: that structured abstractions over actions and interactions determine whether novelty-induced mismatch can be resolved efficiently and whether learned behaviors can generalize across variations. The next section develops this argument formally.

2.4 Structured Abstractions for Open-World Agents

The preceding sections established the setting (open-world environments), the problem (novelty as mismatch), and the agent’s internal structure (hybrid planning and learning). This section addresses the question that connects them: given that novelty is unavoidable, what determines whether an agent can respond to it effectively?

The central argument of this dissertation is that the answer lies in *structured abstractions*—representations that organize the agent’s knowledge and behavior at appropriate levels of granularity, exposing the compositional structure of tasks and interactions. We develop this argument by first situating our use of abstraction within the broader literature, then examining two specific forms of abstraction that are studied in the subsequent chapters.

2.4.1 Abstraction in Agent Systems

Abstraction is a foundational concept in artificial intelligence and reinforcement learning. At its core, an abstraction is a mapping that reduces the complexity of a representation while preserving structure relevant to a particular purpose. Konidaris [75] argues that the role of abstraction is to capture the complexity of the *task* rather than the complexity of the *agent*: a well-chosen abstraction strips away details of the agent’s embodiment and sensory apparatus, exposing the underlying structure of the problem to be solved. This perspective is central to the approach taken in this dissertation.

In the reinforcement learning literature, two primary forms of abstraction have been studied extensively. *State abstraction* compresses the state space by grouping states that are equivalent with respect to some criterion. Li et al. [83] provide a unifying framework for state abstraction in MDPs, defining several abstraction types based on what properties they preserve (optimal value,

optimal action, model equivalence). Abel [3] extends this theory to characterize the tradeoffs between abstraction granularity and value loss, showing that approximate state abstractions can yield near-optimal behavior while substantially reducing the computational burden of learning and planning.

Action abstraction (or temporal abstraction) extends the agent’s decision-making to include temporally extended actions. The options framework [124] formalizes this through the concept of an option $\omega = \langle \mathcal{I}_\omega, \pi_\omega, \beta_\omega \rangle$, consisting of an initiation set $\mathcal{I}_\omega \subseteq \mathcal{S}$ specifying where the option can be initiated, a policy π_ω that governs behavior during execution, and a termination condition $\beta_\omega : \mathcal{S} \rightarrow [0, 1]$. Options can be incorporated into planning and learning algorithms as temporally extended choices alongside primitive actions, enabling agents to reason and learn at multiple levels of temporal granularity.

A key insight connecting these two forms of abstraction comes from work on grounding symbolic representations in continuous environments. Konidaris et al. [78] show that the symbols necessary and sufficient for planning with a given set of skills can be derived from the initiation sets and effect distributions of those skills. This result establishes a principled link between action abstraction (skills or options) and state abstraction (symbolic predicates): the appropriate symbolic vocabulary for planning is determined by the agent’s available actions, not imposed externally. Konidaris [74] further develops this into the *skill-symbol loop*, in which acquiring new skills gives rise to new symbolic representations, which in turn enable planning at higher levels of abstraction. Each level of the hierarchy abstracts away details of the levels below, exposing the structure of the task at an appropriate granularity.

The IPT formalism introduced in Section 2.3.3 instantiates these ideas. The detector $\alpha : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ maps subsymbolic states to symbolic descriptions, and the executor mapping $e : \mathcal{O} \rightarrow \mathcal{X}$ provides action abstraction: each symbolic operator o abstracts over its executor $e(o)$, representing a potentially complex policy as a single named step with well-defined preconditions and effects. What makes these abstractions *structured*—as opposed to, say, a learned latent representation in an end-to-end model—is that they preserve compositional relationships: operators compose into plans, preconditions and effects support causal reasoning, and the abstraction hierarchy creates natural interfaces at which mismatch can be detected and addressed. This compositional structure is what distinguishes the approach taken in this

dissertation from approaches that learn monolithic mappings from observations to actions [75].

2.4.2 Action Abstractions and Adaptation

The first form of structured abstraction studied in this dissertation concerns the representation of actions. In the IPT formalism (Section 2.3.3), the agent’s behavior is organized as a sequence of operators, each associated with an executor. This decomposition creates a structured relationship between planning and execution that can be exploited when novelty causes failure.

When an executor $e(o)$ fails—because the environment dynamics have changed in a way that the executor’s policy can no longer achieve ω_o —the failure can be localized to a specific point in the plan. The agent knows which operator was being executed, what its intended preconditions and effects were, and what actually occurred, assuming the detector can report whether the expected symbolic effects were achieved. This information converts what would otherwise be a global performance degradation into a targeted learning problem: the agent needs to acquire or revise an executor for a specific operator, with initiation and termination conditions supplied by the symbolic specification.

This targeted formulation has two important consequences. First, it restricts the scope of learning: the agent does not need to treat the entire task as a new learning problem, but can focus exploration on the component whose expected effects failed. This leads to substantially improved sample efficiency compared to unstructured learning approaches in the settings studied in Chapter 5. Second, it preserves previously acquired knowledge: executors for operators that were not affected by the novelty remain valid and can be reused. This connects to the broader theme in the abstraction literature that well-chosen abstractions support knowledge retention and transfer [4, 76].

The effectiveness of this approach depends directly on the quality of the action abstraction. If operators are too coarse, failures are difficult to localize. If operators are too fine, the advantage of symbolic reasoning is lost. The appropriate level of abstraction is one where operators correspond to semantically meaningful units of behavior whose preconditions and effects capture the causally relevant aspects of the interaction—precisely the kind of abstraction that Konidaris et al. [78] show is necessary and sufficient for sound planning. Chapter 5 develops this approach concretely, presenting a hybrid planning–learning method that leverages action

abstractions for efficient adaptation under novelty.

2.4.3 Interaction Abstractions and Generalization

The second form of structured abstraction concerns the representation of interaction between the agent and objects in the environment. While action abstractions organize behavior into compositional sequences, interaction abstractions determine *how* individual actions are represented—specifically, what features of the agent–object interaction are captured by the policy’s state and action spaces.

In robotic manipulation, many robot-learning approaches define policies over robot- or controller-specific representations: joint angles, end-effector poses, or joint torques. These representations can tightly couple the learned behavior to a specific robot embodiment and control interface. The abstraction literature offers a lens for understanding this limitation: a robot-centric representation may fail to abstract away the details of the agent, and therefore may not capture the natural complexity of the task [75]. As a result, a policy trained on one robot often cannot be applied directly to another, even when both robots are performing the same physical task on the same object. Similarly, learned behavior may fail to generalize across objects whose task-relevant physical properties differ from the training instance.

An alternative is to represent interaction at the level of the object rather than the robot. Object-centric force-space representations describe what forces are applied to the object, where on the object they are applied, and what motion the object undergoes as a result. This shift constitutes a structured interaction abstraction: it organizes the policy’s state and action spaces around the physically meaningful invariants of the task—the forces and motions that determine success—rather than the robot-specific details that vary across platforms.

This form of abstraction is analogous to the portable option representations studied by Konidaris and Barto [76], where temporally extended skills are defined to transfer across task instances. Object-centric representations more broadly seek to factor manipulation knowledge away from specific robots and scenes [79, 62]. In our case, the “portability” arises not from learning option policies intended to transfer wholesale, but from choosing a representation space—forces and object geometry—in which the physics of the interaction is invariant across the variations that matter (different objects, different robots, different physical parameters).

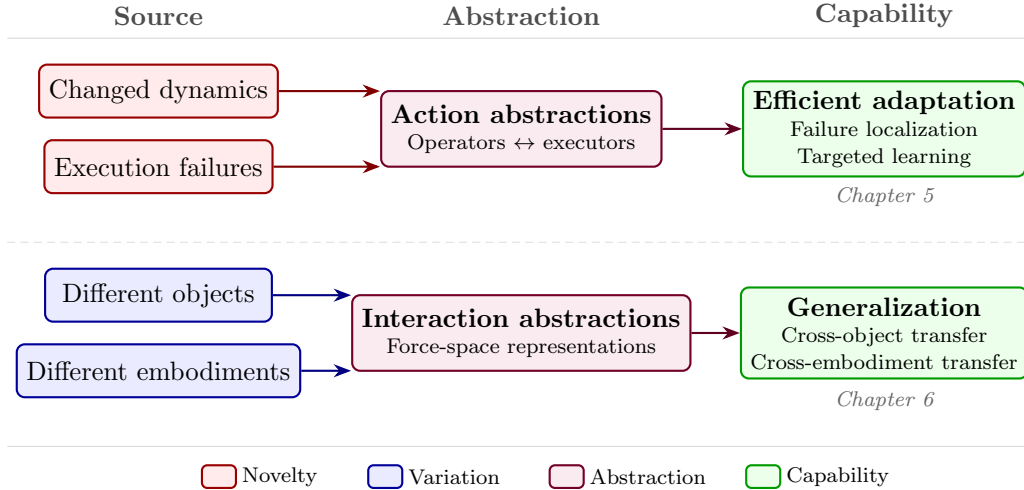


Figure 2.6: The role of structured abstractions in mediating the agent’s response to novelty and variation. **Top:** When novelty causes changed dynamics or execution failures, action abstractions (the operator–executor decomposition) enable failure localization and targeted learning, supporting efficient adaptation (Chapter 5). **Bottom:** When variation arises from different objects or embodiments, interaction abstractions (force-space representations) capture task-relevant physical invariants, supporting generalization without retraining (Chapter 6). In both cases, the abstraction structures the agent’s response rather than requiring the agent to relearn from scratch.

The consequence is a more principled route to generalization. A policy learned in force space over one object is better positioned to generalize to other objects that share similar physical semantics, because the representation captures aspects of the interaction that are invariant across objects rather than those specific to a single training instance. Similarly, a policy learned on one robot can be executed on a different robot when the new robot can realize the required forces at the required contact points. Chapter 6 develops this approach, presenting force-based learning methods for robotic manipulation that generalize across objects, physical parameters, and robotic embodiments.

2.4.4 The Unifying Perspective

Both forms of abstraction share a common principle, illustrated in Figure 2.6: they organize the agent’s representation around aspects of the task and interaction that are causally relevant, rather than aspects that are contingent on a particular environment instance or robot embodiment.

Action abstractions organize behavior into operators with explicit preconditions and effects,

so that when novelty disrupts one operator, the disruption is contained and the agent can focus learning on the affected component. Interaction abstractions organize execution around physical invariants—forces, contact points, object-relative motion, and object dynamics—so that when the specific robot or object changes, the policy’s representation can still capture the structure of the task.

In both cases, the abstraction serves as a buffer between the agent and the variability of the world. It does not eliminate novelty or variation—that would require closed-world assumptions—but it structures the agent’s response to change. In Chapter 5, this structure supports efficient recovery after an execution failure. In Chapter 6, it supports transfer across anticipated variation before failure occurs. The empirical validation of this perspective is the subject of the remaining chapters.

2.5 Summary

This chapter has established the formal and conceptual foundations for the remainder of the dissertation. We defined open-world environments as settings in which an agent’s knowledge is incomplete with respect to the true state of the world, and formalized novelty as a mismatch between the agent’s internal models and its environment. The taxonomy of novelty types—prohibitive, obstructive, beneficial, and nuisance—provides a principled vocabulary for reasoning about how novelty affects task performance, while the classification by domain element—object, attribute, and action novelty—characterizes what aspects of the environment have changed.

We then developed the agent framework, integrating symbolic planning (Section 2.3.1), reinforcement learning (Section 2.3.2), and the integrated planning task (IPT) formalism (Section 2.3.3) that couples them through a detector $\alpha : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ and an executor mapping $e : \mathcal{O} \rightarrow \mathcal{X}$. The framework distinguishes planning success (plannability) from execution success (solvability), making explicit where novelty can manifest: at the symbolic level through incomplete models, or at the execution level through failed executors while symbolic plans remain valid.

Finally, we introduced the concept of structured abstractions and argued that the choice of abstraction determines whether novelty-induced mismatch leads to brittle failure or can be

managed through efficient adaptation and generalizable behavior. Action abstractions expose the compositional structure of plans, enabling failure localization and targeted learning. Interaction abstractions capture the physically meaningful invariants of manipulation tasks, enabling transfer across objects and embodiments.

The subsequent chapters instantiate this framework. Chapter 3 situates the dissertation within the broader literature on open-world planning, learning, execution recovery, and agents built with foundation models. Chapter 4 introduces benchmark environments and evaluation methodologies that operationalize the novelty taxonomy through controlled injection of object, attribute, and action novelties, and provide metrics for measuring adaptation and recovery. Chapter 5 develops hybrid planning–learning methods that exploit action abstractions to achieve efficient adaptation under novelty. Chapter 6 presents force-based learning approaches that use interaction abstractions to enable generalizable robotic manipulation. In each case, chapter-specific related work is discussed in the context of the particular contribution.

Chapter 3

Related Work and Perspectives on Open-World Autonomy

This chapter situates the contributions of this dissertation within the broader landscape of open-world autonomy research. The subsequent technical chapters develop three contributions toward enabling agents and robots to operate in open-world environments: benchmark environments for systematically evaluating agents for open worlds (Chapter 4), action and state abstractions that enable efficient adaptation when execution fails (Chapter 5), and interaction abstractions that support generalization across objects, physical parameters, and robotic embodiments (Chapter 6). Each of those chapters includes focused discussions of related work specific to its technical contribution.

This chapter presents an overview of the current state of research in open-world autonomy and provides a perspective on the future directions of this field.

3.1 Open-World Sensing and Perception

In closed-world settings, perception systems are designed to recognize a fixed set of categories, objects, or states defined at training time. Open-world perception, by contrast, requires the ability to detect and reason about entities and situations that were not anticipated during system design. Early work on open set recognition [113, 11] extended standard classifiers to reject inputs that do not belong to any known category, typically through confidence thresholding or

distribution boundary modeling. Out-of-distribution detection methods [55] further developed principled approaches for identifying inputs that fall outside the training distribution.

More recently, vision-language models (VLMs) have transformed the landscape of open-world perception. CLIP [108] demonstrated that joint vision-language pretraining enables zero-shot recognition of arbitrary categories described in natural language, fundamentally relaxing the closed-vocabulary constraint. This capability has been extended to object detection through open-vocabulary detectors such as ViLD [49] and OWL-ViT [95], which can localize objects described by free-form text queries without category-specific training. Foundation models for segmentation, such as the Segment Anything Model [70], further demonstrate that general-purpose visual representations can support open-ended perception tasks.

These advances substantially reduce the perceptual barriers to open-world operation. An agent equipped with an open-vocabulary detector can, in principle, recognize and localize objects it has never seen during training, which addresses a long-standing bottleneck in the perceive–plan–execute loop. However, a critical gap remains between perceptual novelty detection and the needs of an acting agent. Recognizing that an object is unfamiliar or that a scene has changed is necessary but not sufficient: the agent must also determine whether the novelty is functionally relevant to its current task and how it should modify its behavior in response. In the terminology of Section 2.2.2, the perception system must not only detect that a mismatch has occurred but also assess whether it constitutes a prohibitive, obstructive, beneficial, or nuisance novelty.

Current perception systems largely operate as passive classifiers and do not reason about the implications of detected novelty for downstream planning and execution. This dissertation does not address the perception problem directly; the contributions in Chapters 4 to 6 assume that the agent has access to task-relevant state information. Closing the loop between open-world perception and the adaptation and generalization mechanisms developed in this dissertation remains an important direction for future work, and is discussed further in Section 3.5.

3.2 Open-World Planning and Learning

Planning and learning provide the reasoning and decision-making capabilities that allow agents to select actions in pursuit of goals. This section reviews the major paradigms and identifies the limitations that the contributions of this dissertation address.

Classical symbolic planning. Classical symbolic planning, formalized through STRIPS [28] and PDDL [94], enables structured long-horizon reasoning by searching over sequences of operators with well-defined preconditions and effects. Modern planners such as Fast Downward [54] achieve strong performance across diverse domains through domain-independent heuristics. However, symbolic planning assumes that the domain model is complete and correct; when the model is inaccurate or missing operators, generated plans may fail during execution. This brittleness under model mismatch is precisely the problem addressed by the hybrid planning–learning framework in Chapter 5, where symbolic plans provide structure for targeted adaptation rather than being treated as infallible specifications.

Reinforcement learning. Reinforcement learning offers a complementary approach, enabling agents to learn effective behaviors through interaction without requiring explicit models. Deep RL algorithms, including DQN [96], PPO [117], and SAC [52], have demonstrated success in complex control tasks. Model-based approaches, including world models such as Dreamer [53] and planning-based methods such as MuZero [116], learn environment dynamics and use them for decision-making. These methods reduce the need for hand-specified models but typically assume that the learned dynamics remain stationary—an assumption that open-world settings violate. The adaptation methods in Chapter 5 and the generalization methods in Chapter 6 both build on reinforcement learning but address different aspects of this stationarity limitation: the former enables targeted relearning when dynamics change, while the latter produces representations that are invariant to certain classes of variation from the outset.

Task and motion planning. Task and motion planning (TAMP) systems [34, 65] integrate symbolic task planning with continuous motion planning, enabling agents to reason jointly about high-level task structure and low-level geometric feasibility. These systems represent

a sophisticated form of the hybrid planning–learning framework, but they typically assume complete and correct domain models at both levels. When the environment changes in ways that invalidate either the task-level operators or the motion-level feasibility assumptions, TAMP systems lack mechanisms for acquiring new behaviors. The approach in Chapter 5 can be viewed as extending the hybrid paradigm with the capacity for targeted learning when model assumptions fail.

Transfer and continual learning. When an agent encounters a changed environment, transfer learning and continual learning provide mechanisms for leveraging prior experience. Policy reuse [40], progressive neural networks [110], and elastic weight consolidation [71] address different aspects of this problem. However, these methods treat adaptation as a monolithic learning problem over the full state-action space: they do not exploit the compositional structure of tasks to focus learning on the specific component that failed. As the experimental results in Chapter 5 demonstrate, this global approach is substantially less sample-efficient than the targeted adaptation enabled by action abstractions when novelty affects only a specific component of the task.

Open-world novelty handling. The DARPA SAIL-ON program [118] catalyzed research specifically targeting agents that must detect, characterize, and accommodate novelty. This program motivated formal definitions of open-world novelty [14, 81] and agent architectures for novelty handling [99, 43]. Several approaches explored how agents can adapt their policies or models in response to detected novelty [9, 8], and neuro-symbolic world models have been proposed for updating internal representations under novelty. A related perspective is provided by creative problem solving (CPS) research, which frames novelty accommodation as a form of creative reasoning. Gizzi et al. [39] survey the CPS landscape in AI, identifying action discovery, analogical reasoning, and conceptual blending as complementary strategies for resolving novel situations. Nair et al. [100] examine the extent to which large language and vision models exhibit creative problem-solving capabilities, finding that while these models demonstrate some transfer, they lack the structured adaptation mechanisms needed for reliable novelty resolution. The benchmarks in Chapter 4 and the adaptation framework in Chapter 5 build directly on

this line of work, providing both the evaluation infrastructure and the algorithmic machinery for systematic novelty handling.

Connection to this dissertation. Across these paradigms, a recurring limitation is the difficulty of reconciling structured reasoning (which provides compositional decomposition but assumes correct models) with learned behavior (which can acquire new capabilities but lacks structural guidance for efficient adaptation). The contributions of this dissertation address this gap through structured abstractions: action abstractions that bridge symbolic failure diagnosis with targeted learning (Chapter 5), and interaction abstractions that organize learned behavior around task-relevant invariants rather than embodiment-specific details (Chapter 6). The benchmarks in Chapter 4 provide the controlled evaluation infrastructure needed to measure progress on these problems.

3.3 Open-World Execution and Recovery

Even when perception and planning succeed, action execution in open-world environments may fail due to unexpected dynamics, novel object properties, or environmental changes. Recovery from such failures requires mechanisms that go beyond standard error handling: the agent must identify why the failure occurred and adapt its behavior accordingly.

Plan monitoring and repair. Classical approaches to execution-time adaptation include plan monitoring and repair [29, 30], which detect deviations from expected execution outcomes and revise plans within the existing domain model. However, plan repair assumes that the domain model remains valid and that a correct plan exists within the current model, which may not hold when qualitative novelty has occurred. The cascading recovery pipeline in Chapter 5 extends this idea: it attempts lightweight symbolic recovery first (analogous to plan repair) and escalates to learning-based executor discovery only when symbolic methods are insufficient. This cascade reflects the insight that not all failures require new learning—some can be resolved through replanning within the existing model.

Sim-to-real transfer and domain adaptation. Sim-to-real transfer techniques, including domain randomization [132] and system identification [107], improve policy robustness by training across varied conditions, but the variations are typically parameterized in advance and may not cover the kinds of qualitative changes encountered in open-world settings. The interaction abstractions developed in Chapter 6 adopt a complementary strategy: rather than randomizing over the robot’s parameters to achieve robustness, they remove the robot from the learning problem entirely by operating in object-centric force space. This means that sim-to-real transfer requires only a mapping from learned forces to robot commands, rather than bridging a full embodiment gap.

Continual and lifelong learning. Continual learning methods [71, 110] address the problem of acquiring new capabilities without forgetting previous ones, but most assume known task boundaries and distributions. In the open-world setting, the agent does not know in advance when novelty will occur or what form it will take. The novelty detection and characterization mechanisms studied in Chapter 4 and Chapter 5 address this gap by providing the agent with the ability to detect mismatch between expected and observed execution outcomes, triggering adaptation only when necessary.

Integrated approaches. Some efforts have sought to address multiple stages of the open-world agent loop in an integrated manner. World models [51, 53] offer a potential unifying framework by learning environment dynamics that can serve both perception and planning, though they typically do not handle discrete, qualitative changes in the environment. Task and motion planning systems [34, 65] integrate symbolic and geometric reasoning but inherit the brittle model assumptions of classical planning. Despite growing interest in open-world autonomy, the field remains largely fragmented: methods that address perception under novelty, planning under incomplete models, and recovery from execution failures are typically developed in isolation from one another. The framework developed in this dissertation takes a step toward integration by connecting symbolic planning (for task decomposition and failure localization) with reinforcement learning (for behavior acquisition) through structured abstractions that mediate between the two levels.

3.4 Foundation Models and Open-World Autonomy

The rapid development of large-scale foundation models has fundamentally altered the landscape of autonomous agent design. This section examines how these models relate to the open-world challenges addressed in this dissertation, and argues that the abstraction-based perspective developed here provides a complementary—rather than competing—viewpoint.

3.4.1 Vision-Language-Action Models

A recent and rapidly growing line of work leverages large pretrained models for robotic planning and decision-making. Vision-language-action (VLA) models such as RT-2 [146] directly map visual observations and language instructions to robot actions, drawing on the broad world knowledge encoded in large language and vision models. By training on large-scale robot demonstration datasets, these models can generalize to novel instructions and objects that were not explicitly present in the training data, exhibiting a form of open-world capability that emerges from the scale and diversity of pretraining.

Generalist robot policies such as Octo [128] and π_0 [13] extend this paradigm by training on multi-robot, multi-task datasets with the goal of producing a single policy that works across embodiments and task families. These models represent an implicit approach to the cross-embodiment generalization problem that Chapter 6 addresses through explicit interaction abstractions: rather than designing representations that factor out embodiment-specific details, they attempt to learn embodiment-invariant behavior from sufficiently diverse data.

3.4.2 Language Models as Planners

A parallel line of work uses large language models (LLMs) as high-level planners or task decomposers. SayCan [6] grounds LLM-generated plans in an affordance model that scores the feasibility of each proposed step given the robot’s current capabilities. Code as Policies [84] and related approaches [60] use LLMs to generate executable code or action sequences from natural language instructions. These systems decompose tasks into structured sequences of skills, performing a function analogous to the symbolic planning layer in the hybrid framework of Section 2.3.3—but using statistical language generation rather than formal search over a

domain model.

The appeal of LLM-based planning is its flexibility: the model can propose actions and decompositions for tasks it has never been explicitly trained on, drawing on commonsense knowledge acquired during language pretraining. However, this flexibility comes with limitations that are directly relevant to the open-world setting. LLM-generated plans are not grounded in formal preconditions and effects, making it difficult to verify plan correctness, detect when execution deviates from expectations, or localize the source of a failure. The structured action abstractions developed in Chapter 5—where each operator has explicit preconditions, effects, and a corresponding executor—provide precisely this kind of diagnostic capability, enabling the agent to convert a global failure into a targeted learning problem.

3.4.3 What Foundation Models Solve and What They Do Not

Foundation models bring unprecedented breadth to autonomous systems. Their ability to recognize novel objects, generate plausible task decompositions, and produce behaviors for previously unseen instructions addresses aspects of the open-world problem that were intractable for earlier methods. In the language of Section 2.2, foundation models substantially expand the agent’s prior knowledge $\mathcal{K}$, reducing the set of situations that constitute novelty.

However, several fundamental challenges remain.

Diagnosability and failure localization. When a foundation model-based agent fails—because the VLA produces an incorrect action, or the LLM proposes an infeasible step—the monolithic nature of the model makes it difficult to determine *why* the failure occurred and *what* should change. The failure could stem from incorrect perception, faulty reasoning, an inadequate skill, or a combination. In contrast, the structured decomposition used in this dissertation—symbolic operators with explicit preconditions and effects, each associated with a separate executor—provides natural interfaces at which mismatch can be detected, localized, and addressed. This diagnostic capability is essential for efficient adaptation: without it, the agent’s only recourse is to collect more data and retrain, which is neither sample-efficient nor targeted.

Physical grounding in contact-rich tasks. Foundation models trained primarily on vision and language data encode rich semantic knowledge but have limited understanding of contact physics, force interactions, and the dynamics of sustained manipulation. Tasks such as operating articulated mechanisms under varying resistance or pushing heavy objects under unknown friction—the settings studied in Chapter 6—require policies that reason about forces, contact states, and physical parameters that are not well captured by vision-language representations alone. The object-centric force-space representations developed in this dissertation address precisely this gap: they capture the physically meaningful invariants of manipulation tasks at a level of abstraction that vision-language models do not operate at.

Reliability and verifiability. In safety-critical deployment settings, the stochastic nature of foundation model outputs poses challenges. LLM-generated plans may hallucinate infeasible actions or produce subtly incorrect decompositions. VLA policies may exhibit unpredictable failures on out-of-distribution inputs. Formal planning systems, despite their brittleness under model mismatch, offer guarantees about plan validity given correct models. The hybrid approach in this dissertation inherits some of these guarantees: when the symbolic model is correct, plans are provably valid; when it is not, the agent has structured mechanisms for detecting and addressing the mismatch.

Sample efficiency in adaptation. When a foundation model encounters a genuinely novel situation—one that falls outside its training distribution—adaptation typically requires collecting new demonstrations or fine-tuning on task-specific data, which can be expensive in real-world settings. The targeted learning approach in Chapter 5 demonstrates that structured abstractions can reduce the adaptation problem from a global policy search to a focused learning problem over a single operator’s executor, achieving one to two orders of magnitude improvement in sample efficiency over unstructured approaches. This efficiency advantage is particularly relevant in physical settings where data collection is costly and time-consuming.

3.4.4 Complementary Perspectives

The abstraction-based approach developed in this dissertation and the foundation model paradigm are not mutually exclusive. Several natural points of integration exist.

Foundation models could serve as powerful initialization for the components of a structured agent architecture. An LLM could provide initial domain models that are then formalized into PDDL operators, with the hybrid framework providing the machinery for detecting when these models are incorrect and adapting accordingly. A VLA could provide initial executors that are refined through the targeted learning mechanisms of Chapter 5 when they fail in novel situations. Open-vocabulary perception models could provide the perceptual front-end that this dissertation assumes but does not develop.

Conversely, the structured abstractions developed here could provide the interpretability and diagnostic capabilities that foundation model-based systems currently lack. Organizing a foundation model’s behavior through explicit operator–executor decompositions would enable failure localization and targeted adaptation even when the underlying execution is performed by a large pretrained model. Force-space representations could ground VLA policies in physical interaction dynamics, providing a level of physical reasoning that complements the semantic reasoning these models excel at.

This complementary view suggests that the future of open-world autonomy may lie not in choosing between structured abstractions and large-scale learning, but in developing architectures that combine the breadth and flexibility of foundation models with the diagnosability, efficiency, and physical grounding that structured representations provide.

3.5 Toward Integrated Open-World Agents

The preceding sections reviewed the state of the field across perception, planning, learning, execution, and the emerging role of foundation models. A recurring theme is fragmentation: methods that address different aspects of the open-world problem are typically developed and evaluated in isolation from one another. Table 3.1 summarizes this landscape by mapping the major classes of approaches against the novelty taxonomy developed in Section 2.2.2, indicating which types of novelty each paradigm can detect and address.

Table 3.1: Coverage of major approach families across the novelty taxonomy of Section 2.2.2. **D** = can detect this novelty type; **A** = can adapt to or accommodate it; **–** = not addressed. The final two rows correspond to the contributions of this dissertation.

Approach	Prohibitive	Obstructive	Beneficial	Nuisance
Open-set perception	D	D	D	D
Classical planning	D	–	–	–
Standard RL	–	A	A	–
LLM-as-planner	D	–	–	–
VLA / generalist policy	–	A	–	–
Action abstractions (Chapter 5)	D, A	D, A	–	D
Interaction abstractions (Chapter 6)	–	A	–	–

Several patterns emerge from this mapping. Open-set perception systems detect all novelty types but provide no adaptation mechanism. Classical planning detects prohibitive novelties through plan failure but cannot modify its models. Standard RL can adapt to obstructive and beneficial novelties through continued interaction but requires extensive exploration and cannot detect novelty explicitly. Foundation model-based approaches (LLM planners, VLA policies) bring broad coverage but limited diagnosability. The contributions of this dissertation fill specific cells: action abstractions provide both detection and adaptation for prohibitive and obstructive novelties through structured failure localization, while interaction abstractions preempt obstructive novelties arising from object and embodiment variation through invariant representations. The table also makes visible the remaining gap: beneficial novelties are not exploited by any approach that relies on failure-driven adaptation, confirming the limitation noted in Chapters 4 and 5.

This section briefly discusses how the contributions of this dissertation relate to the broader goal of integrated open-world agents.

The three contributions of this dissertation—benchmark environments (Chapter 4), action abstractions for adaptation (Chapter 5), and interaction abstractions for generalization (Chapter 6)—address complementary aspects of the open-world problem. The benchmarks provide controlled settings for evaluating agents under diverse classes of novelty, measuring both the dynamics of performance degradation and the efficiency of recovery. The action abstractions provide the mechanism for efficient adaptation when the agent’s symbolic model becomes incorrect, converting global failures into targeted learning problems. The interaction abstractions

provide representations that support generalization across objects and embodiments, reducing the scope of what must be relearned when conditions change.

These contributions operate at different levels of the agent architecture, and their integration represents a natural next step. As discussed in the summary of Chapter 6, the force-space policies developed as interaction abstractions could serve as executors within the hybrid planning–learning framework that leverages action abstractions. In such an integrated system, the planning layer would provide task decomposition and failure localization; the interaction abstractions would provide generalizable execution; and when novelty renders an executor ineffective, the action abstraction framework would focus learning on acquiring a replacement. The benchmarks would provide the evaluation infrastructure for measuring whether such an integrated system achieves the adaptation and generalization capabilities that neither component provides alone.

Realizing this integration fully requires addressing the perception gap identified in Section 3.1: the current contributions assume access to task-relevant state information, and connecting open-world perception to the downstream reasoning and execution pipeline remains an important open challenge. It also requires scaling the symbolic planning components to richer domains, and developing the formal connections between the novelty taxonomy of Section 2.2.2 and the adaptation mechanisms to enable more principled selection of recovery strategies based on novelty type. These directions are discussed further in Chapter 7.

3.6 Summary

This chapter situated the contributions of this dissertation within the broader landscape of open-world autonomy research. We reviewed prior work across the stages of the perceive–plan–execute loop—sensing and perception, planning and learning, and execution and recovery—and identified the recurring challenges that motivate the structured abstraction approach developed in this dissertation.

The discussion of foundation models highlighted that while large-scale pretraining substantially expands the range of situations an agent can handle, fundamental challenges remain in diagnosability, physical grounding, reliability, and sample-efficient adaptation. The abstraction-

based perspective developed in this dissertation addresses these challenges through complementary mechanisms: structured action abstractions that enable failure localization and targeted learning, and interaction abstractions that capture task-relevant physical invariants. These approaches are not alternatives to foundation models but potential complements, and their integration represents a promising direction for building agents that combine the breadth of large-scale learning with the efficiency and interpretability of structured representations.

The next and final chapter summarizes the main contributions of this dissertation and discusses future directions for research on autonomous agents and robots operating in open-world environments.

Chapter 4

Benchmark Environments for Open-World Novelty¹

4.1 Motivation

Chapter 2 defined open-world environments as settings in which an agent’s internal models are incomplete with respect to the environment, and formalized novelty as a mismatch between those models and observed outcomes. In such settings, the central question is not only how agents should adapt, but how their ability to detect, localize, and recover from mismatch can be evaluated in a principled and systematic manner.

Evaluating agents under mismatch presents challenges that do not arise in closed-world settings. In standard benchmarks, the environment is fixed and fully specified, and performance can be measured with respect to a known objective. In contrast, open-world environments are characterized by the possibility of unanticipated changes: objects may appear or disappear, action effects may change, and task structure may evolve. These changes induce mismatch between the agent’s expectations and the environment, but the space of possible mismatches is unbounded. As a result, evaluation cannot rely on a fixed set of scenarios, but must instead

¹This chapter is based on: S. Goel, G. Tatiya, M. Scheutz, and J. Sinapov, “NovelGridworlds: A Benchmark Environment for Detecting and Adapting to Novelities in Open Worlds,” in *AAMAS Workshop on Adaptive Learning Agents (ALA)*, 2021; and S. Goel, Y. Wei, P. Lymperopoulos, K. Churá, M. Scheutz, and J. Sinapov, “NovelGym: A Flexible Ecosystem for Hybrid Planning and Learning Agents Designed for Open Worlds,” in *Proc. of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 688–696, 2024.

provide mechanisms for generating controlled and interpretable variations in the environment.

The crafting scenario introduced in Chapter 2 illustrates this challenge. When the agent encounters the axe novelty—where trees become unbreakable without a new tool—its existing plan fails due to a mismatch between the expected and actual effects of the `break` operator. Evaluating the agent’s behavior in this setting requires more than measuring task success: it requires quantifying how quickly the agent detects the mismatch, how severely its performance degrades, and how efficiently it recovers. More generally, evaluation must capture the dynamics of adaptation, not just its final outcome.

Moreover, evaluating the central claim of this dissertation—that structured abstractions over actions and interactions determine whether agents can adapt efficiently under novelty (Section 2.4)—requires benchmarks that can distinguish between agents with different representational commitments. An environment that only measures final task success cannot reveal whether a hybrid agent’s advantage over a monolithic learner stems from its structured decomposition of behavior or from some other factor. The benchmarks introduced in this chapter are designed to expose these differences through metrics that capture adaptation speed, performance degradation, and the ability to exploit beneficial changes.

To evaluate open-world agents along these dimensions, we require environments that can systematically introduce mismatch, expose its effects at both the planning and execution levels, and support agents with different representational and decision-making paradigms. This chapter presents two such environments. NovelGridworlds introduces a structured taxonomy of novelties and evaluation metrics that capture performance degradation and detection. NovelGym extends this framework with a formal environment representation, composable transformations that induce mismatch, and metrics that explicitly characterize adaptation dynamics. Together, these environments provide the experimental foundation for studying adaptation and generalization under novelty in the remainder of this dissertation.

4.2 Related Work

This section reviews prior work on evaluation environments and benchmarks relevant to the study of agents under open-world novelty. The discussion is organized around four themes:

standard reinforcement learning and planning benchmarks that operate under closed-world assumptions, environments that introduce variability or non-stationarity, frameworks specifically designed for open-world novelty evaluation, and evaluation protocols and metrics for measuring agent adaptation.

Closed-world evaluation benchmarks. The dominant paradigm for evaluating sequential decision-making agents assumes fixed, fully specified environments. Reinforcement learning benchmarks such as OpenAI Gym [15] and the Arcade Learning Environment [10] provide standardized tasks with known dynamics, enabling comparison of learning algorithms under controlled conditions. Offline RL benchmarks such as D4RL [31] extend this paradigm to data-driven evaluation but retain the assumption of a stationary environment. Similarly, classical planning benchmarks expressed in PDDL [94] assume that the domain model is complete and correct, and evaluate planners on plan quality and computational efficiency. PDDLGym [119] bridges the gap between planning and learning by providing Gym-compatible environments generated from PDDL domain files, enabling evaluation of agents that combine symbolic and learned components. Multi-agent evaluation has also been supported by platforms such as PettingZoo [129], which provides a standardized interface for multi-agent reinforcement learning. These environments have been productive for advancing algorithms within their respective paradigms, but they do not support evaluation of how agents respond to changes that violate their learned or specified models—precisely the setting formalized in Section 2.1.

Generalization and domain transfer. A separate line of work addresses generalization across environments through domain randomization [132, 107] and procedurally generated environments. Benchmarks such as those developed by Nichol et al. [104] evaluate transfer learning and few-shot learning in RL through procedurally generated levels that test whether agents generalize beyond their training configurations. The WordCraft environment [63] incorporates natural language corpora to develop commonsense agents that can reason about novel situations. These approaches expose agents to a distribution of tasks during training, with the goal of improving robustness to variation at deployment. While effective for improving generalization within the training distribution, they do not explicitly evaluate how agents respond

to post-training changes that introduce mismatch between the agent’s models and the environment. The variability is incorporated into the training distribution rather than introduced as a structured, diagnosable deviation, and consequently these approaches do not exercise the adaptation capabilities that are central to open-world operation.

Out-of-distribution detection and novelty detection. Research on out-of-distribution (OOD) detection addresses the problem of identifying when an input differs from the training distribution. Methods such as confidence-based detection [55], open-set recognition [113, 11], and autoencoder-based anomaly detection [1] have been studied extensively in perception and classification settings, where novelty is defined in terms of input statistics. More recent approaches include vision transformer-based methods [82] and classifier-based novelty scoring [17], which improve detection accuracy but remain focused on perceptual novelty. However, these methods focus on detection rather than adaptation: they identify that something has changed but do not provide mechanisms for the agent to modify its behavior in response. In the framework of Section 2.2, detection addresses only one aspect of the agent’s response to mismatch; the benchmarks introduced in this chapter must additionally support evaluation of how agents recover from and adapt to detected novelties.

Open-world novelty frameworks. The DARPA SAIL-ON program [118] catalyzed research on agents operating in environments with unanticipated novelty, motivating formal definitions of open-world novelty [14, 81] and agent architectures for novelty handling [99, 43]. Within this broader effort, several evaluation environments have been developed. NovGrid [9] provides a flexible gridworld platform for studying novelty, built on MiniGrid, supporting configurable environment parameters and novelty injection. The Polycraft World AI Lab [46] offers a richer, Minecraft-based environment for evaluating novelty detection and accommodation at scale, supporting multi-agent scenarios with cooperative and adversarial actors. NovelCraft [26] provides a dataset and benchmark for visual novelty detection and discovery in open worlds.

Model-based approaches to novelty adaptation have also been evaluated in dedicated environments. Klenk et al. [72] present HYDRA, a model-based novelty adaptation framework that uses a domain-independent planner and learns environment models that can be updated when

novelties are introduced. Stern et al. [121] extend this approach with a domain-independent planning framework evaluated in the Angry Birds domain. The Tonic library [105] provides a configurable benchmarking framework for deep RL but is limited to closed-world settings and does not support novelty injection.

These environments represent important steps toward open-world evaluation, but they address different aspects of the problem than the benchmarks introduced in this chapter. NovGrid focuses on environment configurability but does not provide a formal taxonomy that connects novelty types to their impact on agent task performance, as developed in Section 2.2.2. The Polycraft environment supports large-scale evaluation but is tightly coupled to a specific domain and agent interface, making it difficult to evaluate agents of different architectural paradigms under identical conditions. NovelCraft targets visual novelty detection rather than the full cycle of detection, adaptation, and recovery. None of these platforms provide a unified interface that seamlessly supports symbolic planning agents, reinforcement learning agents, and hybrid neurosymbolic architectures within the same environment.

Gap addressed by this chapter. The benchmarks introduced in this chapter—NovelGridworlds and NovelGym—address gaps that cut across these lines of work. They provide environments that support controlled injection of diverse classes of mismatch, organized according to the taxonomy of novelty by domain element and task impact developed in Section 2.2. They accommodate agents across multiple paradigms—symbolic planning, reinforcement learning, and hybrid architectures—through flexible observation interfaces, including LiDAR-like sensors, image-based local views, and symbolic PDDL representations. The NovelGym ecosystem further supports multi-agent scenarios through its PettingZoo-based architecture [129] and provides modular novelty injection through composable environment transformations. Both environments introduce evaluation metrics that capture not only task success but the dynamics of adaptation: how quickly performance degrades upon novelty injection, how efficiently the agent recovers, and whether beneficial novelties are exploited. These capabilities are essential for evaluating the claims developed in Section 2.4: that structured abstractions over actions and interactions determine whether agents can adapt efficiently and generalize effectively under novelty.

4.3 NovelGridworlds

NovelGridworlds² is an OpenAI Gym-based benchmark environment designed for developing and evaluating AI agents that can detect and adapt to novelties in open worlds. The environment provides four crafting tasks of increasing complexity, a taxonomy of twelve novelties per task, cross-platform support through a socket interface, and evaluation metrics for both planning and reinforcement learning agents.

4.3.1 Environment Design

Each NovelGridworlds environment is a discrete gridworld of configurable size. The arena is enclosed by unbreakable walls and contains a crafting table, an agent, free space (air blocks), and task-specific objects. At each time step, the agent selects from a discrete set of actions and maintains an inventory of collected items. The environment returns an action cost for each action taken, enabling plan-cost-based evaluation. All environments share a common set of primitive actions: `move-forward`, `turn-left`, `turn-right`, and `break`. Task-specific actions extend this set to support crafting and resource extraction.

Four tasks are implemented, each requiring the agent to craft a target item by collecting resources and applying recipes in the correct sequence:

1. **NovelGridworld-Bow-v0**: The agent collects three sticks and three strings from the environment and crafts a bow at the crafting table.
2. **NovelGridworld-Bow-v1**: An advanced version of the previous task in which the agent must collect raw materials (tree logs and wool) and craft intermediary items (planks, sticks, and strings) before crafting the bow.
3. **NovelGridworld-Pogostick-v0**: The agent collects sticks, planks, and rubber (extracted using a tree tap) to craft a pogostick.
4. **NovelGridworld-Pogostick-v1**: An advanced version in which the agent must craft all

²The open-source NovelGridworlds repository is available at <https://github.com/gtatiya/gym-novel-gridworlds>. Additional implementation details, including environment wrappers, observation space configurations, and trajectory recording utilities, are described in Goel et al. [41].

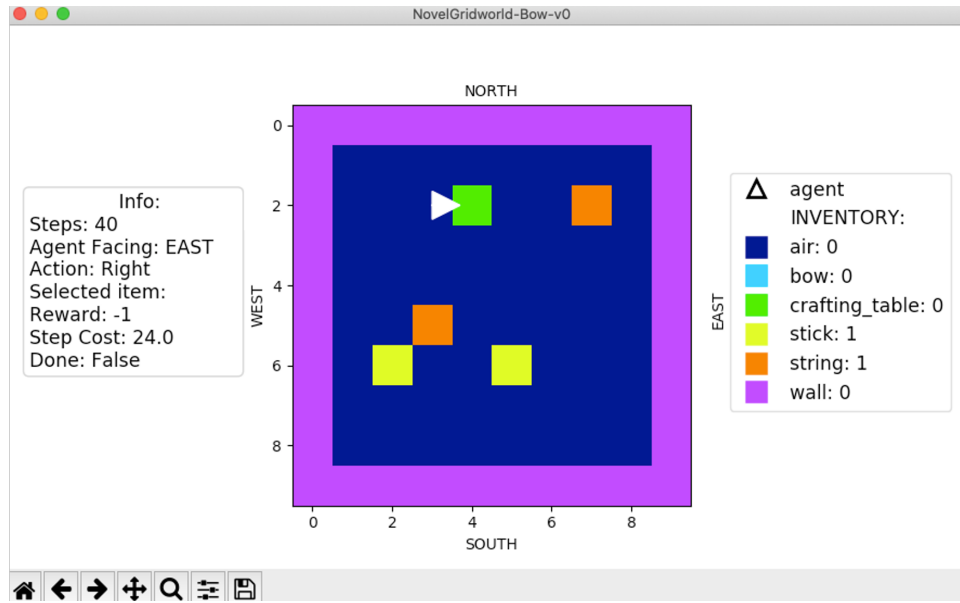


Figure 4.1: Screenshot of the NovelGridworld-Bow-v0 environment. The agent (triangle) faces the crafting table (green). Sticks (yellow) and strings (orange) are distributed in the gridworld. Blue spaces represent air blocks through which the agent can navigate, and purple blocks represent walls.

intermediary items, including the tree tap itself, from raw tree logs before crafting the pogo-stick.

Table 4.1 summarizes the task-specific actions and block types for each environment.

The environment communicates with agents through a socket interface, enabling agents implemented in different programming languages to interact with the same environment. This design supports both planning agents implemented in Java-based architectures and reinforcement learning agents implemented in Python.

4.3.2 Novelty Taxonomy

A central contribution of NovelGridworlds is a structured taxonomy for classifying novelties along two dimensions. The first dimension categorizes novelties by which component of the agent’s model they affect. The second dimension characterizes the impact of the novelty on task performance.

Novelty classes. Using the formalism from Sections 2.3.1 and 2.3.2, novelties are classified into three classes based on the component they alter:

Environment	Actions	Blocks
Default	move-forward, turn-left, turn-right, break, select-⟨block⟩	air, wall, crafting-table
Bow-v0	craft-bow	string, stick
Bow-v1	craft-bow, extract-string, craft-plank, craft-stick	tree-log, wool, string, stick
Pogostick-v0	extract-rubber, craft-pogostick	tree-log, tree-tap, stick, plank
Pogostick-v1	extract-rubber, craft-plank, craft-stick, craft-tree-tap, craft-pogostick	tree-log, tree-tap, stick, plank

Table 4.1: Actions and blocks specific to each NovelGridworlds task. “Default” denotes the actions and blocks common to all environments; each task adds the listed actions and blocks to the default set.

- **Object novelty:** A previously unknown entity appears in the environment, altering the observation space. The resulting state $s' \in \mathcal{S}$ does not come from the distribution of states the agent has previously encountered.
- **Attribute novelty:** The properties of existing entities change, modifying the transition dynamics. Formally, the transition function changes such that for some known state-action pair (s, a) , the resulting state $P(\cdot \mid s, a)$ differs from what the agent has previously observed.
- **Action novelty:** The behavior of actions changes. An action that previously produced a known effect now produces a different outcome, or new actions become available.

Novelty types. Each novelty is further classified by its impact on the agent’s task performance:

- **Beneficial:** The novelty helps the agent complete the task more efficiently.
- **Detrimental:** The novelty hinders or prevents task completion unless the agent adapts.
- **Irrelevant:** The novelty does not affect the task and can be safely ignored.

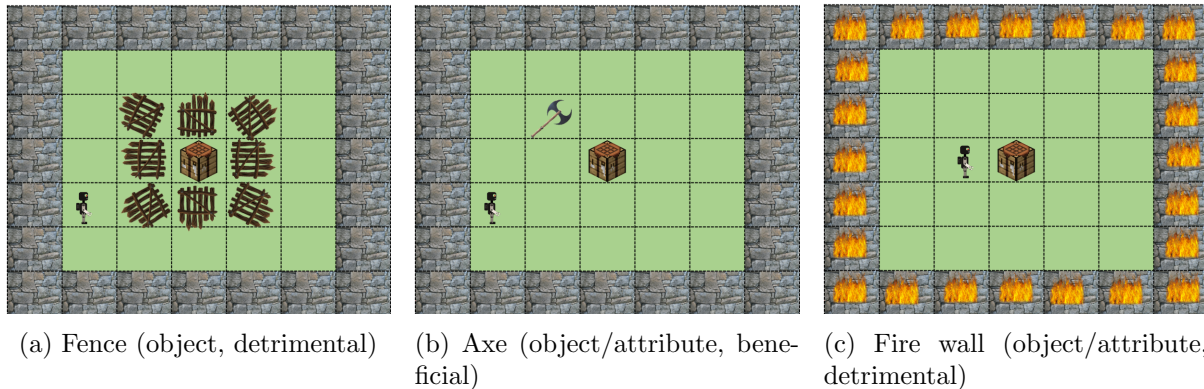


Figure 4.2: Representative NovelGridworlds novelty scenarios discussed in the evaluation (fence, axe, and fire wall). Each panel shows how the grid layout and entities change under the corresponding novelty.

Novelty Class	Beneficial	Detrimental	Irrelevant
Object	axe	fence	spring, arrow
Attribute	break increase, extract-increase	extract-decrease, fire-wall	—
Action	chop, jump	action remap	—

Table 4.2: Classification of all novelties based on their class and type. Some novelties span multiple classes (see Figure 4.3).

Twelve novelties are implemented across all four tasks, spanning all combinations of class and type. Table 4.2 summarizes the classification of each novelty, and Figure 4.3 shows how novelties are distributed across the three classes, highlighting that several novelties span multiple classes simultaneously.

Detailed novelty descriptions. We describe each novelty below, organized by class.

- **Fence** (Object, Detrimental): A new object called *fence* appears in the environment, enclosing some or all existing items to obstruct the agent’s access. The agent must learn to break the fence to reach the resources.
- **Arrow & Spring** (Object, Irrelevant): A new item—*arrow* in the Bow tasks or *spring* in the Pogostick tasks—appears on some grid spaces. These novelties are irrelevant to the task and should be ignored by the agent.
- **Break increase** (Attribute, Beneficial): The number of items received from breaking an

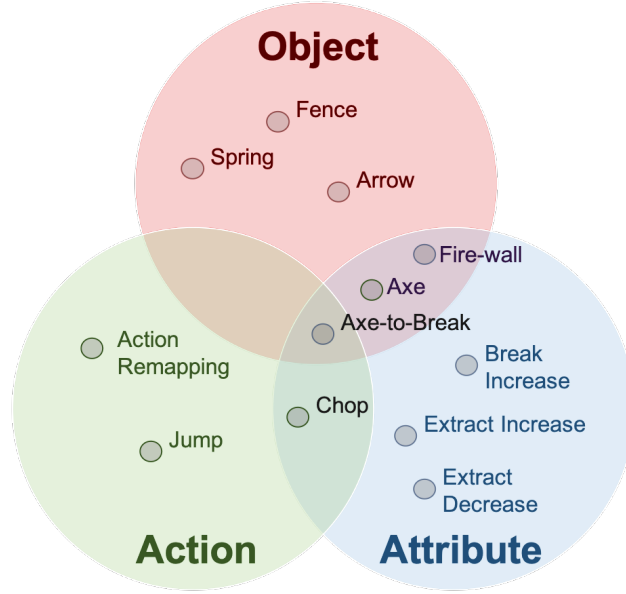


Figure 4.3: Venn diagram classifying all twelve novelties implemented in NovelGridworlds by the novelty class they affect. Novelties in overlapping regions span multiple classes simultaneously.

object doubles (e.g., breaking a tree log yields 2 instead of 1). This beneficial novelty allows the agent to collect resources with fewer actions.

- **Extract increase/decrease** (Attribute, Beneficial/Detrimental): The number of items obtained from executing the *extract* action changes. An increase is beneficial; a decrease is detrimental.
- **Action remapping** (Action, Detrimental): The effects of actions are randomly shuffled among the available action set. For example, *turn-left* may now move the agent forward. This is analogous to a system malfunction requiring the agent to relearn action semantics.
- **Jump** (Action, Beneficial): A new action *jump* is added, allowing the agent to move two blocks forward in its facing direction, enabling faster navigation.
- **Axe** (Object + Attribute, Beneficial): A new object *axe* appears. If the agent selects and uses it, the yield from breaking objects doubles. This novelty introduces both a new object and modifies attribute dynamics.
- **Fire wall** (Object + Attribute, Detrimental): The material of the arena walls changes to *fire wall*. Contact with a fire wall terminates the episode, requiring the agent to navigate more

carefully.

- **Chop** (Action + Attribute, Beneficial): A new action *chop* is added. Using it on breakable objects yields double the items compared to *break*, combining a new action with modified attribute dynamics.
- **Axe-to-break** (Object + Action + Attribute, Detrimental): A new object *axe* appears, and the *break* action only works if the agent is holding the axe. This novelty spans all three classes: it introduces a new object, modifies an existing action’s preconditions, and alters transition dynamics.

4.3.3 Evaluation Metrics

NovelGridworlds introduces three evaluation metrics designed to capture different aspects of an agent’s response to novelty:

- **Metric M1 (Reaction Performance)**: The ratio of cumulative action cost in the post-novelty scenario to the cost in the pre-novelty scenario. A ratio greater than 1 indicates degraded performance, less than 1 indicates improvement, and equal to 1 indicates no change.
- **Metric M2 (Steps)**: The number of steps the agent takes to complete the task in the post-novelty scenario.
- **Metric M3 (Detection Rate)**: A binary indicator of whether the agent correctly reports the presence of a novelty. This metric is specific to agents with explicit novelty detection mechanisms, such as planning agents.

4.3.4 Experimental Results

Two types of agents were evaluated: a symbolic planning agent implemented using the DIARC cognitive architecture [114, 115] with the MetricFF planner [56], and a reinforcement learning agent trained using Proximal Policy Optimization (PPO) [117]. The RL agent used a LiDAR-like observation space consisting of eight beams at 45° increments, each returning the Euclidean distance to the nearest object, augmented by inventory and selected-item vectors.

Table 4.3: Performance of the planning agent across four tasks. Each task reports pre-novelty (Default) and post-novelty (Fence, Arrow/Spring, Axe) scenarios.

Metrics	Default	Fence	Arrow/Spring	Axe
NovelGridworld-Bow-v0				
M1	1.00 ± 0.00	1.80 ± 0.15	1.11 ± 0.135	1.02 ± 0.04
M2	87.6 ± 16.8	123.0 ± 18.0	90.6 ± 15.1	98.0 ± 15.9
NovelGridworld-Bow-v1				
M1	1.00 ± 0.00	1.82 ± 0.24	1.00 ± 0.02	0.99 ± 0.02
M2	36.8 ± 6.5	54.2 ± 4.8	37 ± 5.4	35.4 ± 4.0
NovelGridworld-Pogostick-v0				
M1	1.00 ± 0.00	1.16 ± 0.08	0.97 ± 0.03	0.99 ± 0.04
M2	127.8 ± 9.0	122.0 ± 19.8	105.6 ± 15.7	114.8 ± 4.0
NovelGridworld-Pogostick-v1				
M1	1.00 ± 0.00	1.34 ± 0.06	1.07 ± 0.05	0.99 ± 0.02
M2	94.2 ± 12.9	105.2 ± 14.5	100.8 ± 22.58	85.0 ± 15.4

Planning agent. The planning agent was evaluated across all four tasks under three novelties (fence, arrow/spring, and axe); results are reported in Table 4.3. The agent achieved a 100% detection rate (M3) in all scenarios. Under the fence novelty, the cost ratio M1 consistently exceeded 1 across all tasks (for example, 1.80 ± 0.15 in Bow-v0 and 1.34 ± 0.06 in Pogostick-v1), reflecting the additional cost of breaking fences to access resources. Under irrelevant novelties (arrow/spring), the agent’s performance remained close to the pre-novelty baseline, as expected. The axe novelty, although beneficial, produced negligible improvement because the agent did not always discover or exploit the new object.

Reinforcement learning agent. The RL agent was evaluated on the Bow-v0 task under three novelties: action remapping, fire wall, and break increase. The agent achieved a 92% success rate on the base task after 1.8 million training time steps; results are reported in Table 4.4.

Under the beneficial break-increase novelty, cumulative reward improved but the success rate dropped to 29.2%, suggesting the agent learned to hoard resources rather than complete the task. The fire-wall novelty caused a complete failure, with 0% success rate, as the agent could not learn to avoid the lethal walls. Action remapping showed an average 12% decline in success rate, though performance varied substantially across trials depending on how severely the action mapping was shuffled.

Novelty	Success (%)
Default	92.0 ± 20.8
Action remapping	80.0 ± 40.0
Fire wall	0.0 ± 0.0
Break increase	29.2 ± 19.6

Table 4.4: RL agent success rate on *NovelGridworld-Bow-v0* and three novelty scenarios ($n = 10$ trials). Values are mean $\pm$ std of success rate, computed at 1.8M timesteps (default) and 4M timesteps (novelty scenarios).

4.3.5 Key Insights

The NovelGridworlds experiments reveal a fundamental tension between the two dominant paradigms for sequential decision-making. Planning agents, equipped with explicit models of the environment, excel at detecting novelties through discrepancies between expected and observed states. However, they lack mechanisms for adapting their models in response to detected novelties, and their performance degrades gracefully but without recovery. Reinforcement learning agents, in contrast, can potentially adapt through continued interaction but suffer from high sample complexity and are brittle when novelties alter the observation space or introduce lethal states. Neither paradigm alone is sufficient for robust open-world operation. These observations directly motivate the development of hybrid approaches, as well as a more flexible evaluation ecosystem, which we present in the next section.

4.4 NovelGym

NovelGym³ extends and generalizes the ideas introduced in NovelGridworlds into a flexible ecosystem for developing and benchmarking open-world agents. While NovelGridworlds provides a fixed set of tasks and novelties suitable for initial evaluation, NovelGym introduces a formal framework for defining environments and their transformations, a modular software architecture for rapid development of tasks, novelties, and agent systems, and a comprehensive evaluation protocol with adaptation-focused metrics. The ecosystem supports single-agent and

³The project website and codebase are available at <https://github.com/tufts-ai-robotics-group/NovelGym>.

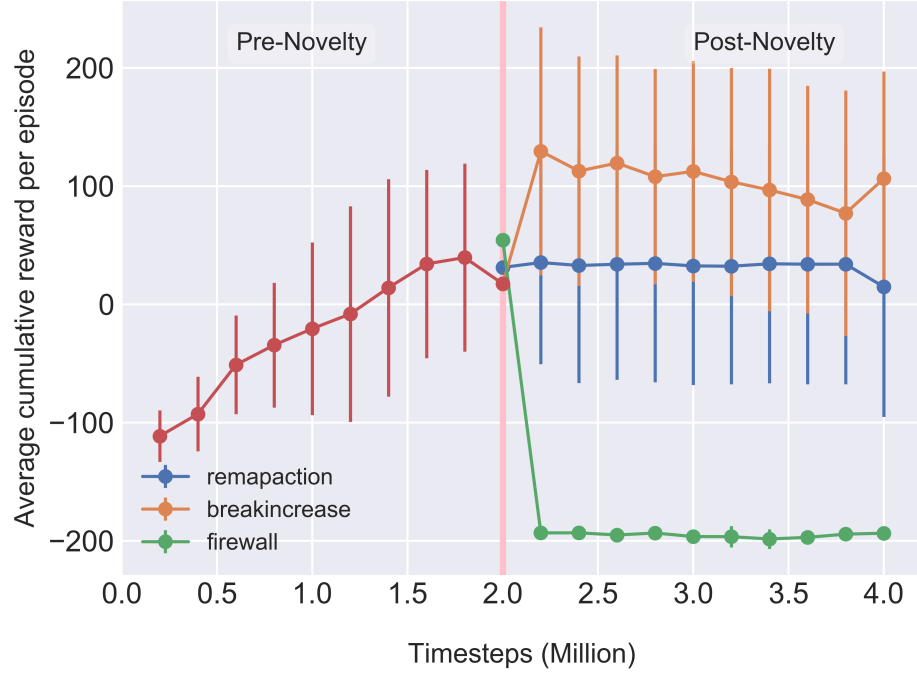


Figure 4.4: Learning performance of the RL agent on NovelGridworld-Bow-v0. The vertical line at 2 million time steps marks the introduction of novelty. Break increase (orange) shows improved reward but reduced success rate. Fire wall (green) causes complete failure. Action remapping (blue) shows partial recovery.

multi-agent scenarios, accommodates symbolic planning agents, reinforcement learning agents, and hybrid architectures, and enables composable novelty injection. The environment representation introduced in Section 4.4.1 serves as the basis for the formalism presented below.

4.4.1 Environment Formalism

We formalize the environment as a tuple $E = \langle G, \mathcal{E}, \mathcal{R}, \mathcal{S}, \mathcal{A}, \tau, C \rangle$, where each component is defined as follows.

Grid. $G \subseteq \mathbb{Z}^2$ represents the set of all grid cells in a two-dimensional gridworld. For an $m \times n$ grid, each cell is identified by its position (i, j) with $1 \leq i \leq m$ and $1 \leq j \leq n$.

Entities. $\mathcal{E} = \{\langle t_1, P_{e_1} \rangle, \langle t_2, P_{e_2} \rangle, \dots, \langle t_n, P_{e_n} \rangle\}$ is the set of all entities in the environment. Each entity $e = \langle t, P_e \rangle$ consists of a type t drawn from the set of all entity types T and a set of properties P_e drawn from a global property set P . Some entities are *dynamic*, meaning they

can take actions in the environment and influence state transitions. Entities can be located on grid cells, in an agent’s inventory, or nested within other entities.

Recipes. $\mathcal{R} = \{r_1, r_2, \dots, r_r\}$ is a set of transformation rules. Each rule $r : \mathcal{E}^* \times L_r \rightarrow \mathcal{E}^*$ defines how one or more entities can be transformed into another set of entities, where $L_r \subseteq G$ restricts the locations at which the recipe can be applied. For example, a recipe might specify that one unit of platinum, one stick, and two planks can be combined at the crafting table to produce a platinum axe.

States, actions, and transitions. $\mathcal{S}$ is the set of all possible states, capturing the locations and properties of all entities. $\mathcal{A} = \{a_1, a_2, \dots, a_a\}$ is the set of primitive actions. While all actions are syntactically available in every state, some produce effects only in specific contexts. The transition dynamics are defined as:

$$\tau : \mathcal{S} \times \mathcal{A} \times (\mathcal{A}_{e_1}, \mathcal{A}_{e_2}, \dots, \mathcal{A}_{e_m}) \rightarrow \mathcal{S},$$

where the tuple $(\mathcal{A}_{e_1}, \dots, \mathcal{A}_{e_m})$ captures the actions taken by dynamic entities in the environment, reflecting the multi-agent nature of the system.

Cost function. $C : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^+$ assigns a non-negative cost to each state-action pair. Costs can depend on context; for example, breaking a tree may cost less when the agent holds a tool.

4.4.2 Agent Formalism

An agent is defined as $\mathcal{A} = \langle \text{Sen}, \text{Act}, \mathcal{K}, \Pi \rangle$, where *Sen* denotes the agent’s sensors that generate observations O of the world, *Act* denotes actuators that can affect the world, $\mathcal{K}$ is a knowledge repository that may be initially empty and accumulate knowledge over time, and $\Pi : O \rightarrow \mathcal{A}$ maps observations to actions.

Observations can take multiple forms depending on the agent’s sensor configuration. NovelGym supports LiDAR-like sensors that return distances to entities at angular increments, image-based local view representations using one-hot vector encodings of entity types on a grid around the agent, and symbolic state descriptions expressed in PDDL. This flexibility allows

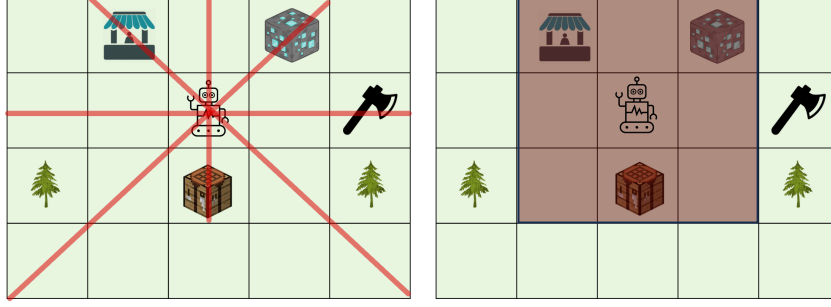


Figure 4.5: Illustration of the agent’s sensor representations in NovelGym. (Left) A LiDAR-like representation emitting beams at angular increments. (Right) An image-based local view representation using one-hot vector encodings.

the same environment to interface with planning agents, reinforcement learning agents, and hybrid systems without modification.

The agent’s behavior function Π can be implemented using symbolic planning, reinforcement learning, or hybrid approaches, as formalized in Sections 2.3.1 to 2.3.3. Actions can be primitive (e.g., `move-forward`) or parameterized operators (e.g., `approach <entity>`) that are executed through sequences of primitive actions by an internal planner.

4.4.3 Novelty as Environment Transformation

NovelGym formalizes novelty through the concept of environment transformations. A transformation is a function $\nu : E \rightarrow E'$ that modifies one or more components of the environment. This formalism complements the class-based taxonomy introduced in Section 4.3.2: while the taxonomy categorizes novelties by which aspect of the agent’s model they affect, the transformation formalism specifies precisely which components of the environment are modified and how. Six categories of transformation are supported:

- **Layout changes:** The grid G is modified such that $G \neq G'$, altering the spatial structure of the environment.
- **Entity alterations:** The set of entities $\mathcal{E}$ changes such that $\mathcal{E} \neq \mathcal{E}'$, introducing new entities or modifying existing ones.
- **Recipe modifications:** The transformation rules $\mathcal{R}$ change such that $\mathcal{R} \neq \mathcal{R}'$, altering how items can be crafted.

- **Action alterations:** The action set $\mathcal{A}$ changes such that $\mathcal{A} \neq \mathcal{A}'$, adding, removing, or modifying available actions.
- **Transition dynamics changes:** For some state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$, $\tau(s, a) \neq \tau'(s, a)$, changing how the environment responds to the agent's actions.
- **Cost function alterations:** The cost function C changes such that $C \neq C'$, modifying the cost associated with certain actions.

Critically, these transformations are *composable*. Multiple transformations can be applied in sequence to construct complex novelty scenarios, enabling the simulation of environments that undergo continual change. Whether a particular transformation constitutes a novelty for a given agent depends on the agent's knowledge, perceptions, and representations. An agent that navigates using LiDAR sensors may be unaffected by a layout transformation that changes the visual appearance of walls, whereas an agent relying on image-based observations would encounter novelty.

4.4.4 Architecture and Implementation

The NovelGym ecosystem is composed of two main components: the environment and the agent training architecture. A novelty injector module transforms the environment based on specified configurations.

Environment. The environment is implemented on top of PettingZoo [129] and OpenAI Gym [15], supporting multi-agent systems natively. It comprises several modules:

- A *specification module* that reads YAML configuration files and initializes the world, including entities, actions, and their properties.
- An *entity module* responsible for entity initialization and customization, including configurable properties such as breakability.
- An *action module* that implements primitive actions and higher-level operators and ensures the world state is updated upon execution.

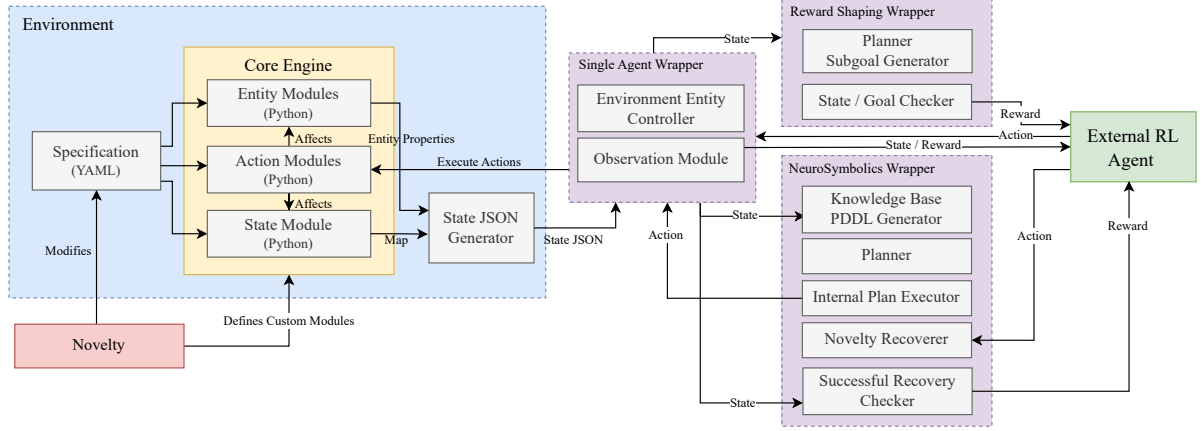


Figure 4.6: System design of the NovelGym ecosystem. Blue highlights the environment modules and purple highlights the agent modules. The novelty injector (red) transforms the environment based on specified configurations.

- A *state module* that maintains the world map, tracks entities, and manages scheduled state updates for durative actions.
- A *novelty module* that modifies entity, action, and state modules according to the novelty configuration.

Agent training architecture. The agent-side architecture is designed to support hybrid planning and learning systems through three components:

- A *single-agent wrapper* that converts the multi-agent PettingZoo environment into a single-agent interface by controlling all non-primary agents. It includes an observation conversion module that can produce LiDAR, local-view, or symbolic state representations. For object-centric observation spaces, the wrapper automatically expands observation and action spaces when new entities or actions appear due to novelty.
- A *neurosymbolic wrapper* that maintains a PDDL-based knowledge base, automatically generates PDDL domain and problem files from the environment configuration, interfaces with a planner (MetricFF [56]) to generate plans, executes plan operators through a plan executor, and provides a novelty recovery component for implementing adaptation routines.
- A *reward-shaping wrapper* that uses the integrated planner to generate subgoals from the symbolic plan and shapes rewards by monitoring whether subgoals are achieved during learn-

ing. This wrapper enables sophisticated routines for novelty handling that combine planning and learning.

The architecture also integrates a modular reinforcement learning framework (Tianshou [136]), enabling users to experiment with various learning algorithms. The modularity of the system played a crucial role in enabling the implementation of complex hybrid agent architectures evaluated in this chapter and developed further in the RapidLearn line of work [42].

4.4.5 Evaluation Protocol and Metrics

NovelGym introduces an evaluation protocol consisting of four phases, illustrated in Figure 4.7:

1. **Initial training:** The agent is trained in a controlled, novelty-free environment until its knowledge base or policy is sufficient to solve the task.
2. **Novelty injection:** A transformation is applied to the environment. The agent’s performance is monitored immediately to assess the impact.
3. **Adaptation:** The agent interacts with the transformed environment to recalibrate its approach.
4. **Post-adaptation evaluation:** After a predefined adaptation period or upon meeting a convergence criterion, the agent’s performance is assessed in the novel environment.

Five evaluation metrics are defined to capture different aspects of an agent’s behavior under novelty:

1. **Pre-novelty asymptotic performance** (S_{pre}): The agent’s success rate after convergence in the pre-novelty task.
2. **Novelty impact** (I_{novelty}): The immediate performance drop upon novelty introduction, defined as $I_{\text{novelty}} = S_{\text{pre}} - S_{\text{immediate post}}$.
3. **Time to adapt** (T_{adapt}): The number of time steps, actions, or wall-clock time required for the agent to reach a convergence criterion after novelty injection.

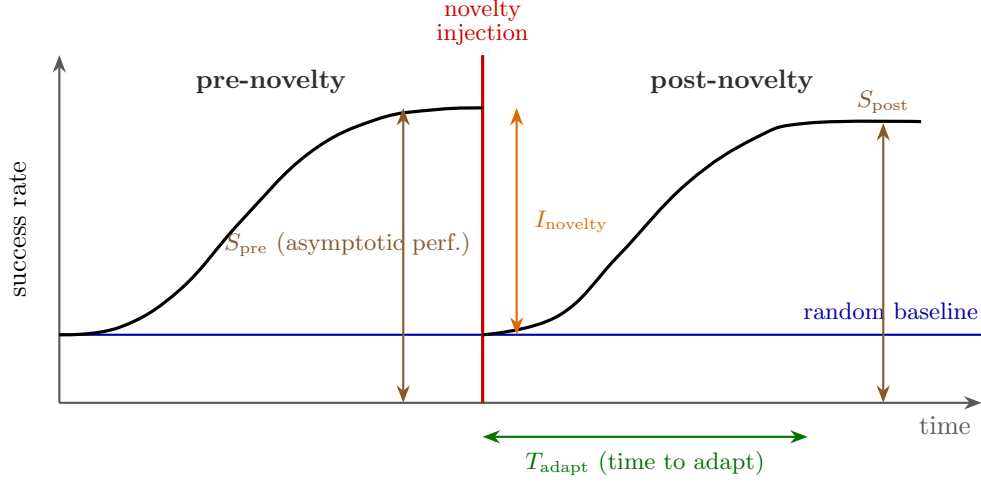


Figure 4.7: Illustration of the NovelGym evaluation protocol and performance over time: the learning agent’s success rate (black) as training progresses, with novelty injection (red) dividing the pre- and post-novelty regimes. At initialization the agent’s policy is effectively random, so the curve begins at the random-action baseline (blue) and learning lifts it above this chance-level reference. Novelty injection collapses performance back to the chance-level baseline; the post-novelty learning curve then restarts from this baseline point and adaptation gradually recovers performance to S_{post} . The annotated regions correspond to the metrics in Section 4.4.5: pre-novelty asymptotic performance (S_{pre}), novelty impact ($I_{novelty}$), time to adapt (T_{adapt}), and post-novelty asymptotic performance (S_{post}).

4. **Asymptotic adaptation performance** (S_{post}): The agent’s success rate after adaptation has converged in the post-novelty scenario.
5. **Post-adaptation efficiency** (Δt): The difference in average task completion time before and after novelty, defined as $\Delta t = t_{pre} - t_{post}$. A positive value indicates that the novelty ultimately allowed the agent to find a more efficient solution.

These metrics extend the M1, M2, and M3 metrics of NovelGridworlds by explicitly capturing adaptation dynamics. While the earlier metrics focus on cost and detection, the NovelGym metrics additionally measure the speed of adaptation, the severity of the initial performance drop, and whether the agent can exploit beneficial novelties.

4.4.6 Benchmark Task and Novelties

The benchmark task in NovelGym is a Minecraft-inspired pogostick crafting task. The agent operates in a two-room gridworld environment containing resources (oak trees, diamond ore, platinum blocks), a crafting table, and a trader (see Figure 4.9, Figure 4.8). The agent must

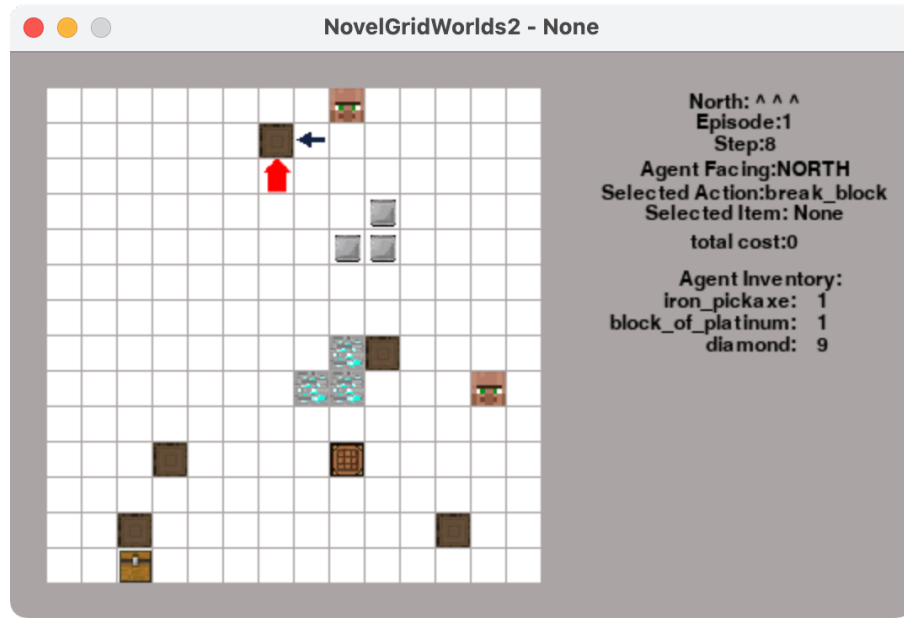


Figure 4.8: Screenshot of the NovelGym environment running the pogostick crafting task. The agent (blue arrow) navigates a grid containing oak logs (brown), diamond ore (teal), platinum blocks (silver), a crafting table, and a trader (green face). The right panel displays the current episode, step count, agent facing direction, selected action, total cost, and inventory contents.

collect resources, craft intermediary items, trade with the trader, and ultimately craft a pogostick. A simplified version of the task was used to make it tractable for reinforcement learning agents while preserving the sequential, multi-step nature of the crafting challenge.

Five environment transformations were selected for evaluation, spanning detrimental and beneficial categories:

1. **Axe** (detrimental): Trees become unbreakable unless the agent uses a newly introduced axe item.
2. **Chest** (beneficial): A chest containing all ingredients for crafting a pogostick is placed in the environment, along with a new action to approach it.
3. **Trader** (detrimental): The agent must maintain one grid cell of distance from the trader for the trade action to succeed.
4. **Fence** (detrimental): All oak logs are surrounded by breakable fences that must be removed before the logs can be accessed.



Figure 4.9: Illustration of the pre-novelty environment and three novelty scenarios: fire, fence, and chest. Each transformation modifies specific components of the environment formalism.

5. **Fire** (detrimental): The crafting table is set on fire, and the agent must collect a water bucket and extinguish the fire before crafting.

Twelve transformations are implemented in total, including additional novelties such as *busy traders*, *moving traders*, *multi-room layouts*, and *portal treasure*⁴.

4.4.7 Agent Architectures

Four agent architectures were benchmarked, spanning hybrid and transfer learning approaches:

- **RapidLearn⁺ (PPO+ICM)**: A hybrid planning and learning architecture that uses symbolic planning for the pre-novelty task and instantiates a reinforcement learning problem when plan execution fails due to novelty. The RL component uses PPO [117] augmented with the Intrinsic Curiosity Module (ICM) [106] for exploration. This architecture is a direct implementation of the method introduced in Goel et al. [42].

⁴A complete list of all implemented transformations and their specifications is provided in Goel et al. [44].

- **RapidLearn (PPO)**: The same hybrid architecture without ICM.
- **Transfer RL (PPO)+ICM**: A pure reinforcement learning approach that pre-trains on the base task and transfers weights to the post-novelty task, with automatic expansion of observation and action spaces, augmented with ICM.
- **Transfer RL (PPO)**: The same transfer approach without ICM.

All agents used a LiDAR-like observation space with beams emitted at $\frac{\pi}{4}$ increments for each entity type, producing an observation vector of size $8 \times |\mathcal{E}|$, augmented by inventory and selected-item sensors. The symbolic description for hybrid agents was represented in PDDL with a type hierarchy, predicates for entity relations, and operators defined with preconditions and effects.

4.4.8 Experimental Results

All agents achieved perfect pre-novelty performance ($S_{\text{pre}} = 1.0$), with the RL agents requiring 4 million training time steps to reach this level. Results are organized across two tables. Table 4.5 reports performance metrics—pre-novelty success rate, novelty impact, and post-adaptation success rate—which capture how severely each novelty degrades the agent and whether the agent ultimately recovers. Table 4.6 reports efficiency metrics—adaptation time and post-adaptation efficiency—which capture how quickly the agent adapts and whether the adapted strategy is more or less costly than the original. Together, these tables provide a comprehensive picture of agent behavior under novelty; see Goel et al. [44] for additional results and analysis⁵. The hybrid agents (RapidLearn and RapidLearn⁺) are instances of the planning-learning architecture that is formalized and extended in Chapter 5.

Novelty impact. As shown in Table 4.5, upon novelty injection all agents experienced performance degradation for detrimental novelties, with the fire novelty producing the most severe impact ($I_{\text{novelty}} = 1.0$ for all agents). Transfer RL agents were generally more sensitive to novelty than hybrid agents, as reflected in higher I_{novelty} values. However, hybrid agents showed lower

⁵All results are averaged across 10 random seeds. A convergence criterion based on success rate and reward thresholds was used to determine when adaptation was complete; see Section 4.4.9 for the precise convergence criteria and Section 4.4.16 for hyperparameter settings.

Novelty	Agent	$S_{\text{pre}} \uparrow$	$I_{\text{nov}} \downarrow$	$S_{\text{post}} \uparrow$
Axe	RapidLearn ⁺ (PPO+ICM)	1 ± 0	0.83 ± 0.205	1.0 ± 0.0
	RapidLearn (PPO)	1 ± 0	0.69 ± 0.207	1.0 ± 0.0
	Transfer RL (PPO)+ICM	1 ± 0	1.0 ± 0.0	0.97 ± 0.021
	Transfer RL (PPO)	1 ± 0	1.0 ± 0.0	0.97 ± 0.018
Chest	RapidLearn ⁺ (PPO+ICM)	1 ± 0	–	–
	RapidLearn (PPO)	1 ± 0	–	–
	Transfer RL (PPO)+ICM	1 ± 0	0.0 ± 0.004	1.0 ± 0.0
	Transfer RL (PPO)	1 ± 0	0.01 ± 0.009	0.98 ± 0.011
Trader	RapidLearn ⁺ (PPO+ICM)	1 ± 0	0.45 ± 0.069	0.95 ± 0.017
	RapidLearn (PPO)	1 ± 0	0.5 ± 0.07	0.96 ± 0.02
	Transfer RL (PPO)+ICM	1 ± 0	0.96 ± 0.108	0.99 ± 0.011
	Transfer RL (PPO)	1 ± 0	0.94 ± 0.128	0.96 ± 0.025
Fence	RapidLearn ⁺ (PPO+ICM)	1 ± 0	0.38 ± 0.102	0.96 ± 0.02
	RapidLearn (PPO)	1 ± 0	0.41 ± 0.052	0.95 ± 0.008
	Transfer RL (PPO)+ICM	1 ± 0	1.0 ± 0.0	0.93 ± 0.02
	Transfer RL (PPO)	1 ± 0	1.0 ± 0.0	0.92 ± 0.024
Fire	RapidLearn ⁺ (PPO+ICM)	1 ± 0	1.0 ± 0.0	0.95 ± 0.02
	RapidLearn (PPO)	1 ± 0	1.0 ± 0.0	0.95 ± 0.016
	Transfer RL (PPO)+ICM	1 ± 0	1.0 ± 0.0	0.94 ± 0.016
	Transfer RL (PPO)	1 ± 0	1.0 ± 0.0	0.96 ± 0.029

Table 4.5: Performance metrics across novelty scenarios. S_{pre} : pre-novelty success rate; I_{nov} : novelty impact (immediate performance drop); S_{post} : post-adaptation success rate. $\uparrow$ indicates higher is better, $\downarrow$ lower is better. – indicates the novelty was not detected by the agent. All values are mean $\pm$ standard deviation over 10 seeds.

immediate impact in some cases due to their targeted adaptation approach, which activates only upon execution failure.

Adaptation speed. Table 4.6 (visualized in Figure 4.10) reveals that hybrid agents consistently outperformed transfer learning methods on the T_{adapt} metric across all novelties. For the fence novelty, hybrid agents adapted in approximately 67,000–71,000 time steps, while transfer RL agents required 700,000–854,000 time steps—an order of magnitude difference. This advantage arises from the hybrid architecture’s ability to focus learning on the specific subproblem caused by the novelty rather than relearning the entire task.

Post-adaptation performance. Returning to Table 4.5, all agents achieved high post-adaptation success rates ($S_{\text{post}} \geq 0.92$), though none reached perfect scores due to the use of a convergence criterion rather than exhaustive training.

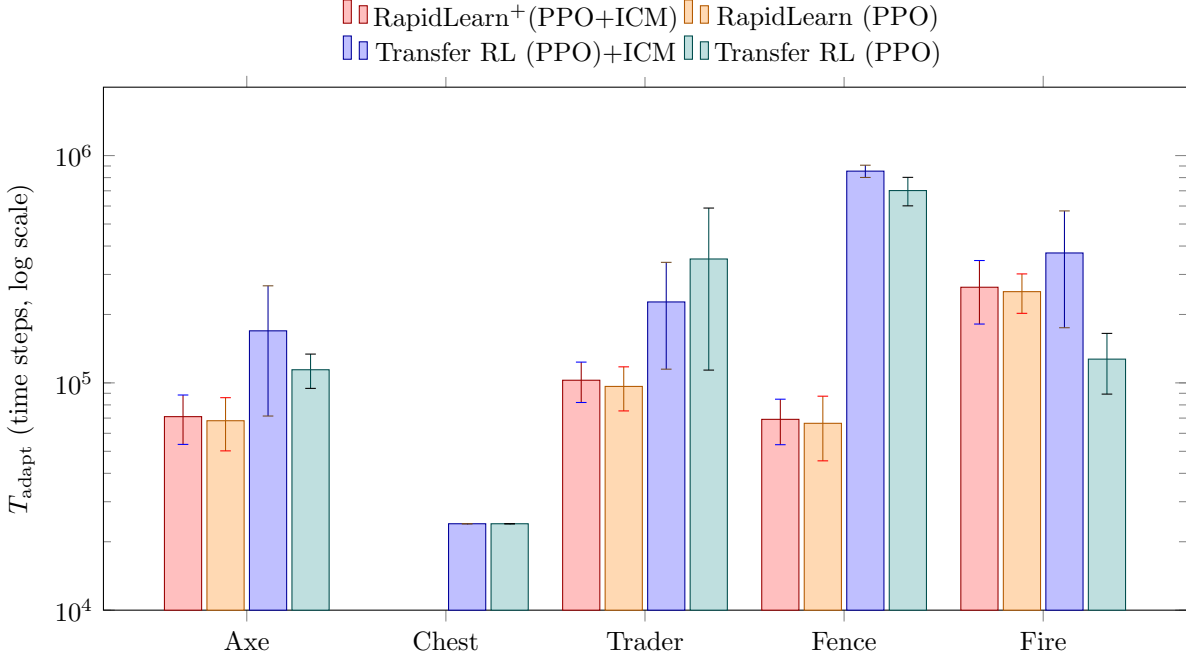


Figure 4.10: Time to adapt (T_{adapt} , lower is better) across novelty scenarios, shown on a logarithmic scale; error bars denote one standard deviation over 10 seeds. Hybrid agents (RapidLearn, RapidLearn⁺) adapt up to an order of magnitude faster than the Transfer RL baselines, most dramatically under the fence novelty. The Chest novelty was detected only by the Transfer RL agents (the hybrid agents’ plans never failed, so no adaptation was triggered), and the Fire novelty is the exception where Transfer RL (PPO) adapts quickly. Full numerical values, including the Δt metric, are reported in Table 4.6.

Post-adaptation efficiency and exploiting beneficial novelty. The post-adaptation efficiency metric Δt (Table 4.6) captures an aspect of agent behavior that success rate alone cannot: whether the agent’s post-novelty strategy is more or less efficient than its original approach. For detrimental novelties, positive Δt values are expected, since the agent must perform additional actions to overcome the novelty (e.g., breaking fences before accessing logs, or extinguishing a fire before crafting). However, the magnitude of Δt reveals how much overhead the novelty imposes even after adaptation. For instance, the fence novelty produces large Δt values for hybrid agents (161.6 ± 7.6 and 168.7 ± 6.15), reflecting the substantial additional navigation and breaking actions required, whereas transfer RL agents achieve lower Δt (73.4 ± 4.7 and 90.9 ± 9.49), suggesting they discover more efficient post-novelty strategies despite their slower adaptation.

The most notable result appears in the chest novelty, where transfer RL agents achieve

Novelty	Agent	$T_{\text{adapt}} \downarrow$	$\Delta t \downarrow$
Axe	RapidLearn ⁺ (PPO+ICM)	71040 $\pm$ 17413	122.7 $\pm$ 15.61
	RapidLearn (PPO)	68160 $\pm$ 17934	116.9 $\pm$ 9.88
	Transfer RL (PPO)+ICM	169440 $\pm$ 97949	61.0 $\pm$ 8.95
	Transfer RL (PPO)	114240 $\pm$ 19651	63.1 $\pm$ 8.17
Chest	RapidLearn ⁺ (PPO+ICM)	–	–
	RapidLearn (PPO)	–	–
	Transfer RL (PPO)+ICM	24000 $\pm$ 0	-3.0 $\pm$ 0.78
	Transfer RL (PPO)	24000 $\pm$ 0	-3.5 $\pm$ 3.13
Trader	RapidLearn ⁺ (PPO+ICM)	102720 $\pm$ 20724	126.9 $\pm$ 9.97
	RapidLearn (PPO)	96480 $\pm$ 21191	122.8 $\pm$ 8.93
	Transfer RL (PPO)+ICM	227040 $\pm$ 112120	93.6 $\pm$ 12.05
	Transfer RL (PPO)	350880 $\pm$ 237039	89.3 $\pm$ 7.87
Fence	RapidLearn ⁺ (PPO+ICM)	69120 $\pm$ 15657	161.6 $\pm$ 7.6
	RapidLearn (PPO)	66400 $\pm$ 20999	168.7 $\pm$ 6.15
	Transfer RL (PPO)+ICM	854400 $\pm$ 52974	73.4 $\pm$ 4.7
	Transfer RL (PPO)	701760 $\pm$ 100589	90.9 $\pm$ 9.49
Fire	RapidLearn ⁺ (PPO+ICM)	263520 $\pm$ 81979	133.2 $\pm$ 21.38
	RapidLearn (PPO)	252000 $\pm$ 49663	142.5 $\pm$ 13.71
	Transfer RL (PPO)+ICM	372960 $\pm$ 198096	105.3 $\pm$ 9.85
	Transfer RL (PPO)	127200 $\pm$ 37932	103.3 $\pm$ 13.22

Table 4.6: Efficiency metrics across novelty scenarios. T_{adapt} : time steps required for adaptation to converge; Δt : difference in task completion time before and after novelty (negative values indicate the agent found a more efficient post-novelty strategy). $\downarrow$ indicates lower is better. All values are mean $\pm$ standard deviation over 10 seeds.

negative Δt values (-3.0 ± 0.78 with ICM and -3.5 ± 3.13 without). A negative Δt means the agent completes the task faster after the novelty than before it—the agent has discovered and exploited the shortcut provided by the chest, which bypasses the full crafting chain. This result validates Δt as a metric that captures not only degradation but also opportunistic improvement: a well-adapted agent should be able to exploit beneficial novelties, not merely survive detrimental ones. Hybrid agents, by contrast, show no entry for the chest novelty because their plan-monitoring mechanism was never triggered—the original plan continued to succeed, so the chest was never discovered. This highlights a fundamental limitation of failure-driven adaptation: if a novelty does not cause a failure, it remains invisible to the agent, even when exploiting it would substantially improve performance.

Role of curiosity-driven exploration. The Intrinsic Curiosity Module (ICM) augments the reward signal with a prediction-error-based intrinsic reward, encouraging the agent to visit states where its learned forward model produces high prediction error. The effect of ICM on

adaptation is nuanced and novelty-dependent, as reflected in the Δt and T_{adapt} metrics.

For novelties that require discovering genuinely new states or action sequences, ICM provides a clear benefit. In the fence novelty, Transfer RL with ICM adapts faster (854,400 vs. 701,760 time steps) and achieves lower Δt (73.4 vs. 90.9), consistent with the hypothesis that curiosity-driven exploration helps the agent discover the fence-breaking strategy. However, for the fire novelty, ICM *increases* adaptation time (372,960 vs. 127,200) and slightly increases Δt (105.3 vs. 103.3). A plausible explanation is that the fire novelty introduces multiple novel state transitions—the burning crafting table, the water bucket, the extinguish action—all of which generate high prediction error in the forward model. The intrinsic reward thus disperses the agent’s exploration across many novel but not equally useful transitions, rather than focusing it on the specific sequence (collect water, extinguish fire, then craft) needed for recovery. In effect, the forward model’s prediction error is high in many irrelevant states, providing a noisy exploration signal that interferes with goal-directed adaptation.

For hybrid agents, the effect of ICM is more muted because the planning component constrains the learning problem to a specific subgoal. The RL component operates within a focused subspace defined by the failed operator, limiting the scope over which curiosity-driven exploration can either help or hinder. This further supports the architectural advantage of hybrid systems: by decomposing the adaptation problem, they reduce sensitivity to the quality of the exploration signal.

Statistical Significance

To demonstrate that the hybrid agents (RapidLearn family) adapt significantly faster than the Transfer RL baselines, we perform an unpaired t -test [69] with a confidence interval of 95%. The primary metric for comparison is T_{adapt} , the number of time steps required to converge after novelty injection, which is the central efficiency claim of the hybrid architecture. For each novelty, we compare the best-performing hybrid agent against each Transfer RL baseline. Table 4.7 reports the p -values and significance verdicts.

The results confirm that the hybrid architecture adapts significantly faster than both Transfer RL baselines across the Axe, Trader, and Fence novelties ($p < 0.01$ in all cases). The Fire novelty presents a notable exception: Transfer RL (without ICM) adapts significantly *faster*

Methods	p -value	Statistically Significant
Axe		
RL(PPO) $\leftrightarrow$ TRL+ICM	0.0097	Yes
RL(PPO) $\leftrightarrow$ TRL	< 0.001	Yes
Chest		
Hybrid agents did not detect this novelty — not applicable		
Trader		
RL(PPO) $\leftrightarrow$ TRL+ICM	0.0050	Yes
RL(PPO) $\leftrightarrow$ TRL	0.0079	Yes
Fence		
RL(PPO) $\leftrightarrow$ TRL+ICM	< 0.001	Yes
RL(PPO) $\leftrightarrow$ TRL	< 0.001	Yes
Fire		
RL(PPO) $\leftrightarrow$ TRL+ICM	0.0902	No [†]
RL(PPO) $\leftrightarrow$ TRL	< 0.001	Yes [‡]

$n = 10$ independent seeds per agent; $\alpha = 0.05$.

[†] High variance in TRL+ICM ($\pm 198k$ steps) limits power.

[‡] Significant but reversed: TRL adapts faster than hybrid on Fire.

Table 4.7: Results of unpaired t -test between the best-performing hybrid agent and each Transfer RL baseline on T_{adapt} (time steps to adapt). The hybrid advantage is statistically significant across Axe, Trader, and Fence novelties.

than the hybrid agent ($p < 0.001$), as the planning component incurs overhead attempting symbolic recovery before delegating to the RL subproblem. The comparison against TRL+ICM on Fire is not significant ($p = 0.09$), driven by the high variance of that agent on this novelty ($\pm 198k$ steps). The Chest novelty is excluded as hybrid agents never triggered adaptation — the original plan continued to succeed, leaving the novelty undetected.

4.4.9 Convergence Criteria

The learner is considered to have converged if the following conditions are met:

- The agent achieves the goal $\delta g \geq \delta G$ in the last η epochs.
- The average reward in these epochs is $\delta r \geq \delta R$.
- The max success rate and the max reward in the past $\eta + v$ epochs is not greater to the those in the last η epochs.

where each epoch is 4800 time steps, $\eta = 5$, $v = 5$, $\delta G = 0.9$, $\delta R = 400$. The episode length is 400 time steps. Episode ends if the agent reaches the goal state or if it completes 400 time steps.

4.4.10 Action Space

The action space for the hybrid agent is composed of higher-level action operators such as `approach <entity>`. These are implemented as executors that use the primitive actions available in the environment to execute the operators. Therefore the hybrid agent's action space includes both higher-level operators (implemented as lower-level action executors) and primitive actions. For the RL agent, each higher-level operator is grounded. The exhaustive list of the actions used:

- `collect`
- `break_block`
- `approach_oak_log`
- `approach_diamond_ore`
- `approach_crafting_table`
- `approach_block_of_platinum`
- `approach_entity_103`
- `interact_103`
- `select_oak_log`
- `select_iron_pickaxe`
- `select_sapling`
- `select_tree_tap`
- `select_crafting_table`

- `deselect_item`
- `craft_stick`
- `craft_planks`
- `craft_block_of_diamond`
- `craft_pogo_stick`
- `trade_block_of_titanium_1`
- `move_forward`, `move_backward`, `move_left`, `move_right`
- `rotate_left`, `rotate_right`
- `place`
- `<slot 1>`, `<slot 2>` (vary during post-novelty situations)

4.4.11 Observation Spaces

LiDAR Representation

In this representation, the agent’s observation space is a Gymnasium Box space. This Box space is made from a one-dimensional vector created by concatenating three vectors for the world as observed by the agent, the agent’s inventory, and the item selected by the agent. The agent’s world observation is generated in the following way: 8 beams of 45° each are sent from 0° to 360° , and the Euclidean distances of the objects that strike the LiDAR are stored in a two-dimensional array whose rows correspond to the integer encodings of all the possible objects and entities in the environment and whose columns correspond to the individual directions in which a LiDAR beam is sent. The assumption is made that occlusions do not hold true. The two-dimensional array is subsequently flattened. The inventory is represented as a one-dimensional array where the integer at each index corresponds to the number of objects encoded by the index that the agent has. The selected item is represented as a one-hot encoding in a one-dimensional array. Objects and entities in all three cases are assigned integer encodings. The low and high bounds of the Box space are arrays of a length corresponding to the maximum number of object and

entity types in the environment and of maximum values corresponding to the maximum count for each object and entity type. The size of the observation used during our experiment is 250, which includes placeholders for two additional objects in post-novelty situations. We added placeholders for convenience in implementation. In principle our architecture can automatically expand the network based on new entities.

Image-based Representation

In this representation, the observation space of the agent is a dictionary with three key-value pairs: for the agent’s local view, for the agent’s inventory, and for the agent’s selected item. Each of the values in the dictionary is a Gymnasium Box space, where the low and high bounds are given by the maximum number of instances of an object or entity in the environment. The agent’s local view is represented as a two-dimensional square image, where the number of one-hot encoded channels corresponds to the number of possible object and entity types in the environment. The inventory is represented as a one-dimensional array where the integer at each index corresponds to the number of objects encoded by the index that the agent has. The selected item is represented as a one-hot encoding in a one-dimensional array. Objects and entities in all three cases are represented by integer encodings.

4.4.12 Novelty Implementations

For a list of the environment transformations currently implemented in the NovelGym environment, see Table 4.8. The components of a novelty being integrated into the system are as follows.

- **YAML config file:** This config file serves as an add-on to the existing YAML config file that configures the environment. Anything included in the new config expands on the settings in the environment config, and anything included under the key “novelties” overrides any chosen component of the original settings. Common entries to the config add or modify actions, action sets, action preconditions and effects, entities, entity properties, and the grid world layout. A novelty can be created through a YAML config file alone when it does not involve adding or modifying any modules, an example of which are the

Multi-room and *Distance* novelties in Table 4.8. This is possible when no new source Python module is required for the novelty to function.

- **Python source file(s)**: In the case that a novelty requires the addition or modification of an entity or action module, it is necessary to include the Python source code for this adjustment. All the novelties specified in Table 4.8 except for *Multi-room* and *Distance* involve the addition of one or more modules. Note that changing the details of an existing module can be accomplished by changing the source Python code for the module in the novelty config file.

4.4.13 Reward Function

The reward function plays a critical role in guiding the RL agent toward completing the multi-step crafting task. Because the pogostick task requires the agent to achieve a sequence of subgoals in order—collecting resources, crafting intermediaries, trading, and finally crafting the pogostick—a sparse reward signal (rewarding only final task completion) leads to prohibitively slow learning. Instead, we use a *plan-based shaped reward* that leverages the symbolic planner to decompose the task into an ordered queue of subgoals and assigns intermediate rewards as subgoals are achieved.

The reward-shaping mechanism works as follows. The planner generates a plan P for the current task, from which an ordered queue Q of subgoals is extracted. The subgoals used in the pogostick task are:

1. `break <block of platinum>`
2. `trade block of titanium`
3. `break block of diamond`
4. `break tree`
5. `craft planks`
6. `craft stick`

Novelty	Category	Description
<i>Axe</i>	<i>Detrimental</i>	The agent is provided with an axe in its inventory at the start of the episode, and a new action “select axe” is added to its action set. The agent cannot break a tree unless it has selected the axe beforehand.
<i>Busy</i>	<i>Nuisance</i>	Each trader is busy during a certain percentage of time steps. During time steps that a trader is busy, the agent cannot successfully execute a trade against them.
<i>Chest</i>	<i>Beneficial</i>	A plastic chest containing all the ingredients required for crafting a pogostick appears in the grid world, and a new action “approach plastic chest” is added to the agent’s action set. The chest can be acted upon with the “collect” action, upon which the ingredients contained in the chest are added to the agent’s inventory.
<i>Distance</i>	<i>Detrimental</i>	One cell must separate the trader and the agent in order for the trade action to execute successfully.
<i>Fence</i>	<i>Detrimental</i>	A breakable fence surrounds each tree in the grid world, and a new action “approach fence” is added to the agent’s action set.
<i>Fire</i>	<i>Detrimental</i>	The crafting table in the grid world is given a new property “on fire”, and this property is set to true at the beginning of the episode. Somewhere else in the grid world, a water bucket is placed. The agent can extinguish the fire by choosing either the “use” action or the “collect” action while near the crafting table and holding the water bucket, whereafter it is returned a “bucket” in its inventory. The crafting table can only be crafted at after the fire has been extinguished.
<i>Moving</i>	<i>Nuisance</i>	The traders are given action sets with “move forward”, “move backward”, “move left”, and “move right” and select one of these actions randomly at every time step.
<i>Multi-interact</i>	<i>Detrimental</i>	There are two variants of this novelty. Either trees cannot be broken with the “break” action or there are no trees in the grid world at all. In either case, the agent can obtain oak logs by interacting with the traders.
<i>Multi-room</i>	<i>Nuisance</i>	Two rooms separated by a wall and doorway comprise the grid world rather than just one large room.
<i>Portal Treasure</i>	<i>Detrimental</i>	The agent is given a treasure in its inventory, a portal is placed in the grid world, and no tree can be broken using the break action. If the agent selects the action “use” while at the portal and with the treasure in its inventory, it loses the treasure but receives a number of oak logs in its inventory instead.
<i>Random drop</i>	<i>Detrimental</i>	The agent can no longer break trees using the break action nor any other action. However, with a certain probability, the action “break block” results in a given number of oak logs being dropped into the grid world.
<i>Space Around</i>	<i>Detrimental</i>	At the beginning of the episode, the agent is given a number of saplings in its inventory. These saplings can be placed in the grid world and grow into trees. The agent cannot place a sapling when there is a tree or wall within radius 1 of the item.

Table 4.8: Novelties currently implemented in the NovelGym environment

7. `craft pogostick`

A reward is only given for the first unachieved subgoal in the queue. Once a subgoal is achieved for the first time, it becomes a *past goal* and the next subgoal in the queue becomes the active target. Two additional mechanisms prevent reward exploitation:

- **Reward decay:** If the agent re-accomplishes the immediately preceding past goal (e.g., by collecting an already-collected resource type), the reward for that subgoal is halved. This discourages the agent from repeatedly achieving easy subgoals for reward.
- **Inventory check:** When a subgoal is achieved, the algorithm scans subsequent subgoals in the queue and checks them against the agent’s current inventory. If any subsequent subgoal is already satisfied (e.g., the agent already has the required items from an earlier action), that subgoal is marked complete with zero reward and skipped.

Algorithm 1 formalizes this training routine.

Algorithm 1 Training Routine with Crafted Reward Function

```

1: Plan  $P$ 
2: Ordered set (queue)  $\mathcal{Q}$  of subgoals derived from  $P$ 
3: Set of past goals  $\mathcal{G}_{\text{past}} \leftarrow \emptyset$ 
4: while  $\mathcal{Q}$  is not empty do
5:   Let  $g_{\text{current}}$  be the first element of  $\mathcal{Q}$ 
6:   if  $g_{\text{current}}$  is achieved then
7:      $\mathcal{G}_{\text{past}} \leftarrow \mathcal{G}_{\text{past}} \cup \{g_{\text{current}}\}$ 
8:     Remove  $g_{\text{current}}$  from  $\mathcal{Q}$ 
9:     if  $g_{\text{current}}$  was the immediate predecessor in  $\mathcal{G}_{\text{past}}$  then
10:       $\text{Reward}(g_{\text{current}}) \leftarrow \frac{1}{2} \times \text{Reward}(g_{\text{current}})$ 
11:    end if
12:    for each  $g_{\text{subsequent}}$  in  $\mathcal{Q}$  do
13:      if  $g_{\text{subsequent}}$  is achieved then
14:        Remove  $g_{\text{subsequent}}$  from  $\mathcal{Q}$ 
15:         $\text{Reward}(g_{\text{subsequent}}) \leftarrow 0$ 
16:      else
17:        break
18:      end if
19:    end for
20:  else
21:    Update model with  $\text{Reward}(g_{\text{current}})$ 
22:  end if
23: end while

```

Complexity. We analyze the overhead of the reward-shaping routine itself, excluding the cost of the underlying policy optimization. Let $n = |\mathcal{Q}|$ denote the number of subgoals derived from the plan (bounded by the plan length, $n \leq |P|$). Assume the queue $\mathcal{Q}$ and the past-goal set $\mathcal{G}_{\text{past}}$ are represented so that achievement checks, membership tests, insertions, and single-subgoal removals are $O(1)$ on average, and the immediate-predecessor test is $O(1)$ via a stored pointer.

Each of the n subgoals is removed from $\mathcal{Q}$ exactly once over the entire run—either as g_{current} when it is achieved or inside the skip-ahead inner loop—and the reward-decay and inventory-skip steps are $O(1)$ per subgoal. The nested loop therefore does not make the bookkeeping quadratic: a subgoal removed in the inner loop is never revisited.

- **Time:** $O(n) = O(|P|)$ for the reward-shaping bookkeeping. The overall wall-clock cost is dominated by the underlying policy optimization, which the routine only augments with a constant amount of work per step.
- **Space:** $O(n) = O(|P|)$ for the subgoal queue $\mathcal{Q}$ and the past-goal set $\mathcal{G}_{\text{past}}$, each holding at most n subgoals (excluding the memory of the underlying RL model).

Pre-Novelty Base RL

During pre-novelty training, the agent receives the plan-based shaped reward described above. The planner generates a complete plan for the pogostick task, and the subgoal queue guides the agent through the correct sequence of resource collection and crafting steps.

Post-Novelty Transfer RL

After novelty injection, the transfer RL agent uses a simplified reward: +1000 for achieving the goal of crafting a pogostick, and -1 for each time step. The plan-based shaping is not used because the planner’s model may no longer be valid in the transformed environment.

Post-Novelty Hybrid Planning and Learning

The hybrid agent’s post-novelty reward combines goal completion (+1000 for crafting a pogostick), a per-step penalty (-1 per time step), and a planning-failure penalty (-250 if the agent

reaches a state from which the planner cannot generate a valid plan). The planning-failure penalty discourages the RL component from exploring states that are irrecoverable from the planner’s perspective.

4.4.14 Sample Configuration File

Here we provide a sample configuration file, which generates trees in the world, allows the user to move around in the world, break the trees, and craft planks from the trees:

```
---
actions:
  break_block:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.Break
    step_cost: 3600
  move_forward:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.SmoothMove
    direction: W
  move_backward:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.SmoothMove
    direction: X
  move_left:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.SmoothMove
    direction: A
  move_right:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.SmoothMove
    direction: D
  craft:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.Craft
  trade:
    module: gym_novel_gridworlds2.contrib.polycraft.actions.Trade
action_sets:
  main:
    - break_block
    - craft_planks
```



```
- move_*
object_types:
  default: gym_novel_gridworlds2.contrib.polycraft.objects.PolycraftObject
  oak_log: gym_novel_gridworlds2.contrib.polycraft.objects.easy_oak_log.OakLog
map_size: [16, 16]
rooms:
  '2':
    start: [0, 0]
    end: [15, 15]
objects:
  oak_log:
    quantity: 5
    room: 2
    chunked: 'False'
entities:
  main_1:
    agent: gym_novel_gridworlds2.agents.KeyboardAgent
    name: entity.polycraft.Player.name
    type: agent
    entity: gym_novel_gridworlds2.contrib.polycraft.objects.PolycraftEntity
    action_set: main
    inventory:
      iron_pickaxe: 1
      tree_tap: 1
    id: 0
    room: 2
    max_step_cost: 100000
recipes:
  planks:
    input:
      - oak_log
      - '0'
      - '0'
```

```

- '0'

output:
  planks: 4
  step_cost: 1200
trades: {}
auto_pickup_agents:
- 0

```

4.4.15 PDDL Specification

Pre-Novelty Domain

The pre-novelty domain is defined as follows:

```

(define (domain polycraft_generated)

  (:requirements :typing :strips :fluents :negative-preconditions :equality)

  (:types
    pickaxe_breakable - breakable
    hand_breakable - pickaxe_breakable
    breakable - placeable
    placeable - physobj
    physobj - physical
    actor - physobj
    trader - actor
    pogoist - actor
    agent - actor
    oak_log - log
    distance - var
    agent - placeable
    trader - placeable
    pogoist - placeable
    bedrock - placeable
    door - placeable

```

```
safe - placeable
plastic_chest - placeable
tree_tap - placeable
oak_log - hand_breakable
diamond_ore - pickaxe_breakable
iron_pickaxe - physobj
crafting_table - placeable
block_of_platinum - pickaxe_breakable
block_of_titanium - placeable
sapling - placeable
planks - physobj
stick - physobj
diamond - physobj
block_of_diamond - physobj
rubber - physobj
pogo_stick - physobj
blue_key - physobj
)

(:constants
  air - physobj
  one - distance
  two - distance
  rubber - physobj
  blue_key - physobj
)

(:predicates ;todo: define predicates here
  (holding ?v0 - physobj)
  (floating ?v0 - physobj)
  (facing_obj ?v0 - physobj ?d - distance)
  (next_to ?v0 - physobj ?v1 - physobj)
)
```

```
(:functions ;todo: define numeric functions here
  (world ?v0 - physobj)
  (inventory ?v0 - physobj)
  (container ?v0 - physobj ?v1 - physobj)
)

; define actions here
(:action approach
  :parameters    (?physobj01 - physobj ?physobj02 - physobj )
  :precondition  (and
    (>= ( world ?physobj02) 1)
    (facing_obj ?physobj01 one)
  )
  :effect  (and
    (facing_obj ?physobj02 one)
    (not (facing_obj ?physobj01 one))
  )
)

(:action approach_actor
  :parameters    (?physobj01 - physobj ?physobj02 - actor )
  :precondition  (and
    (facing_obj ?physobj01 one)
  )
  :effect  (and
    (facing_obj ?physobj02 one)
    (not (facing_obj ?physobj01 one))
  )
)

(:action break
```

```
:parameters    (?physobj - hand_breakable)
:precondition  (and
  (facing_obj ?physobj one)
  (not (floating ?physobj))
)
:effect  (and
  (facing_obj air one)
  (not (facing_obj ?physobj one))
  (increase ( inventory ?physobj) 1)
  (increase ( world air) 1)
  (decrease ( world ?physobj) 1)
)
)

(:action break_holding_iron_pickaxe
  :parameters    (?physobj - pickaxe_breakable ?iron_pickaxe - iron_pickaxe)
  :precondition  (and
    (facing_obj ?physobj one)
    (not (floating ?physobj))
    (holding ?iron_pickaxe)
  )
  :effect  (and
    (facing_obj air one)
    (not (facing_obj ?physobj one))
    (increase ( inventory ?physobj) 1)
    (increase ( world air) 1)
    (decrease ( world ?physobj) 1)
  )
)

(:action break_diamond_ore
  :parameters    (?iron_pickaxe - iron_pickaxe)
```

```

      :precondition (and
        (facing_obj diamond_ore one)
        (not (floating diamond_ore))
        (holding ?iron_pickaxe)
      )
      :effect (and
        (facing_obj air one)
        (not (facing_obj diamond_ore one))
        (increase ( inventory diamond) 9)
        (increase ( world air) 1)
        (decrease ( world diamond_ore) 1)
      )
    )

  (:action select
    :parameters    (?prev_holding - physobj ?obj_to_select - physobj)
    :precondition  (and
      (>= ( inventory ?obj_to_select) 1)
      (holding ?prev_holding)
      (not (= ?obj_to_select air))
    )
    :effect (and
      (holding ?obj_to_select)
      (not (holding ?prev_holding))
    )
  )

  (:action deselect_item
    :parameters    (?physobj01 - physobj)
    :precondition  (and
      (holding ?physobj01)
      (not (holding air))
    )
  )

```

```

      :effect (and
        (not (holding ?physobj01))
        (holding air)
      )
    )
  )

(:action place_sapling
  :parameters (?sapling - sapling ?log - log)
  :precondition (and
    (facing_obj air one)
    (holding ?sapling)
  )
  :effect (and
    (facing_obj ?log one)
    (not (facing_obj air one))
    (increase ( world ?log) 1)
    (decrease ( inventory ?log) 1)
  )
)

(:action place
  :parameters (?physobj01 - placeable)
  :precondition (and
    (facing_obj air one)
    (holding ?physobj01)
  )
  :effect (and
    (facing_obj ?physobj01 one)
    (not (facing_obj air one))
    (increase ( world ?physobj01) 1)
    (decrease ( inventory ?physobj01) 1)
  )
)

```

```
(:action collect_from_tree_tap
  :parameters (?actor - actor ?log - log)

  :precondition (and
    (holding tree_tap)
    (facing_obj ?log one)
  )
  :effect (and
    (increase ( inventory rubber) 1)
  )
)

; additional actions, including craft and trade
(:action craft_stick
  :parameters ()
  :precondition (and
    (>= ( inventory planks) 2)
  )
  :effect (and
    (decrease ( inventory planks) 2)
    (increase ( inventory stick) 4)
  )
)

(:action craft_planks
  :parameters ()
  :precondition (and
    (>= ( inventory oak_log) 1)
  )
  :effect (and
```



```
(decrease ( inventory oak_log) 1)
(increase ( inventory planks) 4)
)
)

(:action craft_block_of_diamond
  :parameters ()
  :precondition (and
    (facing_obj crafting_table one)
    (>= ( inventory diamond) 9)
  )
  :effect (and
    (decrease ( inventory diamond) 9)
    (increase ( inventory block_of_diamond) 1)
  )
)

(:action craft_tree_tap
  :parameters ()
  :precondition (and
    (facing_obj crafting_table one)
    (>= ( inventory planks) 5)
    (>= ( inventory stick) 1)
  )
  :effect (and
    (decrease ( inventory planks) 5)
    (decrease ( inventory stick) 1)
    (increase ( inventory tree_tap) 1)
  )
)
```

```
(:action craft_pogo_stick
  :parameters ()
  :precondition (and
    (facing_obj crafting_table one)
    (>= ( inventory stick) 2)
    (>= ( inventory block_of_titanium) 1)
    (>= ( inventory diamond) 1)
    (>= ( inventory rubber) 1)
  )
  :effect (and
    (decrease ( inventory stick) 2)
    (decrease ( inventory block_of_titanium) 1)
    (decrease ( inventory diamond) 1)
    (decrease ( inventory rubber) 1)
    (increase ( inventory pogo_stick) 1)
  )
)

(:action trade_block_of_titanium_1
  :parameters ()
  :precondition (and
    (facing_obj entity_103 one)
    (>= ( inventory block_of_platinum) 1)
  )
  :effect (and
    (decrease ( inventory block_of_platinum) 1)
    (increase ( inventory block_of_titanium) 1)
  )
)

)
```

Pre-Novelty Problem

The pre-novelty problem is defined as follows:

```
(define (problem polycraft_problem)
  (:domain polycraft_generated)

  (:objects
    agent - agent
    trader - trader
    pogoist - pogoist
    bedrock - bedrock
    door - door
    safe - safe
    plastic_chest - plastic_chest
    tree_tap - tree_tap
    oak_log - oak_log
    diamond_ore - diamond_ore
    iron_pickaxe - iron_pickaxe
    crafting_table - crafting_table
    block_of_platinum - block_of_platinum
    block_of_titanium - block_of_titanium
    sapling - sapling
    planks - planks
    stick - stick
    diamond - diamond
    block_of_diamond - block_of_diamond
    rubber - rubber
    pogo_stick - pogo_stick
    blue_key - blue_key
    entity_0 - agent
    entity_103 - trader
    entity_102 - pogoist
  )

  (:init
```

```
(= (world air) 178)
(= (world bedrock) 60)
(= (world crafting_table) 1)
(= (world entity_102) 1)
(= (world oak_log) 5)
(= (world entity_0) 1)
(= (world entity_103) 1)
(= (world diamond_ore) 4)
(= (world plastic_chest) 1)
(= (world block_of_platinum) 4)
(= (world agent) 0)
(= (world trader) 0)
(= (world pogoist) 0)
(= (world door) 0)
(= (world safe) 0)
(= (world tree_tap) 0)
(= (world block_of_titanium) 0)
(= (world sapling) 0)
(= (inventory iron_pickaxe) 1)
(= (inventory tree_tap) 1)
(= (inventory agent) 0)
(= (inventory trader) 0)
(= (inventory pogoist) 0)
(= (inventory bedrock) 0)
(= (inventory door) 0)
(= (inventory safe) 0)
(= (inventory plastic_chest) 0)
(= (inventory oak_log) 0)
(= (inventory diamond_ore) 0)
(= (inventory crafting_table) 0)
(= (inventory block_of_platinum) 0)
(= (inventory block_of_titanium) 0)
(= (inventory sapling) 0)
```

```

    (= (inventory planks) 0)
    (= (inventory stick) 0)
    (= (inventory diamond) 0)
    (= (inventory block_of_diamond) 0)
    (= (inventory rubber) 0)
    (= (inventory pogo_stick) 0)
    (= (inventory blue_key) 0)
    (facing_obj air one)
    (holding air)
  )

  (:goal (>= (inventory pogo_stick) 1))
)

```

4.4.16 Hyperparameters

PPO

Table 4.9 shows the hyperparameters for the PPO algorithm:

Name	Value
Hidden Dense NN Layers	256, 64
Optimizer	Adam
Learning Rate	10^{-5}
γ (discount factor)	0.99
ϵ (PPO clip)	0.2

Table 4.9: PPO hyperparameters.

ICM

Table 4.10 shows the hyperparameters for the Intrinsic Curiosity Module:

Training Dynamics

Table 4.11 shows the hyperparameters for the pre-novelty training routine:

Name	Value
Feature Net Hidden Layers	256, 64
Feature Dimension	16
Learning Rate Scale	1
Reward Scale	0.01
Forward Loss Weight	0.2
Learning Rate	10^{-4}

Table 4.10: ICM hyperparameters.

Name	Value
Epoch Length	28800
Train Frequency	2400
Maximum Episode Length	1200
Parallel Env Thread Count	8
Batch Size	128
Batches per Train	20
Episodes per Test	100

Table 4.11: Pre-novelty training routine hyperparameters.

Different epoch lengths and maximum episode length are used for the post-novelty situation:

Name	Value
Epoch Length	4800
Train Frequency	800
Maximum Episode Length	400
Parallel Env Thread Count	4
Batch Size	128
Batches per Train	8
Episodes per Test	100

Table 4.12: Post-novelty training routine hyperparameters.

4.4.17 Novelty Implementation Example

Here is an implementation of an example novelty, which makes the traders busy 50% of the time (and they will not trade with the agent when they are busy).

Configuration File

This is the content of the novelty configuration YAML file. It specifies that the novelty needs to be injected at episode 0, and updates the action module associated with the `trade` action to be the new `BusyTrade` module. It also supplies the parameter of the new trade module, the percentage of time when the traders are busy.

```
---
novelties:
  '0':
    actions:
      trade:
        module: novelties.evaluation1.busy_traders.trade_busy.BusyTrade
        busy_ratio: 0.5
```

Modified Trade Action Module

This is the content of the modified `Trade` action module, which inherits from the default module. The new module raises an error when the internally generated random number is below the 50% threshold, and gives the error message saying that it is busy.

```
from gym_novel_gridworlds2.actions.action import PreconditionNotMetError
from gym_novel_gridworlds2.contrib.polycraft.actions import Trade

import numpy as np

class BusyTrade(Trade):
    def __init__(self, busy_ratio=0, *args, **kwargs):
        self.busy_ratio = busy_ratio
        super().__init__(*args, **kwargs)

    def do_action(self, agent_entity, target_type=None,
                  target_object=None, **kwargs):
        threshold = self.dynamics.rng.uniform(0, 1)
        if threshold < self.busy_ratio:
            raise PreconditionNotMetError(
                "Trader is busy. Please try again later.")
```

```

return super().do_action(
    agent_entity, target_type, target_object, **kwargs)

```

More examples of novelty implementations can be found in the NovelGym repository⁶.

4.5 Summary and Future Directions

This chapter presented two benchmark environments for developing and evaluating agents that operate under open-world novelty. NovelGridworlds introduced a structured novelty taxonomy, and metrics that capture both performance degradation and novelty detection. NovelGym extended this foundation with a formal environment representation, composable novelty transformations, a modular architecture supporting hybrid planning-learning agents, and an evaluation protocol with adaptation-focused metrics. Together, these environments provide the experimental infrastructure for the adaptive methods developed in the subsequent chapters of this dissertation.

The experimental results across both environments reveal consistent themes. Planning agents detect novelties reliably through discrepancies between expected and observed state transitions but lack mechanisms for adapting their models in response. Reinforcement learning agents can potentially adapt through continued interaction but require substantially more experience and are brittle when novelties alter the observation space or introduce catastrophic states. Hybrid agents that integrate planning with learning-based recovery consistently achieve faster adaptation, as measured by T_{adapt} , because they decompose the problem and focus learning on the specific subgoal affected by the novelty. However, the failure-driven nature of hybrid adaptation has a blind spot: beneficial novelties that do not cause execution failures, such as the chest shortcut, go undetected and unexploited.

The running example of the axe novelty—in which trees become unbreakable unless the agent acquires and uses a new tool—illustrates these tensions concretely. A planning agent detects the failure of the `break` operator but cannot learn the recovery behavior. A reinforcement learning agent can eventually learn to use the axe but requires orders of magnitude more experience. A hybrid agent localizes the failure and learns a targeted recovery policy. In Chapter 5, we develop

⁶<https://github.com/tufts-ai-robotics-group/NovelGym>

the RAPid-Learn framework, which formalizes and extends this hybrid recovery mechanism, using the axe novelty and other NovelGym scenarios as primary evaluation domains.

Several directions for future work emerge from the benchmarking framework introduced in this chapter.

Multi-agent open-world settings. NovelGym’s architecture, built on PettingZoo, already supports multi-agent environments, but the evaluation in this chapter focused on single-agent scenarios. Extending the framework to fully multi-agent open-world settings—where novelty can originate from other agents’ changing strategies or capabilities—would open a qualitatively different research frontier. The transformation formalism introduced in Section 4.4.3 naturally extends to such settings by defining agent-behavioral transformations alongside environmental ones, and the evaluation metrics can be generalized to capture coordination robustness and collective adaptation. This extension is particularly timely given the emergence of LLM-based multi-agent systems [134, 6, 60], which face distinctive challenges under novelty—hallucination of infeasible actions, brittleness to unexpected tool changes, and difficulty maintaining coherent plans—but lack standardized benchmarks for evaluating their open-world novelty handling capabilities.

Goal-directed curiosity for novelty handling. The experimental results in this chapter reveal that curiosity-driven exploration, as implemented through ICM [106], provides an imperfect signal for novelty adaptation: the forward model’s prediction error correlates with state novelty but not necessarily with task relevance. Recent work by Lorang et al. [88] takes a promising step by combining ICM-driven exploration with symbolic operator learning and reward machines, but the curiosity signal itself remains undirected. A still-open research direction is the development of goal-conditioned or task-aware curiosity mechanisms [85] that shape exploration bonuses by the agent’s current subgoal or by discrepancies between planned and observed operator effects. In hybrid planning–learning systems, the symbolic planner provides a natural source of such goal structure. The benchmark environments introduced in this chapter, with their structured novelties spanning beneficial, detrimental, and irrelevant categories, provide a natural testbed for evaluating such strategies.

Toward physically grounded open-world simulation. The environments presented in this chapter operate in discrete gridworlds, which provide the controlled abstraction necessary for systematic evaluation but lack the physical complexity of real-world robotic settings. Recent work on open-world simulation platforms, such as Virtual Community [144], demonstrates the feasibility of physics-grounded environments that combine robotic manipulation, human-robot interaction, and large-scale scene generation in realistic 3D worlds. Bridging NovelGym’s structured novelty injection framework with the physical fidelity of such platforms would enable evaluation of how novelty-aware agents cope with continuous state spaces, contact dynamics, and perceptual uncertainty—challenges that are central to the robotic interaction methods developed in Chapter 6.

Chapter 5

Adaptive Planning and Learning for Open-World Novelty¹

5.1 Motivation

This dissertation investigates learning, adaptation, and generalization for autonomous agents and robots operating in open-world environments. In such settings, agents must act under the assumption that their models of the world are incomplete: changes in the environment can introduce novelties that invalidate previously learned dynamics, action models, or state representations.

Chapter 4 addressed a key prerequisite for studying this problem by introducing standardized testbeds and evaluation metrics for agent behavior under novelty. The NovelGridworlds and NovelGym benchmarks provide modular environments that support diverse classes of novelty, accommodate agents across multiple paradigms, and evaluate not only task success but also the dynamics of adaptation. These include how quickly an agent detects deviations from expected outcomes, the extent to which performance degrades following a novelty, and how efficiently the agent recovers to achieve its goals. With this evaluation infrastructure in place, we now turn to the central methodological question: *how should an agent adapt its behavior when the*

¹This chapter is based on: S. Goel, P. Lymperopoulos et al., “A Neurosymbolic Cognitive Architecture Framework for Handling Novelties in Open Worlds,” *Artificial Intelligence*, vol. 331, p. 104111, 2024; and S. Goel, Y. Shukla, V. Sarathy, M. Scheutz, and J. Sinapov, “RAPid-Learn: A Framework for Learning to Recover for Handling Novelties in Open-World Environments,” in *IEEE International Conference on Development and Learning (ICDL)*, pp. 1–8, 2022.

underlying model of the agent is incomplete or invalid?

The difficulty of this problem arises from fundamental limitations of existing paradigms. In a symbolic planning framework, actions are defined by operators with preconditions and effects, and plans are generated under the assumption that these models are correct. When a novelty alters the environment dynamics, execution may fail because the expected effects of an operator do not materialize. While such failures can be detected, the planner itself has no mechanism to construct new operators or modify executors to account for the change.

Reinforcement learning methods, in contrast, operate directly over state-action interactions and can, in principle, learn new behaviors in response to such changes. However, in the absence of structural guidance, learning must proceed through broad exploration over the state-action space, often resulting in high sample complexity and the loss of previously acquired knowledge. In open-world settings characterized by abrupt, task-relevant changes, this leads to inefficient adaptation and poor reuse of prior experience.

The central insight motivating this chapter is that these two paradigms offer complementary strengths. Symbolic planning provides a structured representation of the task that can identify where execution has failed—specifically, which operator’s preconditions or effects are violated—thereby localizing the source of the novelty. Reinforcement learning, on the other hand, provides a mechanism for acquiring new behaviors through interaction, enabling the agent to learn executors that restore the validity of the plan. By combining these capabilities, adaptation can be framed as a targeted process: learning is invoked only at points of failure, and prior knowledge is preserved wherever the model remains valid.

To illustrate, consider the axe novelty introduced in Section 4.1. An agent that has learned to craft items by breaking trees encounters a change in which the environment dynamics are altered such that trees can no longer be broken directly, but instead require the use of an axe. The agent generates a plan that includes the **break** operator, whose preconditions are satisfied, but whose expected effects fail to occur during execution. This execution failure signals a mismatch between the agent’s model and the environment. To recover, the agent must incorporate newly relevant entities and actions, and learn a new executor that enables progress toward the goal. Importantly, this adaptation should occur without relearning the entire task. A purely symbolic system cannot synthesize the required behavior, and a purely learning-based

system must rediscover large portions of the task. In contrast, a hybrid approach can use the structure of the failed plan to localize the adaptation problem and learn only the missing capability required to restore successful execution.

This chapter develops methods and architectures for adaptive planning and learning under such conditions. First, Section 5.3 presents a neurosymbolic cognitive architecture [43] for open-world novelty handling that integrates symbolic reasoning, subsymbolic perception, and reinforcement learning to support the full lifecycle of adaptation—from detecting violations of expected operator effects to acquiring and reusing new behaviors under persistent novelty. The architecture embeds a cascading recovery pipeline that progresses from lightweight symbolic strategies to learning-based methods when symbolic approaches are insufficient.

Second, Sections 5.4 and 5.5 develop *RAPid-Learn* (Learning to Recover and Plan Again) [42], the learning-based recovery component at the core of the architecture. RAPid-Learn formalizes the notion of an *integrated planning task*, characterizes execution failures as instances of missing or invalid executors, and employs knowledge-guided exploration to learn recovery policies with substantially fewer interactions than standard reinforcement learning. Section 5.6 evaluates both contributions: the integrated architecture on a large-scale evaluation spanning 288 novelty scenarios, and RAPid-Learn as a standalone method on the NovelGridworlds benchmark.

5.2 Related Work

This section reviews prior work related to the adaptation methods developed in this chapter. The discussion is organized around five themes: hierarchical and temporal abstraction in reinforcement learning, hybrid planning and learning systems, creative problem solving and action discovery, transfer learning and continual adaptation, and prior approaches to novelty handling in open-world settings.

Hierarchical reinforcement learning and temporal abstraction. The executor formalism used in this chapter draws on the options framework [124], which extends reinforcement learning to include temporally extended actions defined by initiation sets, policies, and termination conditions. Options enable agents to reason and learn at multiple levels of temporal

granularity, and have been shown to improve exploration and transfer in hierarchical settings. Hierarchical deep RL methods [80] decompose tasks into subgoal sequences, improving sample efficiency in complex domains. The connection between temporal abstraction and symbolic planning has been formalized by Konidaris et al. [78], who show that the symbols necessary and sufficient for planning can be derived from the initiation sets and effect distributions of a set of skills, and by Konidaris [74], who develops the skill-symbol loop in which new skills give rise to new symbolic representations. The executor discovery problem introduced in this chapter can be viewed as the inverse of the skill-symbol loop: given a symbolic operator whose executor has failed, the agent must learn a new skill (executor) that satisfies the operator’s specification.

Hybrid planning and learning. Integrating symbolic planning with learned execution has been explored in several contexts. Task and motion planning (TAMP) systems [34, 65] combine symbolic task planning with continuous motion planning, but typically assume complete and correct domain models. PDDLStream [33] extends this paradigm by integrating symbolic planners with blackbox samplers, while Silver et al. [120] learn symbolic operators for task and motion planning from demonstrations. Konidaris et al. [77] show how to construct symbolic representations suitable for high-level planning from continuous robotic experience, establishing theoretical connections between skill learning and symbolic abstractions that underpin the operator–executor framework used in this chapter. Plan monitoring [30] and plan repair [29] address execution failures within the symbolic planning paradigm, either by detecting when a plan becomes suboptimal or by modifying a plan to account for unexpected states. However, these approaches operate entirely at the symbolic level and cannot acquire new low-level behaviors when the failure stems from a mismatch between the domain model and the environment dynamics. The SPOTTER framework [112] takes a step toward integrating planning with learning by using reinforcement learning to extend symbolic planning operators, and MacGyver problems [111] highlight the broader challenge of resourceful problem solving that motivates novelty-aware architectures. More recent hybrid approaches leverage approximate symbolic models for skill diversity [50], learn symbolic options to facilitate deep RL [64], or integrate relational planning with reinforcement learning [73]. Lyu et al. [92] demonstrate interpretable deep RL that leverages symbolic planning for both structure and data efficiency. Illanes et al.

[61] use symbolic plans as high-level instructions for RL agents, providing hierarchical structure that improves learning. Yang et al. [141] integrate symbolic planning with hierarchical RL for robust decision-making, and Thierauf and Scheutz [131] combine RL with symbolic planning in object-based action spaces for plan repair. These approaches demonstrate the complementary strengths of planning and learning, but they generally assume a fixed domain model and are not evaluated in open-world settings where plan execution failures are the norm. RAPid-Learn differs from these approaches in that it explicitly targets the executor discovery problem: when an operator fails due to novelty, it instantiates a focused reinforcement learning problem guided by the symbolic structure of the failed plan.

Creative problem solving and action discovery. A closely related line of work addresses the problem of discovering new actions or behaviors when an agent’s existing repertoire proves insufficient. Gizzi et al. [36] introduce a framework for creative problem solving by robots using action primitive discovery, in which novel actions are discovered through segmenting the low-level trajectory representations of known symbolic actions. This work establishes the connection between symbolic action representations and subsymbolic trajectory analysis that is also central to the operator–executor decomposition used in this chapter. Gizzi et al. [37] extend this approach through behavior babbling, in which agents discover new actions by exploring variations of known behaviors, analogous to how motor babbling enables infants to discover new movement capabilities. These discovered actions are represented both symbolically and subsymbolically, enabling the agent to re-plan in novel scenarios with expanded knowledge.

Gizzi et al. [39] provide a comprehensive survey and framework for creative problem solving (CPS) in artificially intelligent agents, situating action discovery within a broader landscape that includes analogical reasoning, conceptual blending, and constraint relaxation. They identify the key challenge that motivates the present chapter: when an agent’s existing knowledge is insufficient to solve a task due to environmental change, the agent must generate novel solutions that go beyond recombining known elements. Gizzi et al. [38] further extend this to a lifelong learning setting, using world models to improve performance in novelty resolution across successive encounters. The adaptation methods developed in this chapter address a specific instance of creative problem solving—recovery from execution failures—using reinforcement

learning to discover new executors guided by the symbolic structure of the failed plan.

Transfer learning and continual adaptation. When an agent encounters a changed environment, one natural approach is to transfer knowledge from the original task to the new setting. Taylor and Stone [127] provide a comprehensive survey of transfer learning for RL domains. Policy reuse [40] transfers a learned policy and selects between the old and new policies during execution. Actor-critic transfer methods [18] transfer both the policy and value function networks. Progressive neural networks [110] avoid catastrophic forgetting by freezing previously learned columns and adding new capacity for new tasks, while elastic weight consolidation [71] protects important weights from modification. Curriculum learning [102, 101] provides a complementary approach by structuring the sequence of training tasks to improve sample efficiency and transfer, with direct connections to how knowledge can be accumulated across novelty encounters. Meta-RL approaches [143] attempt to adapt to multiple MDPs and solve unseen tasks drawn from the same distribution, but assume gradual variation rather than abrupt novelty. Khetarpal et al. [68] provide a comprehensive review of continual RL methods, noting that most approaches assume continuous evolution and struggle with sudden, structural changes. Lorang et al. [86] propose methods for speeding up continual learning through information gains in novel experiences, demonstrating that prior knowledge about novelty structure can accelerate adaptation. These methods address the problem of leveraging prior experience, but they treat adaptation as a monolithic learning problem over the full state-action space. When novelty affects only a specific component of the task, this global approach is inefficient: the agent must relearn or readjust its entire policy rather than targeting the specific behavior that failed. The experimental results in Section 5.6.1 confirm this limitation, showing that transfer learning baselines require one to two orders of magnitude more experience than RAPid-Learn to achieve comparable post-novelty performance.

Open-world novelty handling. The DARPA SAIL-ON program [118] motivated the development of agent architectures for detecting and accommodating novelty in open-world environments. Boult et al. [14] and Langley [81] provide formal frameworks for reasoning about novelty, while Muhammad et al. [99] present an agent architecture that combines novelty detec-

tion with reactive accommodation strategies. Balloch et al. [8] develop neuro-symbolic world models that adapt internal representations in response to detected novelty. Thai et al. [130] present an architecture for novelty handling in multi-agent stochastic environments, evaluated in open-world Monopoly. More recently, Lorang et al. [87] demonstrate the utility of hybrid hierarchical RL and symbolic planning for open-world adaptation, and Lorang et al. [88] propose curiosity-driven imagination for discovering plan operators and learning associated policies in open-world settings. The cognitive architecture presented in Section 5.3 extends this line of work by integrating symbolic and neural novelty detection with a cascading recovery pipeline that progresses from lightweight symbolic strategies to learning-based executor discovery, and by evaluating on a substantially larger and more diverse set of novelty scenarios than prior work.

Gap addressed by this chapter. Across these lines of work, a common limitation is the absence of a principled mechanism for converting a global task failure into a targeted learning problem. Symbolic methods can detect and localize failures but cannot synthesize new behaviors. Learning methods can acquire new behaviors but lack the structural guidance needed for efficient adaptation. Action discovery methods [36, 37] demonstrate that novel behaviors can be discovered through exploration, but they do not leverage the symbolic structure of a failed plan to focus the search. The RAPid-Learn framework addresses this gap by leveraging the action abstractions developed in Section 2.4.2—the operator–executor decomposition—to bridge symbolic failure diagnosis with focused reinforcement learning, enabling adaptation that is both targeted and sample-efficient.

5.3 A Neurosymbolic Architecture for Open-World Novelty Handling

RAPid-Learn addresses the specific problem of recovering from execution impasses caused by novelty. However, a complete open-world agent must handle the full lifecycle of novelty: detecting that something has changed, characterizing the nature of the change, exploring the environment to gather information, and accommodating the novelty to continue task performance. This

section presents a neurosymbolic cognitive architecture [43] that integrates RAPid-Learn within a broader framework for novelty handling. The architecture combines symbolic inference, neural inference, and reinforcement learning, with each component contributing to different aspects of novelty detection and accommodation.

The architecture provides a concrete instantiation of the claim developed in Section 2.4: that structured abstractions over actions determine whether novelty-induced mismatch can be resolved efficiently. The operator–executor decomposition is the structural feature that enables the cascading recovery pipeline described below—without it, execution failures would propagate as undifferentiated performance degradation, and the architecture would have no principled basis for selecting among recovery strategies.

5.3.1 Novelty Typology for Goal-Oriented Agents

The architecture’s recovery strategies are organized around the four-way novelty taxonomy developed in Section 2.2.2: prohibitive, obstructive, beneficial, and nuisance novelties. For a goal-oriented agent equipped with a symbolic planner, the functional impact of a novelty depends on how it affects the agent’s ability to generate and execute plans [43]. The taxonomy maps to recovery strategies as follows.

Prohibitive novelties (Definition 2.2.2) prevent the agent from generating any valid plan—for example, if the crafting table is removed from the environment. These trigger plan-level exploration, in which the agent searches for new knowledge (entities, operators, or recipes) that would restore plannability. Obstructive novelties (Definition 2.2.3) are the primary focus of RAPid-Learn: the agent has a valid plan, but execution fails at a specific operator. The axe novelty from the running example is obstructive—the **break** operator’s executor fails because trees are now unbreakable without an axe. These invoke the failure recovery pipeline described in Section 5.3.3, culminating in the RAPid-Learn executor learner if symbolic strategies are insufficient. Beneficial novelties (Definition 2.2.4) are exploited when discovered, and nuisance novelties (Definition 2.2.5) are identified and ignored.

The distinction between prohibitive and obstructive novelties is operationally significant because they require different recovery strategies: prohibitive novelties demand symbolic model revision (new operators or fluents), while obstructive novelties demand new executors for ex-

isting operators. This distinction is finer-grained than the three-way classification used in the NovelGridworlds benchmark (Section 4.3.2), where the *detrimental* category encompasses both prohibitive and obstructive novelties.

5.3.2 Architecture Overview

The architecture, depicted in Figure 5.1, consists of four main components that interact through the agent’s knowledge repository $\mathcal{K} = (KB, \Theta)$, where KB stores symbolic knowledge (facts, rules, operators in first-order logic) and Θ stores subsymbolic knowledge (neural network parameters). The components are:

Symbolic inference. The knowledge base KB stores facts about the environment using a first-order language, including entity properties, spatial relations, and the planning domain (operators with preconditions and effects, specified in PDDL). The *task planner* retrieves the domain definition and current state from KB and searches for a plan. The *goal manager* orchestrates plan execution: it verifies operator preconditions against the observed state, dispatches executors through the domain interface, monitors whether effects match expectations, and detects novelties when predictions diverge from observations. The general principle for symbolic novelty detection is predicting the next state and comparing the prediction with the observed state—any discrepancy constitutes a potential novelty.

Neural inference. Two neural components supplement symbolic novelty detection. A *vision model*, based on a deep autoencoder [1], detects visual novelties by monitoring reconstruction error on image observations—novel visual elements produce higher reconstruction error than familiar ones. An *agent model*, trained via behavioral cloning on trajectories of known actor types, monitors other actors’ behavior and detects when their actions are unlikely under the learned behavioral model. Both components produce symbolic novelty descriptions that are asserted into KB for further reasoning.

Novelty exploration. When novelty is detected, the *novelty exploration* component employs a cascading sequence of recovery strategies, progressing from lightweight symbolic approaches

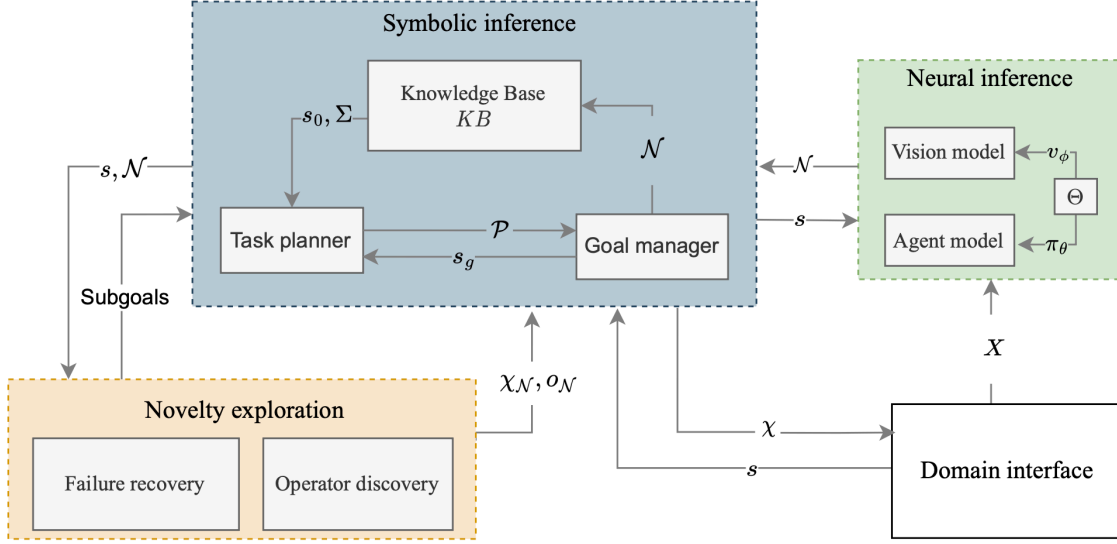


Figure 5.1: High-level architecture of the neurosymbolic cognitive architecture for novelty handling. The four main components communicate through the shared knowledge repository $\mathcal{K} = (KB, \Theta)$. Symbolic inference (task planner, goal manager) generates and executes plans, detecting novelties through expectation violations. Neural inference (vision model, agent model) detects visual and behavioral anomalies and asserts novelty descriptions into KB . When execution fails, the novelty exploration component applies a cascading sequence of recovery strategies—culminating in the RAPid-Learn executor learner—interacting with the environment through the domain interface, which mediates symbolic state observations, images, and LiDAR sensor data.

to more expensive learning-based methods. The strategies are organized around the type of failure encountered, as described in the next subsection.

Domain interface. The domain interface mediates all interactions between the agent and the environment, translating between the agent’s internal representations and the environment’s state and action formats. It provides both symbolic state descriptions (for the planner and goal manager) and subsymbolic observations such as images (for the vision model) and LiDAR-like sensor data (for executors).

5.3.3 Failure Recovery Strategies

When the goal manager detects that novelty has caused a planning or execution failure, the novelty exploration component is invoked. Recovery strategies are selected based on the nature of the failure. For the scope of this chapter, we focus on recovery from operator failures, which are the most common consequence of open-world novelty and the primary target of RAPid-Learn.

When an operator’s executor is executed but its effects are not observed in the resulting state, the architecture distinguishes between two cases:

Partial effect failure. If a subset of the operator’s expected effects are observed, the operator’s description in *KB* is updated to reflect the observed effects, and the agent replans. This handles cases where an operator’s behavior has changed but not disappeared entirely.

Total effect failure. If none of the operator’s expected effects are observed, a cascading sequence of strategies is applied:

1. *Update and replan:* The operator’s effects are removed from *KB* and the agent attempts to find an alternative plan that avoids the failed operator.
2. *Repeated execution:* The operator is retried to account for stochastic or transient failures.
3. *Precondition discovery:* The agent hypothesizes that the operator requires an additional, previously unknown precondition. New preconditions are constructed from predicates encountered in the domain and tested by satisfying them before reattempting the operator. For

instance, if `break(tree)` fails, the agent may hypothesize that `holding(axe)` is a missing precondition.

4. *Operator variations*: The agent tests known operators with similar parameters or on related objects, leveraging the type hierarchy in *KB* to discover operators with unexpected effects that may resolve the impasse.
5. *Executor learner (RAPid-Learn)*: If all symbolic strategies fail, the agent invokes the RAPid-Learn framework (Section 5.5) to learn a new executor through reinforcement learning with knowledge-guided exploration.

The cascading design reflects a deliberate trade-off between computational cost and generality. Symbolic strategies (steps 1–4) are fast but limited to novelties that can be resolved within the agent’s existing representational vocabulary. The executor learner (step 5) is more expensive but can discover recovery behaviors that involve novel predicates, spatial relations, or action sequences not present in the agent’s original domain model—precisely the cases where symbolic reasoning is insufficient. As demonstrated in Chapter 4, this architecture-level integration of planning and learning achieves substantially faster adaptation than pure learning approaches while maintaining the robustness to handle novelties that pure planning cannot resolve.

5.4 Problem Formulation

This section extends the integrated planning task (IPT) formalism developed in Section 2.3.3 to characterize adaptation under novelty. The IPT $\mathfrak{T} = \langle \mathcal{T}, M, \alpha, e \rangle$ couples symbolic planning with subsymbolic execution through the detector α and the executor mapping e . In open-world settings, this correspondence can break: novelty may render an executor invalid, creating a gap between what the symbolic model prescribes and what the agent can actually execute. We extend the operator–executor accuracy conditions from Section 2.3.3 and define the *executor discovery problem* as the core challenge of adaptation when executors fail under obstructive novelty.

5.4.1 Executors

Sections 2.3.2 and 2.3.3 introduced executors as the mechanism connecting symbolic operators to environment interaction. We restate the definition here to establish the notation used throughout this chapter.

Definition 5.4.1 (Executor). *An executor $x = \langle I_x, \pi_x, \beta_x \rangle$ consists of:*

- An initiation set $I_x \subseteq \mathcal{S}$, the set of environment states in which the executor can be initiated.
- An intra-executor policy $\pi_x(a \mid s_0, s)$, which defines a probability distribution over actions $a \in \mathcal{A}$ given initiation state $s_0 \in \mathcal{S}$ and current state $s \in \mathcal{S}$.
- A termination condition $\beta_x : \mathcal{S} \times \mathcal{S} \rightarrow \{0, 1\}$, where $\beta_x(s_0, s) = 1$ if the executor terminates in state s given that it was initiated in state s_0 , and 0 otherwise.

Let $\mathcal{X} = \{x_1, x_2, \dots\}$ denote the set of all available executors. Executors may be hand-coded, learned through reinforcement learning, or implemented by internal planners (e.g., a navigation policy that computes paths using A^*).

5.4.2 Integrated Planning Task

Definition 5.4.2 (Integrated Planning Task). *An integrated planning task (IPT) is a tuple $\mathfrak{T} = \langle \mathcal{T}, M, \alpha, e \rangle$ as defined in Section 2.3.3, where $\mathcal{T} = \langle \Sigma, \tilde{s}_0, g \rangle$ is a symbolic planning task (Section 2.3.1), $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ is a Markov decision process (Section 2.3.2), $\alpha : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ is a detector mapping subsymbolic states to symbolic states, and $e : \mathcal{O} \rightarrow \mathcal{X}$ is an executor mapping.*

An IPT is well-formed when each $e(o)$ is accurate to o as defined in Section 2.3.3. The termination condition is extended below for open-world execution.

Consistency between operators and executors. For an IPT to function correctly, each executor must be consistent with the symbolic specification of its corresponding operator (Section 2.3.1). Let $o \in \mathcal{O}$ have precondition literal set ψ_o and effect literal set ω_o . The executor $e(o)$ must satisfy:

1. *Initiation consistency*:

$$I_{e(o)} \supseteq \{s \in \mathcal{S} : \alpha(s) \models \psi_o\}.$$

In other words, the condition above says that the executor can begin execution if the operator's preconditions are satisfied.

2. *Termination consistency*: The executor terminates either when the operator's effects are achieved or when sufficient progress has been made to continue planning:

$$\beta_{e(o_i)}(s_0, s) = \begin{cases} 1 & \text{if } \alpha(s) \models \omega_{o_i}, \\ 1 & \text{if } \alpha(s) \models \omega_{\hat{o}} \forall \hat{o} \in \hat{\mathcal{O}} \text{ and } \exists \hat{\rho} \text{ from } \alpha(s) \text{ to } g, \\ 0 & \text{otherwise,} \end{cases}$$

where $\hat{\mathcal{O}} = \{o_j \in \rho : j > i \wedge o_i \prec o_j\}$ denotes subsequent operators in the plan that depend on the effects of o_i (Section 2.3.1).

The first case corresponds to exact satisfaction of the operator's effects. The second allows progress beyond the current operator if downstream effects are satisfied and a valid plan to the goal still exists. In other words, once the executor reaches a plannable state, it can be terminated. Later, we will formalize and explain how the plannable states are generated.

Solutions to an IPT. A *planning solution* to an IPT is a plan $\rho = [o_1, \dots, o_n]$ that solves $\mathcal{T}$. A *solution* is the corresponding sequence of executors $[e(o_1), \dots, e(o_n)]$ such that executing them in the environment from an initial state s_0 with $\alpha(s_0) = \tilde{s}_0$ yields a state s' with $\alpha(s') \models g$. An IPT is *plannable* if a planning solution exists and *solvable* if a solution exists.

5.4.3 The Executor Discovery Problem

In closed-world settings, every operator has a valid executor, and a plannable IPT is also solvable (Section 2.3.3). In open-world environments, novelty can break this correspondence: an executor may fail because the environment dynamics no longer match the operator's modeled effects. When operator-executor correspondence breaks while the symbolic plan remains valid, the IPT remains plannable but is no longer solvable (Section 2.3.4 and Definition 2.2.3).

Definition 5.4.3 (Stretch-IPT). *A stretch-IPT $\tilde{\mathfrak{T}}$ is an IPT $\mathfrak{T} = \langle \mathcal{T}', M', \alpha, e \rangle$ whose planning task $\mathcal{T}'$ incorporates novelty, for which a planning solution exists but the current executor mapping e does not yield an execution solution [42].*

Using the novelty formalism from Section 2.2, a novelty introduces elements not captured by the current model. When an operator o_i fails during execution—that is, $e(o_i)$ does not produce a state satisfying ω_{o_i} —the agent faces a stretch-IPT and must discover a new executor for o_i while preserving the existing symbolic plan structure.

Definition 5.4.4 (Executor Discovery Problem). *Given a stretch-IPT $\tilde{\mathfrak{T}} = \langle \mathcal{T}', M', \alpha, e' \rangle$ with $\mathcal{T}' = \langle \Sigma', \tilde{s}'_0, g \rangle$, where $\Sigma' = \langle \mathcal{F} \cup \mathcal{F}', \mathcal{O} \cup \mathcal{O}' \rangle$, and failed operators $\{o_1, \dots, o_m\}$, construct new executors $\{x'_1, \dots, x'_m\}$ such that*

$$e'(o_i) = x'_i$$

for each failed operator o_i , making $\tilde{\mathfrak{T}}$ solvable.

The executor discovery problem captures the essence of adaptation under novelty: the agent must acquire new low-level behaviors that realize high-level operator specifications in a changed environment. This cannot be achieved through symbolic reasoning alone, as the model is incomplete, nor through unguided learning, as exhaustive exploration is inefficient. Instead, the agent must leverage its symbolic structure to guide the learning of new executors. The next section presents RAPid-Learn, which addresses this problem by instantiating focused reinforcement learning problems for failed operators, guided by existing domain knowledge.

5.5 RAPid-Learn

RAPid-Learn (Learning to Recover and Plan Again) solves the executor discovery problem (Definition 5.4.4) by instantiating a targeted reinforcement learning problem for each failed operator and exploiting the agent’s symbolic knowledge to guide exploration. This section describes the framework in detail: the overall planning-execution-recovery loop, the construction of the learning problem for failed operators, and the knowledge-guided exploration strategies that make learning sample-efficient.

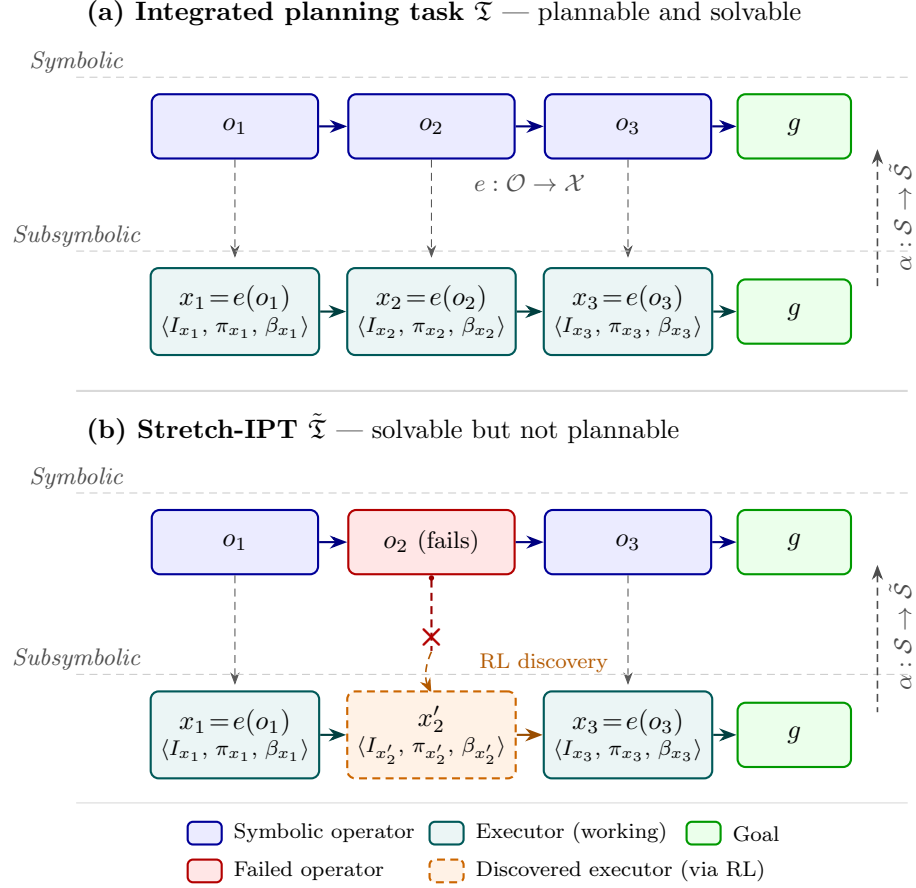


Figure 5.2: Conceptual overview of the integrated planning task (IPT) formalism. Each symbolic operator o_i is mapped to a subsymbolic executor $x_i = \langle I_{x_i}, \pi_{x_i}, \beta_{x_i} \rangle$ via the executor function e . The abstraction function $\alpha : \mathcal{S} \rightarrow \hat{\mathcal{S}}$ maps environment states back to symbolic states for the planner. **(a)** In a working IPT, all executors realize their associated operators and the agent reaches the goal g . **(b)** When novelty causes an operator to fail, the IPT becomes a Stretch-IPT: the task remains solvable but the current executor mapping is broken. RAPid-Learn solves the executor discovery problem by constructing a new executor x'_2 through reinforcement learning, restoring solvability.

5.5.1 Overview

The RAPid-Learn framework operates as follows. The agent begins with an integrated planning task $\mathfrak{T} = \langle \mathcal{T}, M, \alpha, e \rangle$ and uses a symbolic planner (MetricFF [56]) to generate a plan $\rho = [o_1, \dots, o_n]$. It then executes this plan operator by operator, using the executor mapping e to realize each operator in the environment. If all executors succeed, the agent reaches its goal. If an executor $e(o_i)$ fails—detected by observing that $\alpha(s) \not\models \omega_{o_i}$ after execution—the agent enters the *executor discovery* phase. During executor discovery, the agent checks whether it has previously learned an executor for the failed operator. If so, it executes the learned executor

Complexity. Let $|\rho|$ denote the length of the symbolic plan, and let F be the number of failed operator executions during the run. Let C_{plan} denote the cost of calling the symbolic planner, C_{exec} the cost of executing an executor, and C_{disc} the cost of one call to DISCOVER-EXECUTOR. Assume that $\mathcal{X}$ is implemented as a hash set, so membership checks and insertions are $O(1)$ on average.

The algorithm first calls the planner once, which costs C_{plan} . It then constructs the initial executor set by mapping each operator in the plan to its executor, which costs $O(|\rho|)$. The main loop executes each operator once, giving $O(|\rho| \cdot C_{\text{exec}})$ execution cost. For each failed operator, the algorithm either executes an already discovered executor or calls DISCOVER-EXECUTOR and then executes the returned executor. Therefore, the total cost is

$$O(C_{\text{plan}} + |\rho| + |\rho|C_{\text{exec}} + F(C_{\text{disc}} + C_{\text{exec}})).$$

Since $F \leq |\rho|$, the worst-case time complexity is $O(C_{\text{plan}} + |\rho| + |\rho|C_{\text{exec}} + |\rho|(C_{\text{disc}} + C_{\text{exec}}))$, which simplifies to $O(C_{\text{plan}} + |\rho|(C_{\text{disc}} + C_{\text{exec}}))$.

The space required by the top-level algorithm is dominated by the plan and the executor set. Thus, excluding the internal memory used by the planner and DISCOVER-EXECUTOR, the space complexity is $O(|\rho|)$.

5.5.2 Executor Discovery

When operator o_i fails during plan execution, the agent instantiates a reinforcement learning problem to discover a new executor x'_i . This involves three components: defining the MDP for the failed operator, generating the set of *plannable states* that serve as success criteria, and designing the reward function.

MDP construction. For the failed operator o_i , the agent constructs an episodic MDP $M_{x'_i} = \langle \mathcal{S}', \mathcal{A}', P', R_{x'_i}, \gamma \rangle$, where $\mathcal{S}'$ is the set of environment states, and $\mathcal{A}'$ consists of three types of actions: (1) the agent's pre-novelty low-level actions, (2) any novel actions detected in the environment, and (3) higher-level actions corresponding to existing executors (e.g., **approach** operators that use the internal planner for navigation). Including existing executors as actions

Algorithm 3 *PlannableStateGenerator* $(\Sigma, \mathcal{P}, o_i) \rightarrow \mathcal{S}_r$

```

1:  $\mathcal{P} = [o_1, \dots, o_{|\mathcal{P}|}]$ : plan as an ordered list of operators
2:  $\Psi = \{\psi_{o_1}, \psi_{o_2}, \dots, \psi_{o_{|\mathcal{P}|}}\}$ : preconditions of operators in  $\mathcal{P}$ 
3:  $\Omega = \{\omega_{o_1}, \omega_{o_2}, \dots, \omega_{o_{|\mathcal{P}|}}\}$ : effects of operators in  $\mathcal{P}$ 
4:  $\mathcal{S}_r \leftarrow \emptyset$ 
5: for  $o_j$  in reversed( $\mathcal{P}$ ) do
6:   if  $o_j \neq o_i$  and  $\psi_{o_j} \not\supseteq \omega_{o_i}$  then
7:      $\mathcal{S}_r \leftarrow \mathcal{S}_r \cup \psi_{o_j}$ 
8:     for  $\omega \in \omega_{o_j}$  do
9:       if  $\omega \in \mathcal{S}_r$  then
10:         $\mathcal{S}_r \leftarrow \mathcal{S}_r \setminus \{\omega\}$ 
11:       end if
12:     end for
13:   end if
14: end for
15:  $\mathcal{S}_r \leftarrow \mathcal{S}_r \cup \omega_{o_i}$ 
16: return  $\mathcal{S}_r$ 

```

in the MDP allows the learner to compose known behaviors with new ones, significantly reducing the exploration space. Each episode begins by executing the plan from the initial state up to the point of failure, so the learner always starts from the state in which the operator failed.

Plannable state generation. The agent uses its symbolic knowledge to compute the set of *plannable states* $\mathcal{S}_r \subseteq \tilde{\mathcal{S}}$ —symbolic states from which the agent can generate a valid plan to the goal (entire procedure is shown in Algorithm ??). The plannable states are computed from the plan ρ and the failed operator o_i as follows:

1. States that satisfy the effects ω_{o_i} of the failed operator (the direct recovery case).
2. States that satisfy the effects $\omega_{\hat{o}}$ of all subsequent operators $\hat{o} \in \hat{\mathcal{O}}$ whose preconditions depend on the effects of o_i (the skip-ahead case, allowing the agent to bypass the failed operator entirely if an alternative path exists).

We provide this as a disjunction of condition 1 and 2 for the agent to satisfy at each time step. In other words, if the agent satisfies either 1 or 2, it gets a positive reward ϕ_1 and the episode terminates. Formally, $\mathcal{S}_r = \{\tilde{s} : \tilde{s} \models \omega_{o_i}\} \cup \{\tilde{s} : \tilde{s} \models \omega_{\hat{o}} \text{ for all } \hat{o} \in \hat{\mathcal{O}}\}$, where $\hat{\mathcal{O}} = \{o_j \in \rho : j > i \wedge o_i \prec o_j\}$ as defined in Section 2.3.1.

Complexity. Let $|\mathcal{P}|$ denote the plan length, $\psi_{\max}$ the maximum number of preconditions per operator, and $\omega_{\max}$ the maximum number of effects per operator. Assume preconditions,

effects, and $\mathcal{S}_r$ are represented as hash sets over atomic literals, so membership, insertion, and deletion of a single literal are $O(1)$ on average.

The algorithm performs a single reverse pass over $\mathcal{P}$. For each operator, checking whether $\psi_{o_j} \not\supseteq \omega_{o_i}$ costs $O(|\omega_{o_i}|) \subseteq O(\omega_{\max})$. If the condition holds, adding ψ_{o_j} to $\mathcal{S}_r$ costs $O(|\psi_{o_j}|) \subseteq O(\psi_{\max})$, and checking/removing effects of o_j costs $O(|\omega_{o_j}|) \subseteq O(\omega_{\max})$. Finally, adding ω_{o_i} once after the loop costs $O(\omega_{\max})$.

- **Time:** $O(|\mathcal{P}|(\psi_{\max} + \omega_{\max}))$. In bounded-arity domains, this reduces to $O(|\mathcal{P}|)$.
- **Space:** $O(|\mathcal{P}|\psi_{\max} + \omega_{\max})$, reducing to $O(|\mathcal{P}|)$ in bounded-arity domains.

A *bounded-arity domain* is one in which every predicate and operator takes at most a fixed constant number of arguments (as in standard planning benchmarks), so $\psi_{\max}$ and $\omega_{\max}$ are constants independent of the plan length and the number of objects in the problem.

Reward function. The reward function is designed to guide the agent toward plannable states while penalizing irreversible actions. At each time step, the agent receives:

$$R_{x'_i}(S_r, s, a, s') = \begin{cases} +\phi_1 & \text{if } \alpha(s') \in S_r \text{ and } \exists \hat{\rho} \text{ from } \alpha(s') \text{ to } g, \\ -\phi_2 & \text{if } \alpha(s') \in S_r \text{ and } \nexists \hat{\rho} \text{ from } \alpha(s') \text{ to } g, \\ -1 & \text{otherwise,} \end{cases}$$

where $\phi_1, \phi_2 > 0$ are reward constants. The first case provides a large positive reward when the agent reaches a plannable state from which the goal is reachable. The second case penalizes reaching a state that appears plannable symbolically but from which no valid plan exists—this can occur when the agent takes irreversible actions during exploration (e.g., using up a limited resource). The third case provides a small step penalty to encourage efficient solutions. Crucially, each time the agent reaches a state in S_r , the symbolic planner is invoked to verify that a plan to the goal exists, ensuring that the learned executor actually resolves the impasse.

5.5.3 Knowledge-Guided Exploration

A naïve reinforcement learning agent exploring the post-novelty environment would face a large state-action space with a sparse reward signal, leading to slow convergence. RAPid-Learn addresses this through *knowledge-guided exploration*: using the agent’s symbolic domain knowledge to focus exploration on states and actions that are likely relevant to the novelty. Two complementary strategies are employed.

Knowledge-guided curriculum learning. Complex novelties may require the agent to first reach a promising region of the state space before meaningful exploration can begin. With probability ρ at the start of each episode, the agent is provided with a curriculum: a novel entity $\epsilon \in \mathcal{E}'$ is selected, and the symbolic planner is used to generate a plan that brings the agent to the vicinity of that entity. The agent then begins its reinforcement learning exploration from this more informative starting state. This strategy leverages the insight that novelties are typically associated with new entities or changed objects in the environment, and that reaching those entities is a prerequisite for discovering recovery behaviors.

Knowledge-guided action biasing. During exploration, the agent biases its action selection toward a subset of actions Δ that are likely relevant to the novelty. This subset consists of (1) novel actions that appeared in the environment after the novelty, and (2) the failed operator itself (in executor form), since retrying it in different states or with different preconditions satisfied may succeed. Two methods for action biasing are developed:

Upper Confidence Bound biasing (KGE-UCB). With probability ϵ_0 , the selection probability of each action $a' \in \mathcal{A}'$ is modified as:

$$\hat{\pi}(a') \leftarrow \pi(a') + \begin{cases} +c\sqrt{\frac{\ln t}{N_t(a')}} & \text{if } a' \in \Delta, \\ -c\sqrt{\frac{\ln t}{N_t(a')}} & \text{if } a' \notin \Delta, \end{cases}$$

where $c > 0$ is an exploration constant, t is the current time step, and $N_t(a')$ is the number of times action a' has been selected up to time t . This formulation, inspired by the Upper Confidence Bound algorithm [7], encourages the agent to try actions in Δ that have been

attempted the least, systematically exploring novel actions while gradually reducing exploration of familiar ones.

Uniform action biasing (KGE-UAB). An alternative strategy that uniformly increases the probability of selecting actions in Δ by a multiplicative factor $\mu > 1$:

$$\hat{\pi}(a') \leftarrow \begin{cases} \frac{(\mu-1+\sum_{\Delta}) \cdot \pi(a')}{\mu \sum_{\Delta}} & \text{if } a' \in \Delta, \\ \frac{\pi(a')}{\mu} & \text{if } a' \notin \Delta, \end{cases}$$

where $\sum_{\Delta} = \sum_{a \in \Delta} \pi(a)$ is the total probability mass assigned to actions in Δ under the current policy.

Both methods are extensions of ϵ -greedy exploration that incorporate domain knowledge about which actions are most likely to be relevant for resolving the novelty. The rationale follows from two assumptions: (1) the agent should exploit its existing domain knowledge to direct exploration rather than exploring uniformly, and (2) the novelty that caused the impasse is likely related to novel entities or actions in the environment.

5.5.4 Recovery and Executor Reuse

The agent trains its policy using a policy gradient algorithm [137] and monitors convergence through a success rate criterion: if the agent achieves the positive reward (reaching a plannable state with a valid plan to the goal) in at least η out of the last n episodes, the policy is considered converged. The learned policy $\pi_{x'_i}^c$ is then encapsulated as a new executor:

$$x'_i = \langle S'_0, \pi_{x'_i}^c, \beta_{x'_i} \rangle,$$

where S'_0 is the set of states from which the executor was trained (the failure states), and $\beta_{x'_i}$ is the termination condition derived from the plannable states S_r (as defined in Section 5.4.2).

The learned executor is added to the agent's executor set $\mathcal{X}$ and mapped to the failed operator. On subsequent encounters with the same failure, the agent executes the stored executor directly without relearning, amortizing the cost of exploration over future episodes. This mechanism is effective even when multiple operators fail: each failure triggers an independent executor

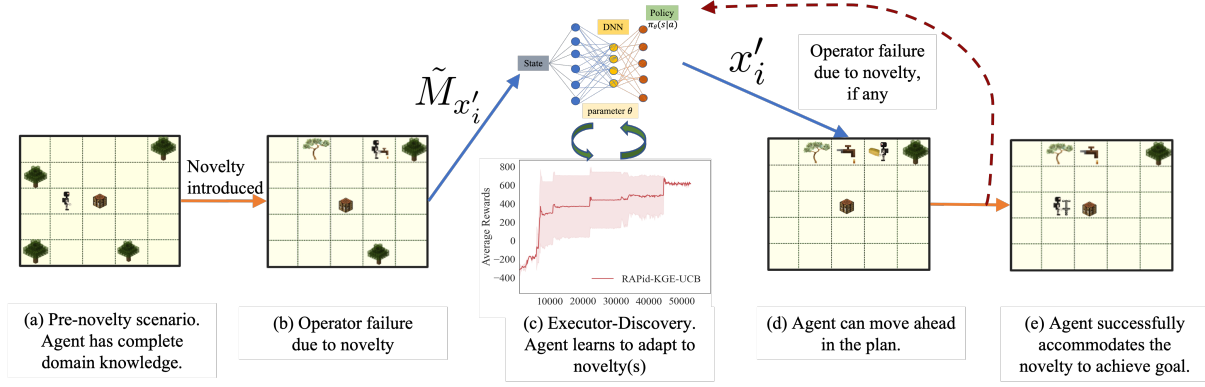


Figure 5.4: Overview of the agent in action. (a) Illustrates the NovelGridworlds domain in the pre-novelty scenario, showing an agent in a walled arena surrounded by *trees*, and facing a *crafting-table*. (b) Shows the scenario when the environment introduces the *rubber-tree-hard* novelty and the agent reaches an impasse. (c) *Executor-Discovery* phase when the agent accommodates the novelty using our approach RAPid-KGE-UCB. (d) Agent successfully resolves the plan execution impasse. (e) Agent reaches the goal state of crafting a *pogo-stick*.

discovery process, and the resulting executors are composed through the original symbolic plan structure. The framework naturally handles three types of multiple-operator failures:

- *Independent failures*: Multiple novelties cause different operators to fail independently. The agent learns a separate executor for each.
- *Cascading failures*: The learned executor for one failure inadvertently causes a downstream operator to fail. The agent detects the new failure and learns an additional executor.
- *Simultaneous failures*: Multiple operators fail in the same plan execution. The agent addresses them sequentially as they are encountered during plan execution.

5.6 Experimental Evaluation

This section presents experimental evaluations at two levels. First, Section 5.6.1 evaluates the RAPid-Learn framework in isolation, comparing its novelty accommodation and sample efficiency against transfer learning baselines on the NovelGridworlds domain introduced in Section 4.3. The primary metrics are adaptation time (T_{adapt}) and post-novelty success rate (S_{post}), following the evaluation protocol established in Section 4.4.5. Second, Section 5.6.3 evaluates the full neurosymbolic architecture on a large-scale systemic evaluation in the Polycraft en-

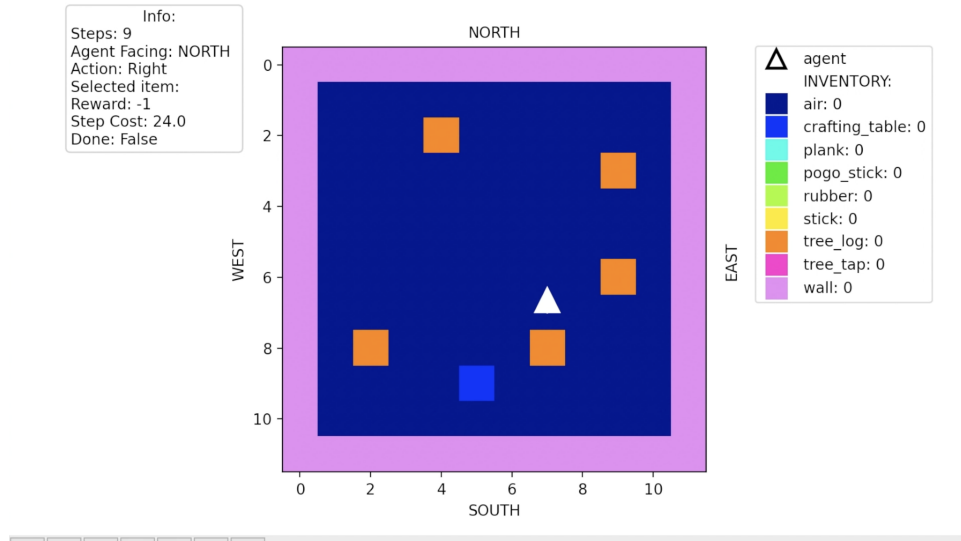


Figure 5.5: Screenshot of the NovelGridworld-Pogostick environment. The agent (triangle) can navigate in the environment and collect items such as trees (shown in yellow). The agent can use the crafting-table to craft complicated items such as tree-tap, sticks etc. Blue spaces represent air blocks through which the agent can navigate, and purple blocks represent walls.

vironment, assessing the integrated capabilities of the architecture across hundreds of novelty scenarios.

5.6.1 RAPid-Learn Evaluation

Experimental Setup

RAPid-Learn was evaluated on a 12×12 NovelGridworlds domain [41] with the pogostick crafting task described in Section 4.3 (shown in Figure 5.5). The agent’s sub-symbolic observation space consists of LiDAR-like sensors emitting beams at $\frac{\pi}{4}$ increments for each entity type (observation size $8 \times |\mathcal{E}|$), augmented with inventory and selected-item vectors. The action space includes navigation actions (`turn-left`, `turn-right`, `move-forward`), interaction actions (`break`, `extract-rubber`), crafting actions, and hierarchical operators (`approach-entity`) implemented by the internal planner. Five novelty scenarios were designed, spanning different types of environmental changes:

1. **Axe-to-break (ATB) hard:** In this novelty, a novel entity *axe* appears in the environment and the agent cannot break *trees* without holding the *axe*. We implement two versions of the novelty, easy and hard. In the easy version, the *axe* is present in the agent’s inventory,

and the agent needs to learn to select the *axe* and use it in front of the *tree* to get *tree-log* into its inventory. In the hard version, the *axe* is present in the environment, and the agent needs to find it to get into its inventory and then use it to break *trees*. We observe that the agent is able to generate executors that can reason about the affordances of new entities in the environment, i.e. learn to pick up the *axe*, select it, and then *break* the *tree* by locating itself next to the *tree*.

2. **Fire-crafting-table (FCT) hard:** In this novelty, the agent cannot access the *crafting-table* since its set on *fire*. Two novel entities *water* and *fire* appears in the environment, and novel actions *spray* and *select-water* are added to the list of available actions. In order to access the *crafting-table* the agent needs to select *water* and use the action *spray* in front of the *crafting-table* in order to put it out from *fire*, and hence successfully access it. In the easy version, the *water* is already present in the agent’s inventory and in the hard version *water* is present in the environment and the agent needs to collect it to use it. The agent’s sub-symbolic representation has information about the status of the *fire* (on/off).
3. **Rubber-tree (RT) hard:** In this novelty, the agent cannot extract *rubber* from the regular trees anymore, but a new entity *rubber-tree* appears in the environment and the agent needs to use the *rubber-tree* for *rubber* extraction. We have two versions of this novelty, in the *easy* version, the agent needs to select the *tree-tap* and standing in front of a *rubber-tree* it needs to use the action *extract-rubber* to extract rubber. In the hard version, the agent’s action space is augmented with a new action called *place-tree-tap*, and in order to get rubber, the agent now needs to place the *tree-tap* in front of the *rubber-tree* and *rubber* can only be extracted when the agent is in front of the placed *tree-tap*. The agent’s knowledge base does not have any *in front of* predicate. The agent’s executor learns the policy to place the new entity of *tree-tap*, locate itself in front of it, and then extract *rubber*. We observe that the agent is successful in synthesizing executors with predicates missing from the knowledge base.
4. **Scrape-plank (SP):** In this novelty, the *break* action has no effect, and a novel action *scrape-plank* is augmented in the agent’s action space. The dynamics of the world changes in a way that the agent can no longer break *trees* to get *tree-logs*, but it can scrape planks from trees and using the action *scrape-plank* in front of a *tree* it gets 4 *planks* into its inventory.

This novelty emphasizes on failure of effects of the operator rather than failure of the action itself. The executor learns a policy to generate *planks*, by utilizing the *scrape-plank* action to move ahead in the plan. Thus, the agent learns to tackle a novel scenario even when there are no new entities in the environment.

5. **ATB + FCT (easy)**: Two novelties are introduced simultaneously—axe-to-break and fire-crafting-table in their easy variants (items are pre-placed in inventory). This tests recovery from multiple independent operator failures.

Baselines

We now describe how the baselines were designed and trained. In early experiments, the pre-novelty task with sparse rewards proved intractable for pure RL approaches. Moreover, the RAPid-Learn agent has access to the full PDDL description of the environment, which heavily biases any comparison in its favor. We therefore employ strong reward shaping to allow the RL baseline to learn the pre-novelty task. A small reward (+50) is given for completing intermediate steps (e.g. crafting sticks or planks) when they are needed for the next major step in the crafting process. Larger rewards are given for the subgoals of crafting the treetap (+200), extracting rubber (+300) and finally crafting the pogostick (+1000). Note that the rewards are only given if the intermediate step is the appropriate thing to do next. I.e. no reward is given for breaking a tree, if the agent already has a treetap and enough sticks and planks in its inventory to craft the pogostick. In this case, a reward would only be given for obtaining the missing ingredient (rubber). Making the reward conditional on the stage within the crafting process was essential in getting the agent to learn the pre-novelty task. In early versions of the reward shaping, we attempted to give reward for completing intermediate steps (approaching a tree, crafting sticks, etc.). This was exploited by the agent, which learned that the reward could be optimized by periodically turning away and re-approaching a tree until the end of its life.

We evaluate their performance to make sure that they are at par with the RAPid-Learn approach (RAPid-Learn, being a planning agent, shows a 100% performance in the pre-novelty scenario). Table 5.1 shows how many timesteps the pre-novelty models have experienced until reaching convergence and their performance on 100 evaluation instances of the environment on

Agent	Pre-novelty	
	Time to learn (timesteps)	Pre-novelty performance (success rate %)
		Mean $\pm$ SD
Actor-Critic Transfer	2.47×10^5	0.95 ± 0.22
Policy Reuse	1.48×10^6	0.95 ± 0.22

Table 5.1: Performance and timesteps trained for the best pre-novelty model that was later used in transfer learning the novelty environments

10 random seeds. Results in Table 5.1 show that both the baseline agents converge to a policy which performs with a success of the task being achieved 95% of the time.

Baseline Transfer Learning The injected novelties alter the shape of the observation and action spaces of the environment. E.g. the Fire crafting table novelty adds new observations for the *water* item in the inventory and the LiDAR-sensor, as well as the *select water* and *spray* actions. In order to allow transferring the model from pre-novelty to post-novelty environments, we pad the observation and action spaces of all environments (pre- and post-novelty) with placeholder elements such that all of them have the same shape. Observation placeholders simply always return zero. When the model chooses to perform a placeholder action, the action actually passed on to the environment is always the *approach treealog*.

Experiments & Results

In this section, we describe the experiments we ran to answer the following questions. (Q1) How does the rapid-learn approach compare to the adapted transfer learning approaches in terms of the adaptation efficiency, and post adaptation success rates, compared across 5 novelty scenarios (Q2) How does each variant of knowledge-guided exploration component of Rapid-Learn fair, in terms of the adaptation efficiency across all novelty scenarios.

Three variants of RAPid-Learn were compared: *RAPid-KGE-UCB* (with upper confidence bound action biasing), *RAPid-KGE-UAB* (with uniform action biasing), and *RAPid-EG* (with standard ϵ -greedy exploration). Two transfer learning baselines were used: *Actor-Critic Trans-*

Novelty	Agent	T_{adapt} (timesteps)	Post-novelty SR
RT hard	RAPid-KGE-UCB	$(2.85 \pm 2.15) \times 10^4$	0.95 ± 0.01
	RAPid-KGE-UAB	$(3.28 \pm 1.51) \times 10^4$	0.93 ± 0.03
	RAPid-EG	$(3.73 \pm 2.08) \times 10^4$	0.91 ± 0.04
	Actor-Critic Tr.	did not converge	N/A
	Policy Reuse	did not converge	N/A
ATB+FCT easy	RAPid-KGE-UCB	$(4.02 \pm 1.82) \times 10^3$	0.97 ± 0.01
	RAPid-KGE-UAB	$(4.56 \pm 3.43) \times 10^3$	0.97 ± 0.02
	RAPid-EG	$(6.93 \pm 4.41) \times 10^3$	0.98 ± 0.01
	Actor-Critic Tr.	$(17.6 \pm 5.13) \times 10^4$	0.93 ± 0.02
	Policy Reuse	$(8.08 \pm 2.38) \times 10^5$	0.92 ± 0.04
ATB hard	RAPid-KGE-UCB	$(2.35 \pm 1.27) \times 10^3$	0.95 ± 0.01
	RAPid-KGE-UAB	$(13.0 \pm 4.89) \times 10^2$	0.85 ± 0.07
	RAPid-EG	$(14.9 \pm 8.65) \times 10^2$	0.84 ± 0.03
	Actor-Critic Tr.	$(9.80 \pm 4.75) \times 10^4$	0.90 ± 0.05
	Policy Reuse	$(5.57 \pm 2.59) \times 10^5$	0.92 ± 0.04
FCT hard	RAPid-KGE-UCB	$(8.61 \pm 4.73) \times 10^3$	0.98 ± 0.00
	RAPid-KGE-UAB	$(4.40 \pm 1.96) \times 10^3$	0.99 ± 0.00
	RAPid-EG	$(5.12 \pm 2.84) \times 10^3$	0.98 ± 0.00
	Actor-Critic Tr.	$(16.7 \pm 3.99) \times 10^4$	0.90 ± 0.04
	Policy Reuse	$(7.90 \pm 2.70) \times 10^5$	0.94 ± 0.03
SP	RAPid-KGE-UCB	$(6.82 \pm 3.83) \times 10^3$	0.86 ± 0.03
	RAPid-KGE-UAB	$(2.01 \pm 1.01) \times 10^3$	0.87 ± 0.03
	RAPid-EG	$(17.2 \pm 7.38) \times 10^2$	0.96 ± 0.01
	Actor-Critic Tr.	$(6.86 \pm 3.89) \times 10^4$	0.93 ± 0.03
	Policy Reuse	$(19.3 \pm 9.76) \times 10^4$	0.94 ± 0.03

Table 5.2: RAPid-Learn results across five novelty scenarios. T_{adapt} : time steps to adaptation convergence. Post-novelty SR: success rate on 100 evaluation episodes. Best performing agent highlighted per scenario. In all scenarios, RAPid-Learn adapts one to two orders of magnitude faster than transfer learning baselines.

fer [18], which transfers both the policy and value function through PPO [117], and *Policy Reuse* [40], which transfers the learned policy. Both baselines were pre-trained on the base task with dense reward shaping until convergence ($> 96\%$ success rate). The reinforcement learning component in all RAPid-Learn variants used the policy gradient algorithm [137]. All results are reported as mean $\pm$ standard deviation over 10 independent trials.

Sample efficiency. Table 5.2 and Figure 5.6 show that RAPid-Learn consistently adapts one to two orders of magnitude faster than the transfer learning baselines across all novelty scenarios. In the axe-to-break hard novelty, RAPid-KGE-UCB adapts in approximately 2.35×10^3 time steps compared to 9.80×10^4 for actor-critic transfer and 5.57×10^5 for policy reuse—improvements of roughly $40\times$ and $230\times$, respectively. The advantage is most pronounced in the rubber-tree hard novelty, where both baselines fail to converge entirely, while all RAPid-Learn variants converge within 3.7×10^4 time steps. The sample efficiency advantage arises from two sources: the symbolic planner constrains the learning problem to a specific subgoal (the failed operator’s effects), and knowledge-guided exploration focuses the agent on actions and states likely related to the novelty.

Effectiveness of knowledge-guided exploration. The relative performance of the three RAPid-Learn variants reveals the contribution of knowledge-guided exploration. In complex novelties such as rubber-tree hard, where the agent must discover a spatial relation not in its symbolic knowledge, KGE-UCB and KGE-UAB provide measurable improvements in convergence speed over standard ϵ -greedy exploration. In simpler novelties such as scrape-plank, where the solution involves a single novel action, the advantage of knowledge-guided exploration is less pronounced, and ϵ -greedy exploration can occasionally converge faster. This pattern is consistent with the expectation that knowledge-guided exploration is most valuable when the exploration space is large and the solution requires interacting with novel entities.

Multiple operator failures. The ATB+FCT easy scenario demonstrates that RAPid-Learn handles multiple simultaneous novelties effectively. The agent learns two independent executors—one for each failed operator—and achieves a 97% post-novelty success rate in approximately 4×10^3 time steps. The rubber-tree hard scenario further demonstrates recovery from cascading

failures: after learning to extract rubber from the rubber-tree, the agent discovers that it can no longer break the now-unbreakable rubber-tree, triggering a second executor discovery process. Despite this implicit multiple-failure structure, RAPid-Learn converges successfully while the baselines fail entirely.

Statistical significance. Unpaired t -tests between the best-performing RAPid-Learn variant and each baseline confirm that the differences in post-novelty success rate are statistically significant ($p < 0.01$) in all scenarios except scrape-plank versus policy reuse ($p = 0.06$), where the baselines achieve comparable final performance albeit at substantially higher sample cost.

Ablation: hierarchical actions. An ablation study removing hierarchical actions (the `approach` operator) from the learner’s action space shows that their inclusion substantially improves sample efficiency, particularly for complex novelties. Without hierarchical actions, RAPid-EG fails to converge on the fire-crafting-table hard novelty, while RAPid-KGE-UCB still converges but requires more episodes. This confirms that composing learned behaviors with existing executors is a key advantage of the hybrid architecture.

5.6.2 Domain PDDL

Below is the original PDDL domain given to RAPid-Learn for the pogostick-crafting task inspired by Minecraft in novelgridworlds.

```
(:requirements :typing :strips :fluents)
(:types
  wall - physobj
  entity - physobj
  plank - physobj
  crafting_table - physobj
  tree_tap - physobj
  var - object
  rubber - physobj
  physobj - physical
  actor - physical
  pogo_stick - physobj
  stick - physobj)
```


Methods	p-value	Statistical significance
Axe to Break (ATB) + Fire Crafting Table (FCT) - Easy		
RAPid-KGE-UCB $\leftrightarrow$ Actor-critic	0.0062	Yes
RAPid-KGE-UCB $\leftrightarrow$ Policy Reuse	0.0012	Yes
Axe To Break (ATB) - Hard		
RAPid-KGE-UCB $\leftrightarrow$ Actor-critic	0.00336	Yes
RAPid-KGE-UCB $\leftrightarrow$ Policy Reuse	0.0012	Yes
Fire Crafting Table (FCT) - Hard		
RAPid-KGE-UAB $\leftrightarrow$ Actor-critic	< 0.0001	Yes
RAPid-KGE-UAB $\leftrightarrow$ Policy Reuse	< 0.0001	Yes
Scrape Plank (SP)		
RAPid-KGE-EG $\leftrightarrow$ Actor-critic	0.0077	Yes
RAPid-KGE-UCB $\leftrightarrow$ Policy Reuse	0.0608	No

Table 5.3: Results of unpaired t-test between best performing RAPid-Learn and transfer learning baselines

```

tree_log - physobj
air - physobj
)

(:predicates
  (holding ?v0 - physobj)
  (floating ?v0 - physobj)
  (facing ?v0 - physobj)
)

(:functions
  (world ?v0 - object)
  (inventory ?v0 - object)
)

(:action approach

```

```

:parameters    (?physobj01 - physobj ?physobj02 - physobj )
:precondition  (and
    (>= ( world ?physobj02) 1)
    (facing ?physobj01)
)
:effect  (and
    (facing ?physobj02)
    (not (facing ?physobj01))
)
)

(:action crafttree_tap ;
:parameters    ()
:precondition  (and
    (>= ( inventory plank) 4)
    (>= ( inventory stick) 1)
    (facing crafting_table)
)
:effect  (and
    (increase ( inventory tree_tap) 1)
    (decrease ( inventory plank) 5)
    (decrease ( inventory stick) 1)
)
)

(:action craftplank
:parameters    ()
:precondition  (>= ( inventory tree_log) 1)
:effect  (and
    (increase ( inventory plank) 4)
    (decrease ( inventory tree_log) 1)
)
)

(:action break
:parameters    ()

```

```

      :precondition (and
        (facing tree_log)
        (not (floating tree_log))
      )
      :effect (and
        (facing air)
        (not (facing tree_log))
        (increase ( inventory tree_log) 1)
        (increase ( world air) 1)
        (decrease ( world tree_log) 1)
      )
    )

(:action craftstick
  :parameters ()
  :precondition (>= ( inventory plank) 2)
  :effect (and
    (increase ( inventory stick) 4)
    (decrease ( inventory plank) 2)
  )
)

(:action extractrubber
  :parameters ()
  :precondition (and
    (>= ( inventory tree_tap) 1)
    (facing tree_log)
    (holding tree_tap)
  )
  :effect (and
    (increase ( inventory rubber) 1)
  )
)

(:action craftpogo_stick
  :parameters ()

```

```


```

:precondition (and
 (>= (inventory plank) 2)
 (>= (inventory stick) 4)
 (>= (inventory rubber) 1)
 (facing crafting_table)
)
:effect (and
 (increase (inventory pogo_stick) 1)
 (decrease (inventory plank) 2)
 (decrease (inventory stick) 4)
 (decrease (inventory rubber) 1)
)
)

(:action select
 :parameters (?physobj01 - physobj)
 :precondition (and
 (>= (inventory ?physobj01) 1)
 ; (holding air)
)
 :effect (and
 (holding ?physobj01)
 (not (holding air))
)
)
)

```


```

Listing 5.1: Domain PDDL for the pogostick-crafting task.

5.6.3 Integrated Architecture Evaluation

To evaluate the full neurosymbolic architecture, a large-scale systemic evaluation was conducted in the Polycraft World domain [46], a partially observable, multi-agent environment based on Minecraft. The evaluation spanned 288 novelties across 10 categories (object, attribute, representation, actor, action, relation, interaction, environment, goal, event), with 864 novelty

tournaments. Of these, 216 novelties were known during architecture development; the remaining 72 were *concealed novelties* designed by a separate evaluation team and never shared with the architecture developers.

Four agent configurations were evaluated: a *Base* agent using only symbolic inference and symbolic novelty exploration; *Base+Agent Model*, augmenting the base with neural behavioral modeling of other actors; *Base+Executor Learner*, augmenting the base with the RAPid-Learn executor learner; and a combined *Base+Agent Model+Executor Learner* configuration.

Agent	FN _{CDT} ↓	CDT% ↑	FP% ↓	NRP% ↑	GA% ↑
Base	0.69	83.3	3.2	73.5	64.4
Base+Agent Model	0.58	86.1	6.1	72.4	64.0
Base+Executor Learner	0.76	84.7	2.3	74.2	66.6
Base+AM+EL	0.39	88.7	1.6	68.1	61.9

Table 5.4: Overall systemic evaluation results in Polycraft across 288 novelties. FN_{CDT}: average false negative episodes before detection; CDT%: correctly detected trials; FP%: false positive rate; NRP%: novelty reaction performance; GA%: goal achieved rate. The executor learner improves accommodation (NRP%, GA%), while the agent model improves detection (CDT%, FN_{CDT}).

Novelty detection. Table 5.4 shows that configurations including the agent model achieve better novelty detection performance, with the combined configuration reaching 88.7% correctly detected trials and a false negative rate of 0.39 episodes. The agent model is particularly effective for action and goal novelties, where actors exhibit changed behavior that symbolic inference alone cannot detect. The vision model (evaluated separately) achieved 84.9% true positive rate with 85.1% precision on concealed visual novelties, demonstrating the potential of subsymbolic novelty detection, though its higher false positive rate (34.1% at the 95% TPR threshold) limited its integration into the task-oriented agent.

Novelty accommodation. Figure 5.8 breaks down performance by novelty category. The Base+Executor Learner configuration achieves the highest novelty accommodation performance, with 66.6% goal achievement rate and 74.2% novelty reaction performance, outperforming all other configurations. The executor learner proves most valuable in attribute novelties (e.g., learning to break birch trees that fail to produce rubber) and concealed novelties where symbolic exploration strategies are insufficient. A notable example is the “different items in chests”

novelty, where the base agent fails because it assumes all chests contain identical items, while the executor learner discovers a policy to open all chests and collect diverse ingredients.

Interaction effects. The combined Base+Agent Model+Executor Learner configuration achieves the best detection performance but lower accommodation scores than the Base+Executor Learner alone. This arises from an unforeseen interaction: the executor learner’s exploration changes the environment state, which triggers false novelty detections from the agent model, leading to unnecessary exploration that hinders task performance. This finding highlights the challenge of integrating multiple adaptive components and suggests that coordination mechanisms between detection and accommodation modules are an important direction for future work.

Performance on concealed novelties. The executor learner showed a measurable improvement on concealed novelties (68.2% NRP and 61.3% GA for the Base+Executor Learner, compared to 64.8% NRP and 57.1% GA for the Base agent), confirming that the learning-based accommodation generalizes to novelty scenarios not anticipated during development. This is a strong validation of the architecture’s open-world capabilities: the reinforcement learning component discovers recovery behaviors for novelties that neither the system designers nor the symbolic exploration strategies could have foreseen.

5.7 Summary and Future Directions

This chapter presented two contributions for enabling autonomous agents to adapt to novelty in open-world environments. The RAPid-Learn framework formalizes the integration of symbolic planning and reinforcement learning through the concept of integrated planning tasks, defines the executor discovery problem as the core challenge of novelty recovery, and introduces knowledge-guided exploration strategies that exploit domain knowledge to achieve sample-efficient adaptation. The neurosymbolic cognitive architecture embeds RAPid-Learn within a broader system that handles the full lifecycle of novelty—from detection through characterization to accommodation—by combining symbolic inference, neural inference, and reinforcement learning in a cascading recovery pipeline.

The experimental evaluations demonstrate three key findings. First, RAPid-Learn achieves one to two orders of magnitude faster adaptation than transfer learning baselines by decomposing the global recovery problem into focused subproblems defined by failed operators. Second, knowledge-guided exploration provides measurable benefits for complex novelties that require the agent to interact with novel entities or discover spatial relations not present in its symbolic model. Third, the integrated architecture achieves robust performance across 288 novelties spanning 10 categories, including concealed novelties not anticipated during system development, confirming that the combination of symbolic reasoning and learning-based recovery generalizes to a broad range of open-world changes.

Several limitations of the current approach point to important directions for future work.

Failure-driven adaptation. As observed in Chapter 4, the failure-driven nature of RAPid-Learn means that beneficial novelties that do not cause execution failures go undetected and unexploited. The chest novelty in the NovelGym evaluation (Section 4.4.8) illustrates this clearly: hybrid agents never discovered the shortcut because their plans continued to succeed. Extending the framework to proactively detect and exploit beneficial opportunities—for example, through periodic re-planning with updated models—would address this blind spot.

Fixed goal assumption. The current framework assumes a fixed goal state throughout the adaptation process. In realistic open-world settings, goals themselves may need to be revised when novelty renders them unachievable or when new opportunities arise. Integrating goal reasoning [5] with the executor discovery mechanism would enable more flexible responses to environmental changes.

Exploration cost in physical environments. The reinforcement learning component of RAPid-Learn requires trial-and-error interaction with the environment, which is acceptable in simulation but potentially costly or dangerous in physical robotic settings. Reducing the number of exploratory interactions—through model-based reinforcement learning, sim-to-real transfer, or more informed exploration strategies—is essential for deploying these methods on real robots. This challenge motivates the work presented in Chapter 6, which develops reinforcement learning approaches for robotic manipulation that emphasize generalization across

objects and embodiments, reducing the need for environment-specific exploration.

Symbolic-to-subsymbolic grounding. The current architecture relies on a domain interface that provides pre-processed symbolic state descriptions. In real-world settings, the agent would need to ground these descriptions in raw sensory data—a challenging perception problem that interacts with novelty detection (e.g., visually recognizing a novel object and inferring its symbolic type). Tighter integration of the visual novelty detection capabilities with the symbolic reasoning components would enable end-to-end novelty handling from raw observations.

Coordination between adaptive components. The systemic evaluation revealed that the interaction between the agent model and the executor learner can be counterproductive: the learner’s exploration triggers false novelty detections from the agent model, leading to unnecessary exploration. Developing mechanisms for coordinating multiple adaptive components—for example, by suppressing novelty detection during active learning phases, or by sharing information about the source of environmental changes between components—is an important engineering and research challenge for complex novelty-handling architectures.

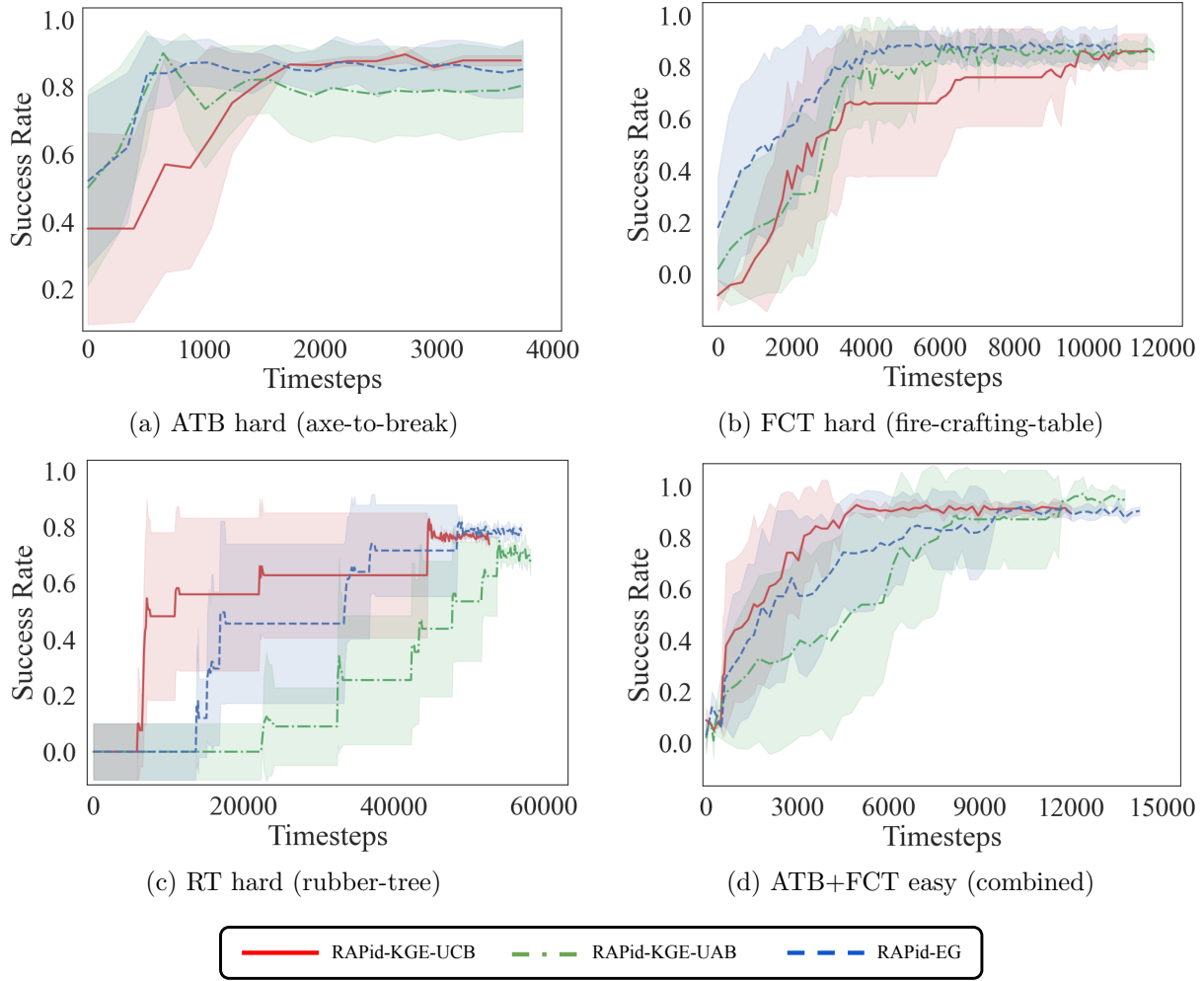


Figure 5.6: Learning curves (success rate vs. training time steps) for four of the five novelty scenarios. All RAPid-Learn variants (KGE-UCB, KGE-UAB, EG) converge one to two orders of magnitude faster than the transfer learning baselines (Actor-Critic Transfer and Policy Reuse). Shaded regions denote one standard deviation over 10 independent trials. Transfer learning baselines did not converge on the RT hard scenario within the evaluation budget and are therefore omitted from that panel.

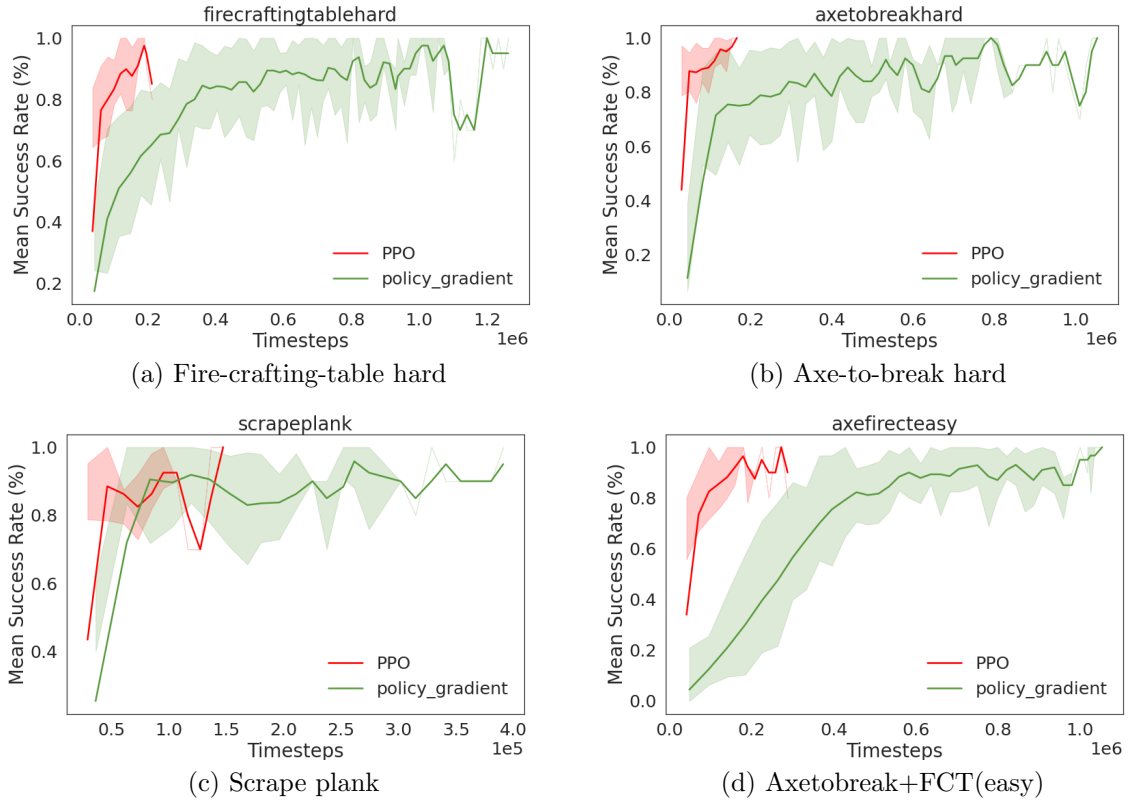


Figure 5.7: Baseline learning curves on the four novelties. Green curve shows the actor-critic transfer, and the red curve shows policy-reuse. Each plot demonstrates the performance during the novelty adaptation phase.

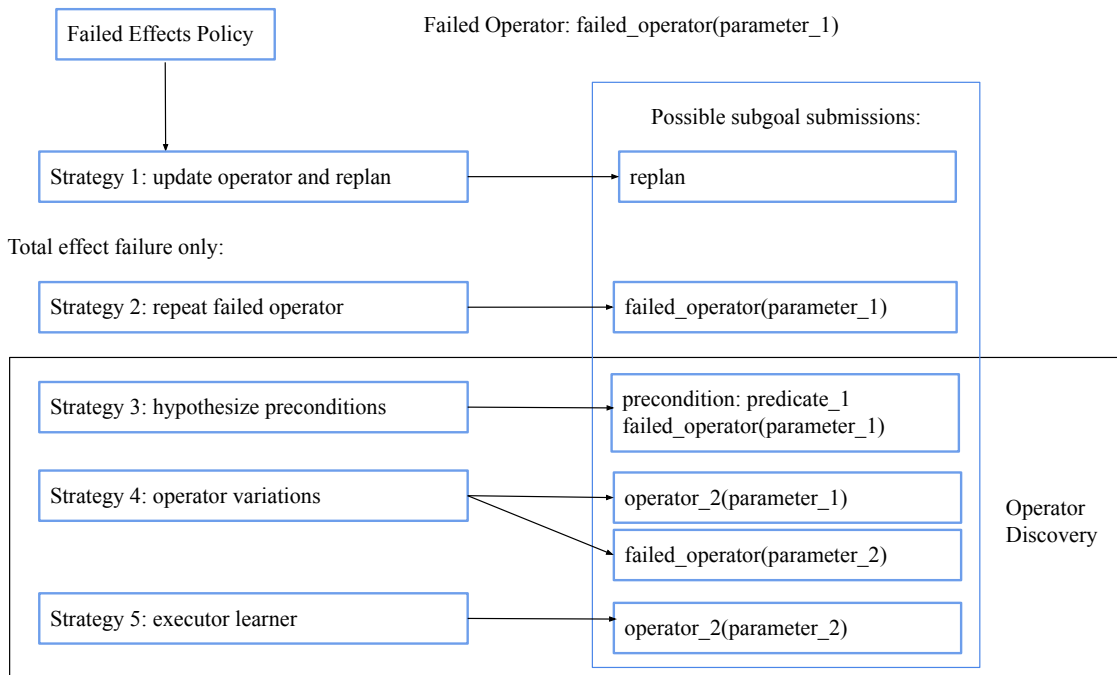


Figure 5.8: Breakdown of novelty accommodation performance across novelty categories in the Polycraft systemic evaluation. Each bar shows the goal achievement rate for the four agent configurations under each novelty type. The executor learner (Base+EL) provides the most consistent improvement on attribute, action, and concealed novelties, where symbolic recovery strategies are insufficient and learning-based executor discovery is essential.

Chapter 6

Generalization via Interaction Abstractions¹

6.1 Motivation

The preceding chapters addressed how agents can detect, evaluate, and recover from novelty in open-world environments. Chapter 4 introduced benchmark environments and metrics for studying agent behavior under mismatch, and Chapter 5 developed hybrid planning–learning methods that exploit action abstractions to achieve efficient adaptation when execution fails. Both contributions operate within the framework established in Chapter 2: novelty arises from mismatch between the agent’s internal models and the environment, and structured abstractions determine whether this mismatch can be managed effectively.

This chapter addresses the second dimension of the dissertation’s central claim: that structured abstractions over interactions enable learned behaviors to *generalize* across variations in objects, physical parameters, and robotic embodiment. While Chapter 5 showed that action abstractions support efficient adaptation *after* failure, the work in this chapter asks a complementary question: can interaction abstractions be designed so that learned skills remain valid

¹This chapter is based on: S. Fang, W. Gao, S. Goel, C. Thierauf, M. Scheutz, and J. Sinapov, “FLEX: A Framework for Learning Robot-Agnostic Force-based Skills Involving Sustained Contact Object Manipulation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025; and S. Goel, T. Duggan, T. Maranga, M. Scheutz, and J. Sinapov, “Pushing Heavy Objects: Learning Object-Centric Force Policies for Sustained-Contact Manipulation,” submitted to *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.

before failure occurs—that is, so that a policy trained on one object or robot transfers to new objects and robots without retraining?

The challenge is rooted in how robotic manipulation policies represent their interaction with the world. As discussed in Section 2.4.3, the standard approach in robot learning defines policies over robot-centric representations: joint angles, end-effector poses, or motor torques. These representations tightly couple the learned behavior to a specific embodiment and control interface. A policy trained to open a drawer with a Panda robot encodes not just the task-relevant forces and motions, but also the Panda’s specific kinematics, joint limits, and control dynamics. The result is a policy that solves the task on the training platform but fails when the robot changes, the object changes, or the physical parameters (mass, friction, dimensions) change—even when the underlying task physics remain the same.

This limitation is not merely a practical inconvenience. It reflects a representational failure: the policy’s state and action spaces do not separate the aspects of the interaction that are invariant across variations (the forces and motions required to manipulate the object) from those that are contingent on the training instance (the robot’s kinematics, the specific object’s dimensions, the particular friction coefficient). In the language of Section 2.4, the representation fails to abstract away the agent, and therefore does not capture the natural complexity of the task [75].

An important clarification is in order regarding the relationship between generalization and novelty. The variations studied in this chapter—different objects, different robots, different physical parameters—are anticipatable in the sense that the representation is explicitly designed to be invariant to them. This differs from the unanticipated novelties addressed in Chapter 5, where the agent encounters changes it could not have foreseen. However, the connection to the dissertation’s central claim is direct: the right choice of abstraction can *preempt* certain classes of mismatch entirely, ensuring that variations which would constitute novelty under a robot-centric representation are absorbed by the invariant structure of the force-space representation without any adaptation being required. In this sense, interaction abstractions and action abstractions play complementary roles: interaction abstractions can preempt the need for adaptation across anticipated variation, while action abstractions provide the machinery for recovery when genuinely unanticipated changes occur. Together, they define a spectrum of

responses to open-world variation, from preemptive invariance to reactive adaptation.

This chapter develops an alternative: *object-centric force-space representations* for robotic manipulation. The core idea is to define policies that operate in the space of forces applied to the object, conditioned on object-relative state descriptions, rather than in the space of robot commands. By representing interaction at the level of the object’s dynamics rather than the robot’s control, the learned behavior captures the physical invariants of the task—what forces to apply, where on the object to apply them, and what motion results—while remaining agnostic to the specific robot that executes them. This constitutes a structured interaction abstraction in the sense of Section 2.4.3: the representation is organized around the causally relevant aspects of the manipulation task, creating a buffer between the learned policy and the variability of objects and embodiments.

We instantiate this idea through two complementary contributions that address different classes of sustained-contact manipulation:

- **FLEX: Force-based learning for articulated object manipulation** (Section 6.3). FLEX learns object-centric force policies for manipulating articulated objects—drawers, doors, microwaves, dishwashers—whose motion is constrained by prismatic or revolute joints [24]. Policies are trained entirely in simulation without modeling any robot, using only the object’s joint configuration and the applied force as state and action. An interactive joint parameter estimation procedure enables deployment on physical robots, and an operational space controller translates the learned forces into robot commands. FLEX demonstrates generalization across unseen objects of the same joint type and direct transfer to three different robotic platforms (Kinova, Panda, UR5) without retraining.
- **Pushing heavy objects: Force policies for rigid body transport** (Section 6.4). This work extends the object-centric force-space approach to the transport of rigid objects along prescribed paths under unknown mass and friction conditions [45]. A policy is trained in simulation to output planar boundary forces conditioned on a normalized contact-resistance descriptor, with domain randomization over mass and friction. A lightweight impedance-based probing routine estimates the interaction regime at deployment, enabling zero-shot sim-to-real transfer. Experiments on a Boston Dynamics Spot mobile manipulator demonstrate

robust transport of heavy payloads (up to 12.5 kg) along curved trajectories.

Together, these contributions demonstrate that object-centric force-space representations provide a practical and principled pathway to generalizable manipulation. FLEX shows that the approach generalizes across objects and robots for articulated manipulation; pushing heavy objects shows that it generalizes across physical parameters and transfers zero-shot from simulation to a legged mobile manipulator for rigid body transport. Both instantiate the interaction abstraction framework developed in Section 2.4.3 and provide empirical evidence for the claim that structured abstractions over interactions enable generalization under the kind of variation that characterizes open-world environments.

6.2 Related Work

This section reviews prior work related to the force-based learning methods developed in this chapter. The discussion is organized around four themes: robot learning for sustained-contact manipulation, object-centric representations for manipulation, model-based and control-theoretic approaches to pushing, and sim-to-real transfer under contact uncertainty.

Robot learning for sustained-contact manipulation. Sustained-contact manipulation—tasks requiring continuous force application throughout the interaction, such as pushing, sliding, or operating articulated mechanisms—poses challenges due to high-dimensional action spaces and extended task duration [79]. Reinforcement learning approaches have been applied to contact-rich manipulation [48, 66], but typically require extensive training data and produce policies tightly coupled to specific robot platforms and task configurations. Simulated environments such as MuJoCo [133], Robosuite [145], and SAPIEN [139] have improved training efficiency but policies still struggle with stable interactions throughout extended manipulation and managing complex state-action dynamics [23].

Recent work has sought to simplify reinforcement learning for sustained contact by decoupling high-level policies from low-level motion planning. Abbatematteo et al. [2] introduce composable interaction primitives that structure the policy class for efficient learning of sustained-contact skills. Lu et al. [90] decouple object dynamics from robot control to enhance

generalization. Nasiriany et al. [103] augment reinforcement learning with behavior primitives, and Rosen et al. [109] combine structured RL with task and motion planning. While these approaches reduce action-space complexity, they remain in robot-centric or end-effector spaces, limiting transferability across platforms. The methods in this chapter take a further step by moving policy learning entirely into object-centric force space, eliminating robot-specific dynamics from the learning problem altogether.

Object-centric representations for manipulation. A growing body of work explores object-centric representations that focus on the object being manipulated rather than the robot, aiming to improve generalization across tasks and platforms [79]. Methods leveraging 3D perception have produced object-centric actions [97], trajectories [138], and affordances [35] that enhance skill transferability. For articulated objects specifically, point-cloud-based methods can infer joint configurations [142] and predict articulation flow to guide manipulation [22]. UMP-Net [140] generates closed-loop $SE(3)$ action trajectories from RGB-D inputs, enabling generalization across unseen articulation structures without explicit joint modeling. GAMMA [142] generalizes articulation modeling and manipulation from partial point clouds, using pre-trained perception to infer joint parameters.

While these approaches simplify robot learning through object-centric action representations, they typically remain in pose space and rely on robot-specific execution, limiting transferability for sustained-contact tasks that require stable force application. The work in this chapter differs by operating entirely in force space, where the policy’s actions are forces applied to the object rather than robot motions, and by decoupling the policy from any specific robotic platform during both training and transfer.

Learning-based pushing and mobile manipulation. Reinforcement learning has been applied to planar pushing under contact-rich dynamics. Ferrandis et al. [27] propose RL with multimodal categorical exploration to better cover hybrid contact behaviors. Bergmann et al. [12] study precision-focused RL for pushing to improve goal accuracy. Other work explores sensing combinations such as vision and proprioception to improve pushing under varying object and contact properties [19, 135]. For mobile manipulators and legged systems, Dadiotis

et al. [20] present model-free constrained RL for object goal pushing with a mobile manipulator, reporting robustness across object properties. For multi-robot settings, Tang et al. [126] address collaborative planar pushing of polytopic objects with coordinated contact forces. NMPC planners have also been developed for mobile robots maintaining stable contact while satisfying nonholonomic constraints [125]. Most learning approaches operate in robot-centric action spaces, with generalization addressed through architecture design, randomization, or adaptation. This motivates the alternative abstractions developed in this chapter that aim to reduce embodiment dependence while retaining robustness in sustained contact.

Model-based pushing and control-theoretic approaches. Planar pushing has been studied extensively through analytical models and control-theoretic formulations. Early work by Mason [93] and Lynch and Mason [91] characterized the quasi-static regime of frictional contact during pushing. Limit-surface models [47] compactly represent frictional force-moment pairs during planar sliding. More recently, optimization-based methods have cast pushing as constrained optimal control, handling hybrid contact modes explicitly. Hogan et al. [58], Hogan and Rodriguez [57] propose hybrid MPC formulations for reactive planar non-prehensile manipulation, and contact-implicit trajectory optimization enables planners to reason directly over contact interactions [16]. Impedance control [59] provides a foundational framework for interaction control, with whole-body impedance formulations extending compliance to mobile manipulators [21]. These approaches are physically grounded and interpretable but often rely on accurate contact models and nontrivial online optimization, limiting their applicability when object properties are unknown—precisely the setting addressed by the learning-based methods in this chapter.

Sim-to-real transfer under contact uncertainty. A central challenge in manipulation is the reality gap induced by uncertain contact parameters and unmodeled interaction phenomena. Domain randomization [132, 107] addresses this by training over diverse simulated conditions so that real-world variations appear in-distribution at deployment. For non-prehensile manipulation, Lowrey et al. [89] demonstrate sim-to-real transfer of learned pushing policies using carefully identified simulators and model ensembles. Data-driven contact models have been

shown to outperform analytical counterparts in planar contact settings [25], and broader surveys characterize how modeling assumptions and uncertainty-handling strategies shape pushing performance [122].

The methods developed in this chapter adopt a complementary strategy: rather than training robot-centric policies under randomized conditions, they learn in an object-centric force space where the robot is entirely absent from the training environment. Domain randomization is applied to the *object’s* physical parameters (mass, friction, joint configurations, dimensions) rather than to the robot’s dynamics. This separation means that sim-to-real transfer requires only a mapping from learned forces to robot commands—realized through operational space control (FLEX) or compliant impedance control (pushing heavy objects)—rather than bridging a full embodiment gap.

Gap addressed by this chapter. Across these lines of work, robot learning for sustained-contact manipulation faces a common tension: policies trained in robot-centric action spaces achieve strong performance on specific platforms but do not transfer; model-based methods offer physical interpretability but require accurate contact parameters; and sim-to-real techniques improve robustness but remain coupled to specific embodiments. The methods in this chapter resolve this tension through object-centric force-space learning: by abstracting interaction to the level of forces and object dynamics, learned policies generalize across objects, physical parameters, and robotic platforms. This constitutes the empirical instantiation of the interaction abstraction framework developed in Section 2.4.3.

6.3 FLEX: Force-Based Learning for Articulated Object Manipulation

This section presents FLEX (Force-based Learning for EXtended Manipulation) [24], a framework for learning object-centric manipulation policies in force space for articulated objects such as drawers, doors, microwaves, and dishwashers. The key idea is to decouple policy learning from robot-specific dynamics entirely: policies are trained in simulation by applying forces directly to objects, without modeling any robot. At deployment, the learned forces are transferred to

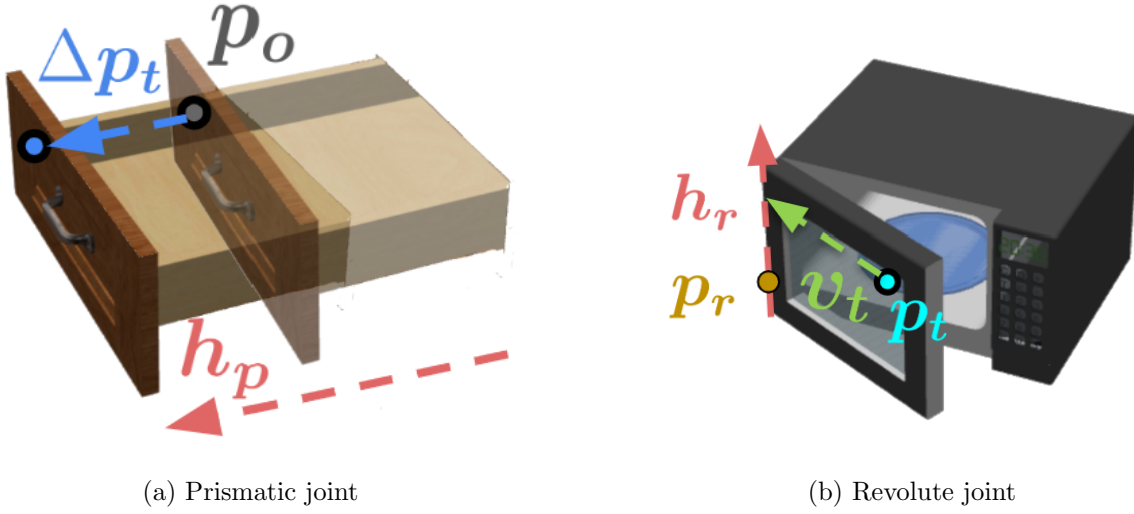


Figure 6.1: Illustration of the two joint configurations used in FLEX. (a) A prismatic joint allows linear motion along a fixed axis h_p . (b) A revolute joint allows rotational motion around an axis h_r with radius r . The state representation for each joint type captures the joint axis direction and the current displacement or projected position, providing an object-centric description independent of robot kinematics.

physical robots through an operational space controller, enabling direct cross-platform transfer without retraining.

6.3.1 Articulated Object Parametrization

Articulated objects consist of multiple rigid bodies (links) connected by joints that permit relative motion. FLEX focuses on objects with a single degree of freedom, either rotational (revolute) or translational (prismatic), which covers a wide range of common household articulated objects.

A *prismatic joint* allows linear motion along a fixed axis. The position of a point p along the joint is defined as $p = p_0 + k \cdot h_p$, where $p_0 \in \mathbb{R}^3$ is the initial position, $h_p \in \mathbb{R}^3$ is the unit vector along the joint axis, and $k \in \mathbb{R}$ is the scalar displacement. A *revolute joint* allows rotational motion around a fixed axis. The position of a point p on a revolute joint satisfies $\|(p - p_r) - [(p - p_r) \cdot h_r]h_r\| = r$, where $p_r \in \mathbb{R}^3$ is the joint origin, $h_r \in \mathbb{R}^3$ is the rotation axis, and r is the radius of the circular motion.

6.3.2 MDP Formulation

The manipulation of articulated objects is formulated as a reinforcement learning problem using two MDPs: one for prismatic joints, $M_p = \langle S_p, A, T_p, R, \gamma \rangle$, and one for revolute joints, $M_r = \langle S_r, A, T_r, R, \gamma \rangle$.

The state spaces are defined in terms of the object’s joint-specific information. For prismatic objects, the state $s_t^p = (h_p, \Delta p_t)$ consists of the unit vector h_p representing the joint axis direction and the displacement $\Delta p_t = p_t - p_0$ between the current force application point and the initial position. For revolute objects, the state $s_t^r = (h_r, v_t)$ consists of the joint axis direction h_r and the projection $v_t = (p_t - p_r) - [(p_t - p_r) \cdot h_r]h_r$ of the current force point onto the plane perpendicular to the joint axis.

The action space is shared across both MDPs and consists of 3D force vectors: $A = \{F \in \mathbb{R}^3 \mid \|F\| \leq \eta\}$, where η is the maximum allowable force. At each time step, the policy selects an action $a_t \in A$ representing the force applied at the point of contact. The reward function encourages efficient manipulation by maximizing object displacement with minimal applied force, plus a large positive reward upon reaching the goal state:

$$R(s_t, a_t, s_{t+1}) = \begin{cases} \|\Delta x\| \cos(\theta_{a_t, \Delta x}) + \mathbb{I}(s_{t+1} = s_{\text{goal}})R_{\text{goal}} & \text{if } \Delta q \geq 0, \\ -\|\Delta x\| \cos(\theta_{a_t, \Delta x}) & \text{otherwise,} \end{cases}$$

where $\theta_{a_t, \Delta x}$ is the angle between the applied force and the object’s movement direction, Δq is the change in joint configuration, and R_{goal} is the goal reward.

The policies π_p and π_r are learned using Twin Delayed Deep Deterministic Policy Gradient (TD3) [32], with the actor network predicting force direction $\pi_{\text{dir}}(s_t)$ and magnitude $\pi_{\text{mag}}(s_t) \in [0, 1]$ separately: $a_t = \pi_{\text{dir}}(s_t) \cdot \pi_{\text{mag}}(s_t) \cdot \eta$. During training, contact points are randomly sampled from accessible regions of the object (handles, front panels), ensuring that the learned policies are contact-agnostic.

A critical feature of this formulation is that no robot is present in the training environment. The simulation consists only of the articulated object and the applied force. This eliminates robot-specific kinematics, collision dynamics, and control from the learning problem, producing policies that are inherently robot-agnostic.

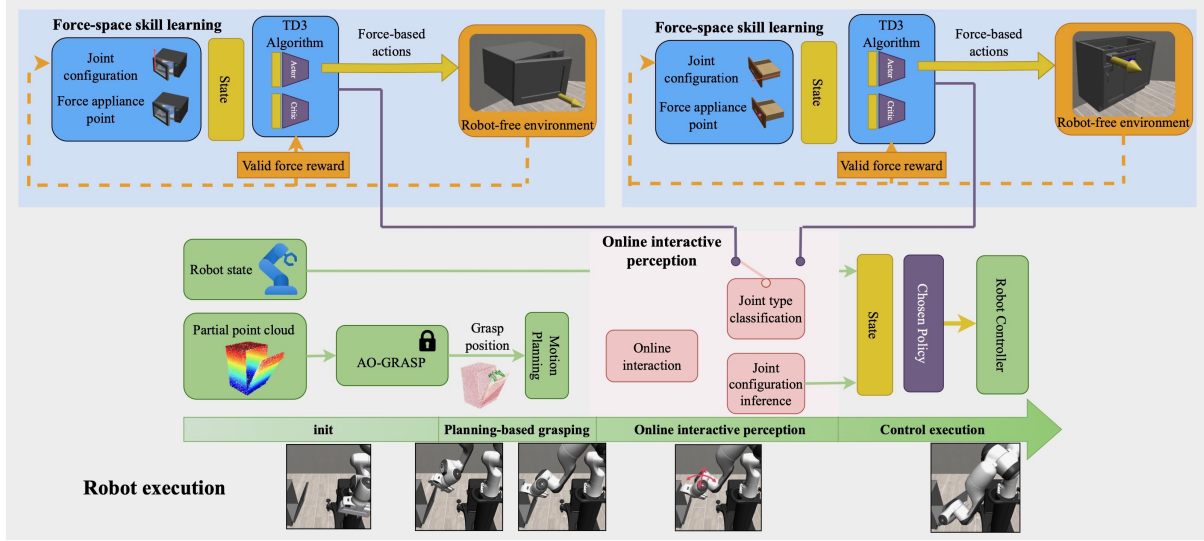


Figure 6.2: Architecture of FLEX. Force-based skill learning (left) trains object-centric policies in simulation without any robot. Interactive joint parameter estimation (center) infers the object’s joint type and configuration from physical interaction. Robot execution (right) maps the learned forces to joint torques through an operational space controller, enabling deployment on any robot platform.

6.3.3 Interactive Joint Parameter Estimation

At deployment, the robot must identify the object’s joint type and parameters to select and instantiate the appropriate policy. FLEX uses an interactive estimation procedure that operates through physical interaction rather than prior knowledge.

The robot uses an RGB-D camera to generate a partial point cloud of the object. A pre-trained grasp model (AO-GRASP [98]) identifies suitable grasp points, and a motion planner guides the robot to establish contact. The robot then applies small forces in a sequence of pre-determined directions (forward, backward, left, right), generating a short end-effector trajectory $\xi = \{p_i^{\text{eff}} \in \mathbb{R}^3\}_{i=1}^N$.

The joint type $J \in \{\text{prismatic}, \text{revolute}\}$ is inferred by maximum likelihood estimation:

$$\hat{J} = \arg \max_{J \in \{\text{prismatic}, \text{revolute}\}} p(\xi | J),$$

where the likelihood $p(\xi | J)$ models the trajectory as linear motion (prismatic) or circular motion (revolute). Joint parameters (axis direction, origin, radius) are estimated by applying Principal Component Analysis to the trajectory and refining through least-squares optimization.

Object	Algorithm	Training (timesteps)	Training (wall time)	Success Rate
Cabinet	FLEX	0.4 ± 0.26 M	0.36 ± 0.2 h	0.91 ± 0.08
	End-to-end RL	≥ 10 M	≥ 24 h	≤ 0.01
	CIPS	3 M	1.46 ± 0.02 h	0.34 ± 0.31
	GAMMA	N/A	N/A	0.49 ± 0.29
Trashcan	FLEX	0.4 ± 0.26 M	0.36 ± 0.2 h	0.88 ± 0.10
	End-to-end RL	≥ 10 M	≥ 24 h	≤ 0.01
	CIPS	3 M	1.46 ± 0.02 h	0.19 ± 0.25
	GAMMA	N/A	N/A	0.22 ± 0.18
Microwave	FLEX	0.9 ± 0.3 M	0.63 ± 0.22 h	0.67 ± 0.14
	End-to-end RL	≥ 10 M	≥ 24 h	≤ 0.01
	CIPS	5 M	2.84 ± 0.2 h	0.33 ± 0.26
	GAMMA	N/A	N/A	0.49 ± 0.18
Dishwasher	FLEX	0.9 ± 0.3 M	0.63 ± 0.22 h	0.64 ± 0.11
	End-to-end RL	≥ 10 M	≥ 24 h	≤ 0.01
	CIPS	5 M	2.84 ± 0.2 h	0.55 ± 0.36
	GAMMA	N/A	N/A	0.47 ± 0.38

Table 6.1: FLEX results on novel object instances evaluated using the Panda robot. Training time in hours and time steps in millions. FLEX achieves the highest success rates across all object types while requiring substantially less training time than all RL baselines.

The robot then selects the pre-trained policy π_j corresponding to the inferred joint type.

6.3.4 Robot Execution

The final component translates the learned force-based policies to physical robot commands. After grasping the object, the robot constructs the policy state from the estimated joint parameters and the current end-effector position. The policy outputs a desired force, which is mapped to joint torques via an Operational Space Controller (OSC) [67], ensuring smooth adaptation to variations in object weight and dynamics. This execution layer is the only robot-specific component: the same learned policies are deployed on Kinova, Panda, and UR5 robots by changing only the OSC parameters.

6.3.5 Experimental Evaluation

FLEX was evaluated on four articulated object types (cabinets, trashcans, microwaves, dishwashers) in the Robosuite simulation environment [145], using objects from the PartNet-Mobility dataset [139]. Policies were trained on one or two representative objects per joint type and evaluated on 13 previously unseen objects.

Object	Panda	UR5e	Kinova Gen3
Cabinet	0.91 ± 0.08	0.82 ± 0.13	0.77 ± 0.12
Trashcan	0.88 ± 0.10	0.75 ± 0.15	0.79 ± 0.10
Microwave	0.67 ± 0.14	0.53 ± 0.08	0.55 ± 0.12
Dishwasher	0.64 ± 0.11	0.62 ± 0.11	0.59 ± 0.13

Table 6.2: Cross-robot transfer results for FLEX. The same learned policies are deployed on three different robotic platforms without retraining. Performance remains consistent across platforms, demonstrating that the force-space representation enables direct embodiment transfer.

Baselines. Three baselines were compared: (1) an end-to-end RL baseline using TD3 with a hand-crafted dense reward, (2) CIPS [109], a hybrid RL and planning method that reduces the task horizon by planning up to the point of object contact, and (3) GAMMA [142], a pre-trained visual perception model that infers joint configurations and grasp poses from partial point clouds. All RL baselines were trained on the same objects and ground-truth observations as FLEX.

Training efficiency. FLEX converges substantially faster than baselines. For prismatic objects, FLEX converges in approximately 0.4 ± 0.26 million time steps (0.36 hours wall time), compared to ≥ 10 million time steps (≥ 24 hours) for end-to-end RL and 3 million time steps for CIPS. This advantage arises from two sources: eliminating robot dynamics from the simulation accelerates each time step, and the force-space action representation reduces unnecessary exploration.

Generalization to unseen objects. Table 6.1 reports success rates on 13 unseen objects evaluated using the Panda robot. FLEX achieves success rates of 91% (cabinets), 88% (trashcans), 67% (microwaves), and 64% (dishwashers), consistently outperforming all baselines. The end-to-end RL baseline achieves negligible success ($\leq 1\%$) even after 10 million training steps, highlighting the difficulty of the task in robot-centric action spaces. CIPS and GAMMA achieve moderate performance but with substantially higher variance.

Cross-robot transfer. Table 6.2 reports success rates when transferring the same learned policies—without retraining—to three different robot platforms: Panda, UR5e, and Kinova Gen3. FLEX maintains consistent performance across all platforms with minimal degrada-

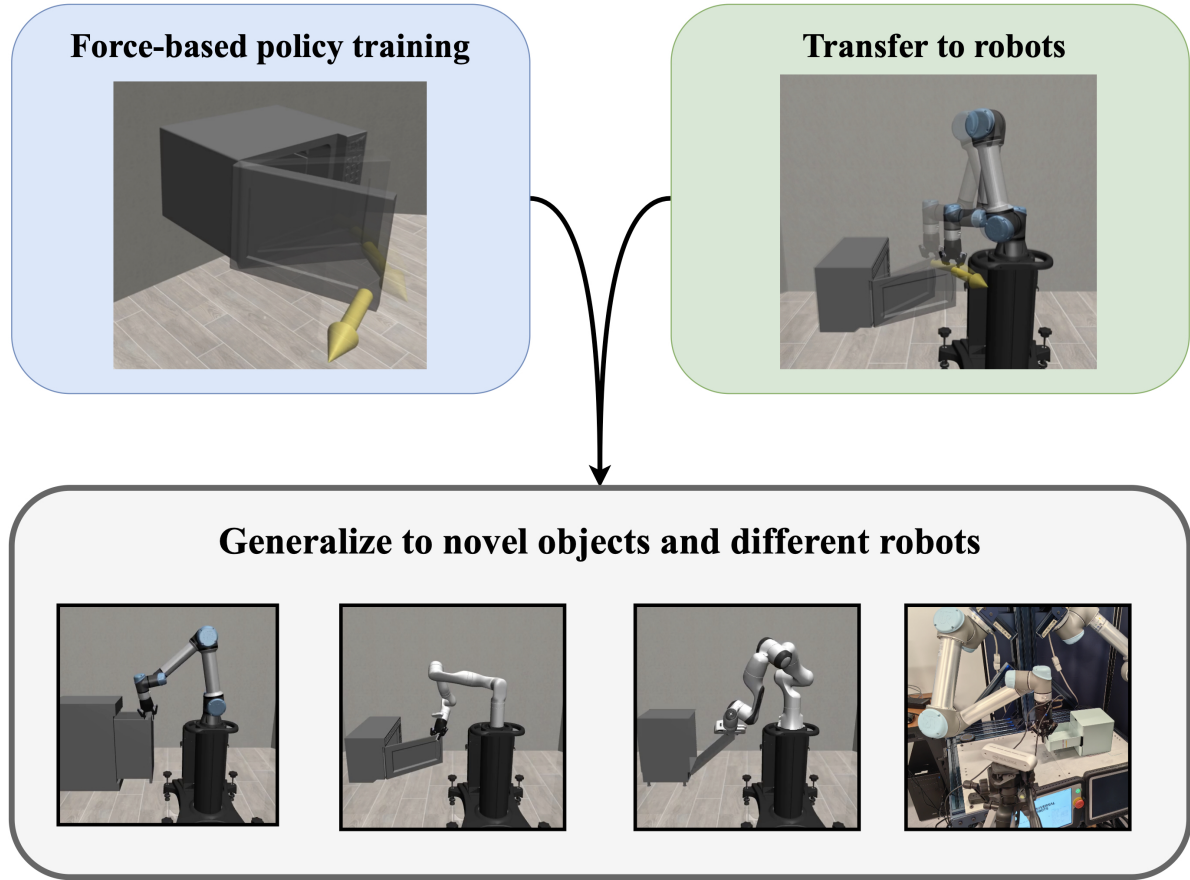


Figure 6.3: FLEX learns force-based skills for manipulating articulated objects across various robots by decoupling policy learning from robot dynamics. The UR5e opens a cabinet, Kinova opens a microwave door, Panda opens a dishwasher rack, and a real UR5 opens a drawer—all using the same learned policies without retraining.

tion, demonstrating that the object-centric force-space representation enables direct cross-embodiment transfer. In contrast, all baselines are trained and evaluated on a single platform and would require full retraining for each new robot.

Real-world demonstration. FLEX was validated on a real UR5 robot opening a physical drawer, demonstrating that the force-space policies trained entirely in simulation transfer to real-world articulated object manipulation.

6.3.6 Simulation Details

Our experiments are performed in three simulation environments:

- **TrainingEnv:** This environment excludes the robot and focuses on force-based interactions with objects. Random objects from the training set are selected and placed at $[-1, 0]$ with randomized rotations around the Z-axis. Force application points are randomly sampled, and actions are applied using the simulator.
- **BaselineTrainingEnv:** Includes both a Panda robot and objects. The objects are placed on a partial circle around the robot to ensure reachability. Object orientations are randomized for evaluation consistency.
- **EvalEnv:** Both FLEX and baseline methods are evaluated in this environment with consistent settings.

6.3.7 Learning Process

Network Design

The actor network for TD3 has two heads:

Action Direction: 3 dimensions,

Action Scale: 1 dimension.

Both the action dimensions and action scale are limited to $(0, 1]$ using bounded activation functions. The action is then computed as:

$$a_t = \pi_{\text{direction}}(s_t) \cdot \pi_{\text{scale}}(s_t) \cdot \eta$$

where η is the maximum allowable force.

Curriculum Learning for Revolute Objects

Training for revolute objects uses the following curricula with increasing rotation randomization:

$$\begin{aligned} &(-0.25, 0.25), \\ &(-\pi/2, 0), \\ &(-\pi/2, \pi/2), \\ &(-\pi, 0), \\ &(-\pi, \pi). \end{aligned}$$

The next curriculum is used when the agent applies 80% of force in the correct direction over 100 episodes.

Network Structure

The actor network is a two-headed MLP network. One input layer and two hidden layers encode the state. The scale head and direction head then map the encoded features to normalized three-dimensional action direction and one-dimensional action scale in order to compute actions.

Parameter	Value
Actor Hidden layers	2
Actor Hidden layer dimensions	(400,300)
$\pi_{\text{direction}}$ Activation function	Sigmoid
π_{scale} Activation function	Tahn
Critic Hidden Layers	2
Critic Hidden layer dimensions	(400,300)
Critic Activation Function	ReLU

Table 6.3: Agent Network Struture

RL Hyperparameters

The training was performed with 24 parallel environments on an Intel i9-12900K processor with an Nvidia RTX 3080 GPU.

Parameter	Value
Discount Factor (γ)	0.99
Batch Size	100
Learning Rate	0.001
Policy Update (τ)	0.995
Noise Clip	1
Policy Delay	2
Max Timesteps per Episode	200
Rollouts	5
State Dimension	6
Action Dimension	3
Max Action	5
max force magnitude (η)	0.02

Table 6.4: Reinforcement Learning Hyperparameters

6.3.8 Execution Details

Execution Assumptions and Object Placement

To avoid collisions, revolute objects are placed 0.7m away with a randomized rotation angle between $[-\pi/2, 0]$, while prismatic objects are placed 0.9m away with an angle between $[-3\pi/4, \pi/4]$.

6.3.9 Baselines

End-to-End RL

The end-to-end RL agent is trained using the StableBaselines3 TD3 agent, with objects placed at a *fixed* location. The reward function is designed to guide the robot through sequential stages: approaching, grasping, and manipulating the object. . Despite the use of a dense reward function, the poor (almost nil) performance highlights the complexity of the task and demonstrates why *conventional* RL-based control is an inefficient approach.

CIPS Training

To improve efficiency, CIPS training begins with the robot *already* grasping the object. Four pre-recorded initial states are uniformly sampled from the 1/4 circle around the robot. This method accelerates training by reducing unnecessary exploration.

Parameter	Value
Policy Learning Rate	Default (SB3)
Discount Factor (γ)	Default (SB3)
Replay Buffer Size	Default (SB3)

Table 6.5: Baseline Hyperparameters for CIPS

GAMMA

As stated in the paper, we assume knowledge of joint type for GAMMA evaluation. We designed a simple planning method that complies with the ideal joint dynamics. Specifically, for revolute objects,

$$\mathbf{a}_t^{GAMMA} = -\hat{\mathbf{h}}_r \times \hat{\mathbf{v}}_t$$

For prismatic objects,

$$\mathbf{a}_t^{GAMMA} = \hat{\mathbf{h}}_p$$

Where $\hat{\mathbf{h}}_r$ and $\hat{\mathbf{h}}_p$ are the selected prediction results of joint parameters given by GAMMA.

We observed that GAMMA tends to overestimate the number of joints on complex objects with multiple moving parts, leading to incorrect joint inferences. To mitigate this, we only select the first inference result that complies with our ground-truth knowledge of the joint type. If no correct inference exists, we randomly choose from the remaining detected joints.

6.3.10 Real-world setup

Hardware and simplifications

The real-world experiments used a UR-5 arm and an Intel Realsense D455 RGBD camera. To simplify, we used an inverse kinematics (IK) controller and a toy drawer. We assumed knowledge of the drawer pose and skipped interactive perception. The hyperparameters stayed the same with simulation experiment settings.

6.3.11 Additional experiments and results

80% opening criterion

In our experiments, success is defined as opening the object to at least 80% of its joint limit.

Additional experiments

Several additional experiments were conducted on objects with different joint configurations, but due to space limitations, they are not included in the main paper.

6.3.12 Object details and dataset

Object scaling and preprocessing

Objects were selected from the Partnet-Mobility dataset and scaled for simulation. Prismatic objects were scaled to 50% and revolute objects to 30% of their original size.

Collision handling

Due to self-collision issues in the dataset, we disabled collisions for the main body of objects during training. Since the objects were partially opened before manipulation, this did not affect the results.

6.4 Pushing Heavy Objects: Force Policies for Rigid Body Transport

The previous section demonstrated that object-centric force-space policies enable generalizable manipulation of articulated objects whose motion is constrained by joints. This section extends the approach to a complementary class of sustained-contact manipulation: transporting rigid objects along prescribed paths through sustained boundary contact, where the primary challenge is not joint-constrained motion but unknown and varying contact resistance due to object mass and surface friction [45].

6.4.1 Problem Setting

We consider a rigid object with planar pose $(c, \theta) \in \mathbb{R}^2 \times (-\pi, \pi]$ driven by boundary contact forces $f = (F_x, F_y)^\top \in \mathbb{R}^2$ under a quasi-static regime in which inertial effects are negligible relative to frictional resistance. The object must be transported along a prescribed planar reference path $\Gamma \subset \mathbb{R}^2$ while maintaining bounded lateral deviation and orientation error over a finite horizon T . The object mass m and surface friction coefficient μ are not assumed known a priori and jointly determine the contact resistance required to initiate and sustain motion.

This setting presents three coupled sub-problems: (1) learning an object-centric path-following policy that outputs boundary forces conditioned on a compact interaction descriptor; (2) inferring the interaction descriptor from onboard sensing, without explicit identification of mass and friction; and (3) physically realizing the desired boundary forces on a robot through compliant control. The decomposition mirrors the structure of FLEX—policy learning, interactive estimation, robot execution—but addresses a fundamentally different object class and interaction regime.

6.4.2 Object-Centric Policy Learning

The force policy is learned entirely in simulation, without modeling robot embodiment. Learning is formalized as a parameterized Markov Decision Process:

$$M(z) = \langle S, A, T_z, R, \gamma \rangle,$$

where $z \in \mathcal{Z}$ is a task-relevant interaction descriptor sampled once per episode and held fixed during rollout. The state $s_t \in S \subset \mathbb{R}^6$ is an egocentric, path-relative representation:

$$s_t = \left(e_t^{\text{lat}}, e_t^\theta, v_t^\parallel, v_t^\perp, \omega_t, z \right),$$

where e_t^{lat} is the signed lateral deviation from the path, e_t^θ is the orientation error relative to the path tangent, $v_t^\parallel$ and $v_t^\perp$ are velocity components along the tangent and normal, ω_t is the angular velocity, and $z \in [0, 1]$ is a normalized contact-resistance scalar encoding the interaction regime.

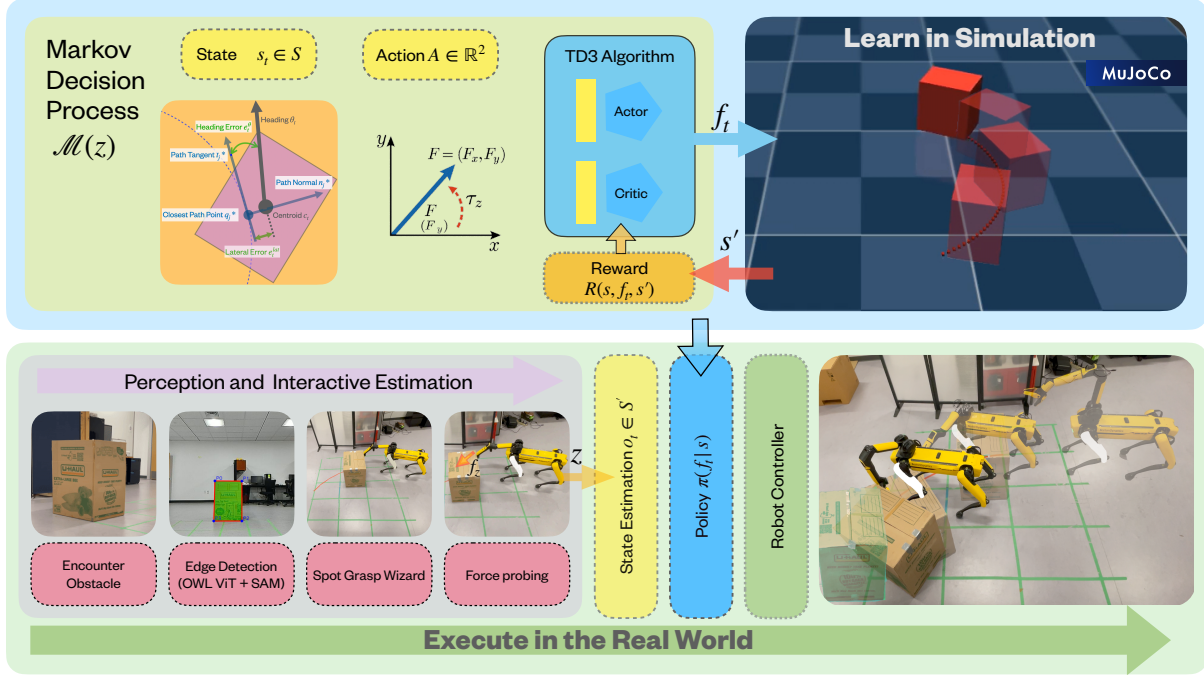


Figure 6.4: Architecture of the pushing heavy objects framework. The policy is trained in simulation (top right) using a parameterized MDP conditioned on an interaction descriptor z . At deployment (bottom), the robot estimates z through impedance-based force probing and constructs the object-centric state from proprioceptive sensing, then executes the learned boundary forces through compliant impedance control.

The action space $A = \{f \in \mathbb{R}^2 \mid \|f\| \leq F_{\max}\}$ consists of planar boundary forces applied at the contact point, with $F_{\max} = 200$ N. Object rotation arises implicitly from the lever-arm geometry $\tau = r \times f_t$, without requiring an explicit torque command.

The reward function promotes efficient path tracking through a combination of progress, speed regulation, lateral deviation, orientation alignment, and spin penalties:

$$R(s_t, f_t, s_{t+1}) = \alpha v_t^{\parallel} e^{-\beta |e_t^{\text{lat}}|} - \lambda (v_t^{\parallel} - v_{\text{ref}})^2 - \kappa (e_t^{\text{lat}})^2 - \eta (e_t^{\theta})^2 - \xi \omega_t^2 + \delta \mathbb{I}\{\text{goal reached}\}. \quad (6.1)$$

The policy is trained using TD3 [32] with a segment-based curriculum and domain randomization over mass (5–15 kg) and friction ($\mu \in [0.4, 0.6]$). During training, the normalized descriptor is computed as $z = \text{clip}(\mu mg / F_{\max}, 0, 1)$, providing the policy with a compact summary of the interaction regime.

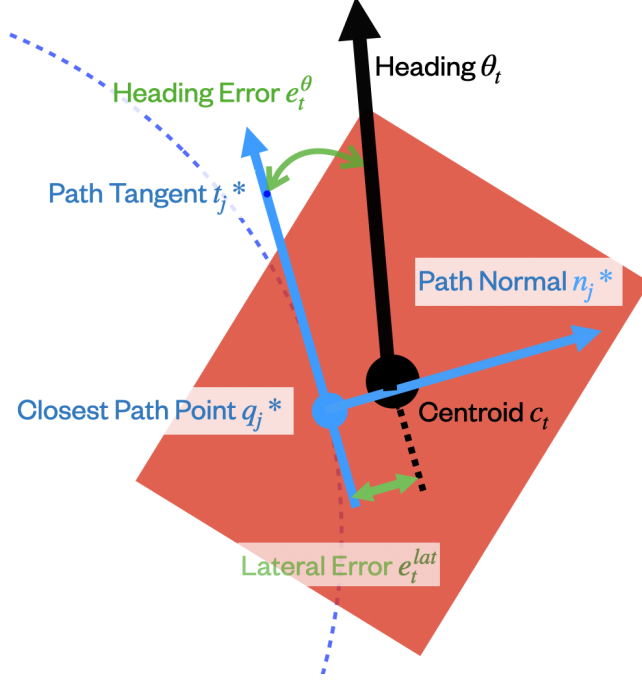


Figure 6.5: Diagrammatic representation of the state space in the parameterized MDP. The object’s pose is represented relative to the reference path through lateral deviation e_t^{lat} , heading error e_t^θ , and velocity components along the path tangent and normal. The normalized contact-resistance descriptor z conditions the policy on the interaction regime.

6.4.3 Interactive Estimation

At deployment, the robot does not have access to the object’s mass or friction coefficient. The interaction descriptor z is estimated through a lightweight impedance-based probing routine that requires only proprioceptive force sensing.

Starting from an established boundary contact, the robot incrementally displaces the impedance controller’s equilibrium point along a probe axis while measuring the contact force. As the offset increases, the reaction force rises due to static friction until breakaway occurs. The peak force $f_{\max}$ prior to breakaway is recorded, and the interaction descriptor is computed as:

$$z = \text{clip} \left(\frac{f_{\max}}{F_{\max}}, 0, 1 \right).$$

The descriptor z approximates the breakaway friction force required to initiate sliding. Under quasi-static planar pushing, this force scales with μmg , capturing the combined influence of object mass and surface friction without requiring explicit identification of either parameter

Algorithm 4 Reactive Force Estimation via Impedance Probing

Require: Probe axis $\hat{u}$, step size δ , max steps N , window size w , break threshold Δ_f , $F_{\max}$

Ensure: $z \in [0, 1]$

```

1: Activate impedance controller and wait for settling
2:  $f_{\text{base}} \leftarrow \text{mean}(\text{baseline samples})$ 
3:  $f_{\text{max}} \leftarrow 0$ 
4:  $\mathcal{W} \leftarrow \emptyset$ 
5: for  $i = 1$  to  $N$  do
6:   Advance virtual reference by  $i\delta$  along  $\hat{u}$ 
7:    $f_i \leftarrow \max_t(f_{\text{proj}}(t) - f_{\text{base}})$ 
8:    $f_{\text{max}} \leftarrow \max(f_{\text{max}}, f_i)$ 
9:   Append  $f_i$  to  $\mathcal{W}$  (retain last  $w$  samples)
10:  if  $|\mathcal{W}| = w$  then
11:     $\bar{f} \leftarrow \text{mean}(\mathcal{W})$ 
12:    if  $f_{\text{max}} - \bar{f} > \Delta_f$  then
13:      break {breakaway detected}
14:    end if
15:  end if
16: end for
17:
18: return  $z \leftarrow \text{clip}(f_{\text{max}}/F_{\text{max}}, 0, 1)$ 

```

individually. The scalar z remains fixed for the duration of the episode.

After estimating z , the remaining state components are computed at each control step from proprioceptive measurements. The object centroid is recovered from the known contact offset, and geometric errors (lateral deviation, heading error) and kinematic quantities (velocities, angular velocity) are computed relative to the reference path.

Complexity. Let N be the maximum number of probing steps and w be the window size. Assume the sliding window is maintained in a fixed-size buffer so that appending a sample and evicting the oldest are $O(1)$ operations. The algorithm performs at most N probing iterations. In each iteration, it updates the force estimate, maintains a sliding window of at most w samples, and computes the window mean. If the mean is computed directly, each iteration costs $O(w)$, giving a total time complexity of $O(Nw)$. If the sliding-window mean is maintained incrementally using a running sum, then each iteration takes constant time, and the total time complexity reduces to $O(N)$. The space complexity is $O(w)$ since the algorithm stores only the most recent w force samples.

6.4.4 Robot Execution

The learned boundary forces are realized on hardware through compliant impedance control. At each control step, the policy outputs a desired planar force f_t , which is converted into a Cartesian displacement command via an admittance-style mapping:

$$\Delta x_t = \frac{f_t}{K_{\text{eff}}},$$

where K_{eff} is the effective planar stiffness of the impedance controller. The resulting contact force $F_{\text{contact}} \approx K_{\text{eff}}(x_{\text{ref}} - x) + \zeta \frac{dx}{dt} \approx f_t$, where ζ is the virtual damping. This compliant execution layer allows small pose deviations to generate interaction forces while preventing excessive contact instability.

The policy operates in object-centric force space; the impedance controller serves only to realize boundary forces through compliant motion, independent of the robot’s specific kinematic structure. This separation between force-level decision-making and embodiment-specific control enables zero-shot transfer from simulation to hardware without policy retraining.

6.4.5 Experimental Evaluation

Simulation Experiments

Policies were trained in MuJoCo [133] with mass randomized over $[5, 15]$ kg and friction over $[0.4, 0.6]$, using short arc segments from a circular path of radius 1.5 m. Generalization was evaluated under single-parameter out-of-distribution (OOD) shifts.

Parameter generalization. Table 6.6 reports tracking performance (centroid-to-path RMSE) under controlled OOD shifts. The fully in-distribution baseline achieves 2.9 ± 0.4 cm. Across all OOD conditions—mass, friction, path radius, and box dimensions—RMSE remains within a narrow range of 2.7–3.3 cm, demonstrating stable generalization. Box dimension and path radius variations were introduced only at evaluation time and not included during training.

Path generalization. Table 6.7 evaluates generalization to novel path geometries with physical parameters kept in-distribution. Performance degrades gradually as curvature increases:

Parameter	OOD Low RMSE (cm)	OOD High RMSE (cm)
Mass	3.1 ± 0.4	3.3 ± 0.5
Friction	3.3 ± 0.8	2.8 ± 0.3
Path radius	2.7 ± 0.3	3.1 ± 0.4
Box dimensions	3.1 ± 0.6	2.8 ± 0.2

Table 6.6: Parameter generalization for pushing heavy objects. Centroid-to-path RMSE (cm) under single-parameter out-of-distribution shifts. The in-distribution baseline achieves 2.9 ± 0.4 cm. Performance remains stable across all OOD conditions.

Path geometry	RMSE (cm)
Arc 60°	2.2 ± 0.3
Arc 120°	2.8 ± 0.3
Arc 180°	5.2 ± 2.2
S-Path	9.0 ± 6.2
Sine-Wave	10.1 ± 6.6

Table 6.7: Path generalization for pushing heavy objects. RMSE (cm) under novel path geometries with in-distribution physical parameters. Performance degrades gracefully with increasing curvature complexity.

60° and 120° arcs remain close to baseline (2.2–2.8 cm), while S-shaped and sinusoidal paths—which introduce curvature sign reversals absent from training—show higher RMSE (9–10 cm). Even in the most challenging cases, the deviation remains well below one-fifth of the object width, indicating bounded transport despite the distribution shift.

Interaction descriptor ablation. Table 6.8 presents an ablation study evaluating the role of the normalized contact-resistance descriptor z . Removing the descriptor ($z = 0$) increases tracking error by 45%, while setting $z = 1$ produces an order-of-magnitude increase in RMSE with high variance. Replacing the descriptor with its in-distribution mean ($\mathbb{E}[z] = 0.245$) yields nearly identical performance to the full system, suggesting that the policy uses z as a coarse regime indicator rather than a precise estimate.

Real-World Experiments

The learned policy was deployed zero-shot on a Boston Dynamics Spot mobile manipulator without retraining. Only impedance gains ($K_{\text{eff}} = 300$, $\zeta = 45$) were tuned to ensure stable, non-oscillatory contact. Experiments were conducted with two unknown payload conditions (8.8 kg and 12.5 kg), with 10 trials per condition for both the proposed policy and a kinematic

Descriptor setting	z value	RMSE (cm)
Estimated (full system)	0.10–0.44	2.87 ± 0.35
Mean value	0.245	2.79 ± 0.40
Fixed at minimum	0	4.16 ± 0.77
Fixed at maximum	1	34.15 ± 33.50

Table 6.8: Ablation study for the interaction descriptor z . The descriptor provides meaningful conditioning for the learned policy; removing it significantly degrades performance.

trajectory-following baseline, for a total of 40 real-world trials.

Reactive force estimation. Prior to each trial, the robot performs the impedance-based probing routine to estimate z . The estimated reactive forces are 26.18 ± 2.07 N for the 8.8 kg object and 41.74 ± 2.20 N for the 12.5 kg object, consistent with expected increases in contact resistance and showing low variance within each condition.

Execution performance. The proposed policy successfully transported the object to the target endpoint in 17 out of 20 attempts across both payload conditions. The trajectory-following baseline did not reach the goal region in any trial (0/20). This consistent separation across repeated trials indicates that the performance difference is not attributable to stochastic variation in execution.

Qualitative observations. The baseline controller frequently deviated from the intended trajectory and struggled to maintain sustained contact, particularly under the heavier payload. The force-based policy maintained consistent pushing behavior across both masses and completed the curved transport task in the majority of trials. The dominant failure mode was contact loss caused by slip at the gripper–object interface, which was partially mitigated by a brief zero-stiffness impedance recovery step.

6.5 Summary and Future Directions

This chapter presented two contributions that demonstrate how object-centric force-space representations enable learned manipulation behaviors to generalize across objects, physical parameters, and robotic embodiments. FLEX learns force-based policies for articulated object

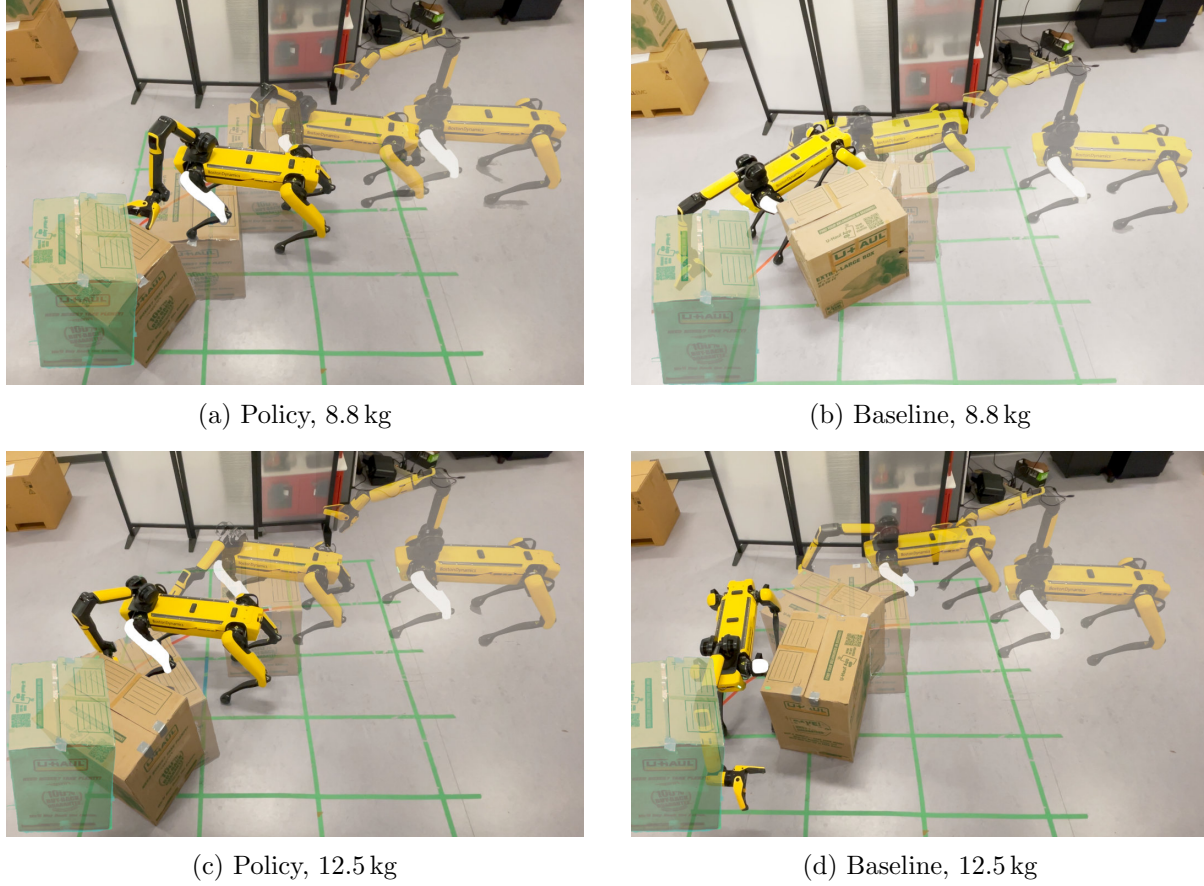


Figure 6.6: Chronophotographic composites of real-world box transport on the Boston Dynamics Spot. Top row: proposed force-based policy under 8.8 kg (a) and 12.5 kg (c) payloads, showing consistent trajectory tracking. Bottom row: kinematic baseline under the same conditions (b, d), showing frequent deviation and failure to reach the goal.

manipulation that transfer directly to three different robot platforms without retraining, achieving over an order-of-magnitude improvement in training efficiency compared to robot-centric baselines. The pushing heavy objects framework extends this approach to rigid body transport under unknown mass and friction, demonstrating zero-shot sim-to-real transfer on a legged mobile manipulator.

Both contributions instantiate the interaction abstraction framework developed in Section 2.4.3. The key design principle is the same in both cases: by defining policies in the space of forces applied to the object—conditioned on object-relative state descriptions—the learned behavior captures the physical invariants of the task while remaining agnostic to the specific robot, object instance, or physical parameters. This constitutes a structured interaction abstraction that serves as a buffer between the learned policy and the variability of the world,

consistent with the unifying perspective of Section 2.4.4.

The parallel with Chapter 5 is instructive. In that chapter, action abstractions enabled efficient adaptation by localizing failure to specific operators and focusing learning on the affected component. In this chapter, interaction abstractions enable generalization by organizing the policy’s representation around task-relevant physical invariants rather than embodiment-specific details. Both forms of abstraction reduce the scope of what must be learned or relearned when conditions change—but they operate at different levels: action abstractions structure the *sequence* of behaviors, while interaction abstractions structure the *representation* of each behavior.

Several limitations and future directions emerge from this work.

Richer interaction descriptors. The pushing heavy objects framework conditions on a single scalar z capturing the effective contact resistance. This is sufficient for quasi-static transport under uniform friction but cannot represent spatially varying friction, anisotropic contact, or time-varying resistance. Extending the descriptor to a richer representation—potentially learned from interaction history—would broaden the range of applicable scenarios.

Dynamic manipulation. Both FLEX and the pushing framework assume quasi-static interaction regimes. Extending to dynamic manipulation, where inertial effects dominate and contact modes switch rapidly, would require richer state representations and potentially hybrid quasi-static/dynamic control formulations.

Contact recovery and robustness. The dominant failure mode in real-world experiments was contact loss, either from slip at the gripper–object interface or from changes in contact geometry during body repositioning. Incorporating contact-state estimation from wrist force/torque sensing and developing reactive re-grasping strategies would improve robustness in unstructured environments.

Integration with planning and adaptation. The force-space policies developed in this chapter operate at the execution level. Integrating them as executors within the hybrid planning–learning framework of Chapter 5 would create a system that can both plan over abstract task

structure and execute through generalizable force-based skills. When a novel object is encountered whose physical properties differ from prior experience, the planning layer could detect the mismatch and invoke targeted adaptation—using the interaction descriptor estimation as a lightweight mechanism for characterizing the new regime—while the force-space representation ensures that the adapted behavior transfers across embodiments. This integration represents a natural convergence of the action and interaction abstractions studied in this dissertation.

Multi-contact and collaborative manipulation. Both frameworks consider single-contact manipulation. Extending to multi-contact settings—where multiple forces are applied simultaneously, or where multiple robots collaborate—would require coordinating force policies across contact points and managing the interactions between them.

6.5.1 Physics Engine and Scene Configuration

All simulation experiments are conducted in MuJoCo [133]. The scene consists of a ground plane (half-size $5 \times 5 \times 0.1$ m) and a single rigid box body with a free joint (6-DoF). Table 6.9 lists the relevant physics and contact parameters.

Table 6.9: MuJoCo physics and contact parameters.

Parameter	Value
Integration timestep	0.0025 s
Substeps per RL step	10
Effective control Δt	0.025 s
Contact dimension (<code>condim</code>)	4 (tangent + torsional)
<code>solimp</code>	[0.9, 0.95, 0.001, 0.5, 2]
Linear damping (box DoFs)	0.05
Angular damping (box DoFs)	0.05
Warm-start settle steps	50 (0.125 s)

6.5.2 Object Geometry

The box geometry is specified as half-extents in the MuJoCo model. The full dimensions are $0.465 \times 0.61 \times 0.63$ m (width $\times$ depth $\times$ height), matching the real-world cardboard box used in hardware experiments. The nominal inertial properties (overridden at each episode by domain randomization) are: mass 15.0 kg, diagonal inertia [0.2403, 0.1916, 0.1839] kg m². The box is

initialized at height $z = 0.315$ m (half the box height) at the first waypoint of the reference path.

6.5.3 Reference Path Generation

During training, the reference trajectory is a circular arc with radius 1.5 m, spanning 120° (from $-\pi/3$ to $\pi/3$), discretized into 50 equispaced waypoints. Each training episode samples a short segment of arc length 0.3 m from this full arc. The segment start position is sampled uniformly along the arc.

6.5.4 Domain Randomization

At each episode reset, the following physical parameters are independently sampled from uniform distributions:

- **Object mass:** $m \sim \mathcal{U}(5, 15)$ kg
- **Sliding friction coefficient:** $\mu \sim \mathcal{U}(0.4, 0.6)$

The Coulomb friction coefficient is applied symmetrically to both the box and the floor geom. Additional friction parameters (spinning friction = 0.5, rolling friction = 0.01) remain fixed. Box dimensions are held constant during training; variation is introduced only at evaluation time.

6.6 MDP Specification

6.6.1 State Space

The state is a 6-dimensional vector defined in the object-centric, path-relative frame:

$$s_t = [e_t^{\text{lat}}, e_t^\theta, v_t^\parallel, v_t^\perp, \omega_t, z] \in \mathbb{R}^6, \quad (6.2)$$

where:

- e_t^{lat} : signed lateral deviation from the nearest path waypoint (m),

- e_t^θ : wrapped orientation error relative to the local path tangent (rad),
- $v_t^\parallel$: velocity projected onto the path tangent (m/s),
- $v_t^\perp$: velocity projected onto the path normal (m/s),
- ω_t : angular velocity about the vertical axis (rad/s),
- $z = \text{clip}(\mu mg/F_{\max}, 0, 1)$: normalized contact-resistance descriptor.

Velocities are computed via finite differences over $\Delta t = 0.025$ s. The interaction descriptor z is computed from ground-truth physics in simulation and held constant within each episode. In the push-from-edge configuration, the “heading” used for orientation error is the direction from the box centroid to the contact edge, expressed in the box body frame as $[0.465, 0.63, 0]$ (normalized).

6.6.2 Action Space

In the push-from-edge mode used for all reported experiments, the action space is 2-dimensional. The actor network outputs a unit direction vector $\hat{\mathbf{d}} \in \mathbb{R}^2$ and a scalar magnitude $m \in [0, 1]$ (see Section 6.8), combined as $\mathbf{a}_t = \hat{\mathbf{d}} \cdot m$. The environment scales this to physical forces:

$$F_x = \text{clip}(a_x, -1, 1) \cdot F_{\max}, \quad F_y = \text{clip}(a_y, -1, 1) \cdot F_{\max}, \quad (6.3)$$

where $F_{\max} = 200$ N. Torque arises implicitly from the lever-arm geometry: $\tau = \mathbf{r} \times \mathbf{F}$, where $\mathbf{r}$ is the vector from the box centroid to the contact edge point. The direction-magnitude factorization constrains the action to lie within the unit disk, so the effective force magnitude is bounded by $F_{\max}$.

Forces are applied in the box body frame and rotated into the world frame at each physics substep.

6.6.3 Reward Function

The reward at each step is a sum of five terms:

$$R = r_{\text{prog}} + r_{\text{speed}} + r_{\text{lat}} + r_{\text{ori}} + r_{\text{spin}}, \quad (6.4)$$

defined as follows. Table 6.10 lists each component, its expression, and the corresponding weight.

Table 6.10: Reward components. $\Delta t = 0.025$ s; $v_{\text{ref}} = 0.3$ m/s.

Term	Expression	Weight
Progress	$v_t^{\parallel} \Delta t \cdot e^{-5 e_t^{\text{lat}} }$	+100
Speed	$(v_t^{\parallel} - v_{\text{ref}})^2$	-8
Lateral	$(e_t^{\text{lat}})^2$	-10
Orientation	$(e_t^{\theta})^2$	-2
Spin	ω_t^2	-0.5

Terminal rewards and penalties:

- **Success bonus:** +100 when progress > 0.95 and deviation $<$ goal threshold.
- **Failure penalty:** -500 when $|e_t^{\text{lat}}| >$ deviation tolerance (0.5 m).

The goal threshold is computed dynamically as $\max(0.10 \times L_{\text{seg}}, 0.03)$ m, where L_{seg} is the current segment arc length. Episodes are truncated after 500 steps (short-segment training) or 1000 steps (full-arc evaluation).

6.7 TD3 Hyperparameters

We use Twin Delayed Deep Deterministic Policy Gradient (TD3) [?]. Table 6.11 reports all hyperparameters.

Replay buffer management. When the buffer exceeds its maximum capacity, the oldest 1/5 of transitions are discarded.

Model selection. The best model is selected based on a composite score: $0.5 \times \text{full-arc success rate} + 0.5 \times \text{generalization success rate}$, evaluated every 25 episodes on in-distribution conditions. The generalization evaluation uses random arc geometries with radius $r \sim \mathcal{U}(1, 2)$ m and angular span variation of $\pm 60^\circ$ from the nominal 120° training arc.

Table 6.11: TD3 training hyperparameters.

Hyperparameter	Value
Learning rate (actor & critics)	1×10^{-4}
Optimizer	Adam
Discount factor (γ)	0.99
Target smoothing coefficient (τ)	0.995
Target policy noise (σ)	0.2
Target noise clip	0.5
Policy delay	2
Gradient steps per env step	2
Exploration noise (σ_{expl})	0.1
Batch size	256
Replay buffer capacity	500,000
Random exploration steps	2,000
Training episodes	3,001
Evaluation frequency	every 25 episodes
Evaluation episodes per condition	25
Checkpoint save frequency	every 500 episodes
Seeds	0–9 (10 seeds total)

6.8 Network Architecture

6.8.1 Actor

The actor network takes the 6D state as input and produces a 2D force action. The architecture uses a direction-magnitude decomposition:

1. **Shared trunk:**

Linear($6 \rightarrow 400$) $\rightarrow$ ReLU $\rightarrow$ Linear($400 \rightarrow 300$) $\rightarrow$ ReLU

2. **Direction head:**

Linear($300 \rightarrow 2$) $\rightarrow$ tanh $\rightarrow \ell_2$ -normalize

3. **Magnitude head:**

Linear($300 \rightarrow 1$) $\rightarrow$ sigmoid

The output force is $\mathbf{f} = \hat{\mathbf{d}} \cdot m$, where $\hat{\mathbf{d}} \in \mathbb{R}^2$ is the unit direction vector and $m \in [0, 1]$ is the magnitude. The direction-magnitude factorization ensures the force direction is always a valid unit vector, improving training stability.

6.8.2 Critic

Each of the two critic networks (twin critics) has the architecture:

$$\begin{aligned} &\text{Linear}(8 \rightarrow 400) \rightarrow \text{ReLU} \\ &\rightarrow \text{Linear}(400 \rightarrow 300) \rightarrow \text{ReLU} \\ &\rightarrow \text{Linear}(300 \rightarrow 1), \end{aligned}$$

where the input dimension is $8 = 6$ (state) + 2 (action).

Target policy smoothing. During target Q-value computation, Gaussian noise is added to the target actor’s output. The force component of the noisy action is re-normalized to a unit vector after noise addition to preserve the direction-magnitude structure.

6.9 Simulation Evaluation Protocol

6.9.1 In-Distribution Evaluation (Table II in main paper)

For each trained seed, we run 100 evaluation rollouts per condition with a maximum of 1000 steps per episode. Physical parameters not under test are sampled from the in-distribution ranges ($m \sim \mathcal{U}(5, 15)$ kg, $\mu \sim \mathcal{U}(0.4, 0.6)$) via the environment’s domain randomization at each episode reset. The evaluation seed is fixed at 42 for reproducibility. Results are reported as mean $\pm$ standard deviation of per-episode RMSE values across seeds.

Table 6.12 summarizes the out-of-distribution evaluation ranges.

Table 6.12: Out-of-distribution evaluation ranges. When testing one parameter OOD, all others remain in-distribution.

Parameter	ID Range	OOD Low	OOD High
Mass (kg)	[5, 15]	[2.5, 5]	[15, 17.5]
Friction (μ)	[0.4, 0.6]	[0.3, 0.4]	[0.6, 0.7]
Path radius (m)	1.5	[1.0, 1.5]	[1.5, 2.0]
Box dims (cm)	$46.5 \times 61 \times 63$	40^3	70^3

6.9.2 Path Generalization Evaluation (Table III in main paper)

Path generalization is evaluated with all physical parameters in-distribution. The tested trajectories are:

- **Arc** 60°, 120°, 180°: Circular arcs of radius 1.5 m with the indicated angular span, centered at 0°.
- **S-path**: A sinusoidal path of total length 3.0 m and amplitude 0.5 m, discretized into 100 waypoints.
- **Sine-wave (meandering)**: A sinusoidal path of total length 4.0 m and amplitude 0.4 m, discretized into 150 waypoints.

6.9.3 RMSE Computation

Tracking performance is measured using centroid-to-path RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{t=1}^N d_t^2}, \quad (6.5)$$

where $d_t = \min_j \|\mathbf{c}_t - \mathbf{q}_j\|_2$ is the Euclidean distance from the object centroid to the closest waypoint on the reference path, and N is the total number of steps across all episodes in a given condition.

6.9.4 Ablation: Interaction Descriptor (Table IV in main paper)

The ablation study evaluates four descriptor settings under fully in-distribution conditions ($m \sim \mathcal{U}(5, 15)$ kg, $\mu \sim \mathcal{U}(0.4, 0.6)$):

- **Estimated (ours)**: $z = \text{clip}(\mu mg / F_{\max}, 0, 1)$, yielding $z \in [0.10, 0.44]$.
- **Mean value**: Fixed $z = 0.245$ (the in-distribution mean of z).
- **Fixed at minimum**: $z = 0$.
- **Fixed at maximum**: $z = 1$.

6.10 Hardware Deployment Details

6.10.1 Robot Platform

All real-world experiments are conducted on a legged mobile manipulator equipped with a 6-DoF arm and a two-finger gripper. The learned policy is deployed zero-shot without any retraining or fine-tuning.

6.10.2 Perception Pipeline

The robot detects the target object using the onboard hand camera. Object detection uses OWL-ViT [95] for bounding-box proposal, followed by SAM [70] for precise mask extraction. A pixel on the detected mask edge is passed to the robot’s grasp service [?] for boundary grasp acquisition. After grasping, the gripper is opened to establish a push-from-edge contact configuration.

6.10.3 Impedance-Based Force Probing

The interaction descriptor z is estimated via impedance-based probing (Algorithm 1 in main paper). Table 6.13 lists the probing parameters.

Table 6.13: Impedance probing parameters.

Parameter	Value
Probe step size (δ)	0.005 m
Maximum probe steps (N)	30
F/T sampling rate	~ 50 Hz
Sample window per step	1.0 s
Force drop threshold (Δf)	4.0 N
Consecutive drop steps for breakaway	2
Probing stiffness	250 N/m
Probing damping	30 Ns/m
Pre-probe settling time	3.0 s
$F_{\max}$ (normalizer)	200 N

6.10.4 Policy Execution

Table 6.14 lists the parameters governing policy execution on hardware.

Table 6.14: Hardware execution parameters.

Parameter	Value
Maximum policy steps	30
Control step duration (Δt_{ctrl})	2.0 s
Action scale (admittance gain)	0.75 m
Yaw correction scale	0.05 rad
Success progress threshold	0.95
Deviation tolerance (abort)	1.0 m
Inter-step settling duration	3.0 s

6.10.5 Impedance Controller Gains

The Cartesian impedance controller gains used during pushing are:

- Translational stiffness K_{eff} : 300 N/m (planar x - y), 500 N/m (vertical z).
- Translational damping ζ : 45 Ns/m (planar), 45 Ns/m (vertical).
- Rotational stiffness: 20 Nm/rad per axis.
- Rotational damping: 1 Nms/rad per axis.

These gains were tuned experimentally to ensure stable, non-oscillatory contact during pushing without excessive compliance.

6.10.6 Test Object

The object used in all hardware experiments is a rigid cardboard box with dimensions $0.465 \times 0.61 \times 0.63$ m (width $\times$ depth $\times$ height), consistent with the simulation model. Two payload conditions were tested: 8.8 kg and 12.5 kg.

6.10.7 Reference Path

Hardware experiments use a 60° circular arc of radius 1.5 m, marked on the floor with red tape.

6.11 Software and Reproducibility

6.11.1 Software Versions

Table 6.15 lists the primary software dependencies used for training and evaluation.

Table 6.15: Software dependencies.

Package	Version
Python	3.10+
PyTorch	2.7
MuJoCo	3.3
Gymnasium	1.2
NumPy	2.2
SciPy	1.15

6.11.2 Training Seeds and Reproducibility

Training is conducted across 10 random seeds (seeds 0–9) with deterministic algorithm settings enabled in PyTorch. Separate RNG instances with fixed seed offsets control replay buffer sampling, exploration noise, evaluation arc geometry randomization, and target policy noise, ensuring full reproducibility.

For the results reported in the main paper, 5 seeds are used for computing mean and standard deviation.

6.11.3 Compute

Training runs were executed on a single machine. Each seed trains for 3,001 episodes (approximately 150k–300k environment steps depending on episode length). Training all 10 seeds to convergence takes approximately 1–3 hours on a consumer-grade laptop GPU (in our case, Apple M2 Pro, using Metal Performance Shaders (MPS)).

Chapter 7

Conclusion

This dissertation investigated the role of structured abstractions in enabling autonomous agents and robots to operate effectively in open-world environments. The central thesis is that while novelty—mismatch between an agent’s internal models and the environment—is unavoidable in real-world deployment, the structure of an agent’s representations determines whether this mismatch leads to brittle failure or can be managed through efficient adaptation and generalizable behavior. We developed this thesis through three complementary contributions, summarized below, and conclude with a discussion of future directions.

7.1 Summary of Contributions

Evaluation frameworks for open-world novelty. Chapter 4 introduced NovelGridworlds and NovelGym, benchmark environments that enable the systematic study of agent behavior under novelty. These environments support controlled injection of diverse classes of mismatch—including changes to objects, environment dynamics, and task structure—organized according to the novelty taxonomy developed in Section 2.2.2. They accommodate agents across multiple paradigms (symbolic planning, reinforcement learning, and hybrid architectures) through flexible observation interfaces, and provide evaluation metrics that capture the dynamics of adaptation: how quickly performance degrades following a change, and how efficiently the agent recovers. These benchmarks established the experimental foundation on which the subsequent contributions were evaluated.

Action abstractions for efficient adaptation under novelty. Chapter 5 developed RAPid-Learn, a hybrid planning–learning framework that enables agents to recover from execution failures caused by novelty. The key insight is that structured action abstractions—the operator–executor decomposition formalized in Section 2.4.2—allow failures to be localized to specific components of a plan. When an executor fails because the environment dynamics have changed, the framework converts a global task failure into a targeted learning problem: the agent acquires only the new executor needed to restore successful execution, without relearning previously valid behaviors. Experimental results demonstrated one to two orders of magnitude improvement in sample efficiency compared to unstructured reinforcement learning and transfer learning baselines. The cognitive architecture further extended this framework with a cascading recovery pipeline, progressing from lightweight symbolic strategies to learning-based adaptation, evaluated across a substantially larger and more diverse set of novelty scenarios.

Interaction abstractions for generalizable skill learning. Chapter 6 introduced object-centric, force-based representations for robotic manipulation that capture interaction at the level of object dynamics rather than robot-specific control. FLEX demonstrated that policies learned entirely in force space transfer directly to three different robotic platforms without retraining, achieving over an order-of-magnitude improvement in training efficiency compared to robot-centric baselines. The pushing heavy objects framework extended this approach to rigid body transport under unknown mass and friction, demonstrating zero-shot sim-to-real transfer on a legged mobile manipulator. Both contributions instantiate the interaction abstraction framework of Section 2.4.3: by defining policies in the space of forces applied to the object, the learned behavior captures the physical invariants of the task while remaining agnostic to the specific robot, object instance, or physical parameters.

7.2 A Unified Perspective

The three contributions of this dissertation, while addressing different technical problems, share a common underlying principle: structured abstractions determine how effectively an agent can manage the mismatch between its internal models and the environment. This section develops

the connections between the contributions and articulates the unified perspective they support.

Abstraction as a mediator of novelty. The formal framework developed in Chapter 2 characterized novelty as a mismatch that can affect any component of the agent–environment interaction: objects, attributes, dynamics, or actions. Left unmanaged, such mismatch degrades performance globally—the agent’s entire policy may become ineffective, and recovery requires extensive relearning. The contributions of this dissertation demonstrate that structured abstractions can transform this global problem into a collection of localized ones.

Action abstractions achieve this at the level of task structure. By decomposing behavior into a sequence of operators with explicit preconditions and effects, each associated with a separate executor, the agent creates natural interfaces at which mismatch can be detected and isolated. When novelty causes a specific executor to fail, the agent knows *which* component is affected and *what* the replacement must achieve, reducing adaptation from a full policy search to a targeted learning problem. Interaction abstractions achieve this at the level of individual behaviors. By representing policies in force space rather than robot-centric space, the agent factors out the embodiment-specific details that vary across platforms and objects, ensuring that the learned behavior captures only the task-relevant physical invariants. When a new object or robot is encountered, the invariant representation means that transfer requires only a mapping between force commands and the specific embodiment, rather than relearning the task from scratch.

Complementary roles of adaptation and generalization. Adaptation and generalization represent two complementary strategies for managing novelty. Adaptation responds to novelty after it occurs: the agent detects mismatch, diagnoses its source, and acquires new knowledge to restore performance. Generalization prevents certain classes of novelty from causing mismatch in the first place: by building representations that are invariant to expected variations, the agent’s behavior remains effective without any modification.

The contributions of this dissertation illustrate both strategies and their interplay. Action abstractions support efficient adaptation (Chapter 5): when novelty violates the agent’s model, structured decomposition enables targeted recovery. Interaction abstractions support broad generalization (Chapter 6): force-space representations transfer across objects and em-

bodiments without retraining. Neither strategy alone is sufficient for open-world operation—adaptation without generalization requires relearning for every variation, while generalization without adaptation provides no recourse when genuinely novel situations arise. The unified perspective is that a robust open-world agent requires both, organized through abstractions that determine which variations are handled through invariant representations and which require active learning.

Toward integrated architectures. As discussed in Section 3.5, the natural integration of these contributions would produce an agent architecture in which the planning layer provides task decomposition and failure localization (action abstractions), the execution layer provides generalizable force-based skills (interaction abstractions), and the adaptation layer provides targeted learning when execution fails. The benchmarks developed in Chapter 4 would provide the evaluation infrastructure for such an integrated system. While this full integration was not realized within this dissertation, the formal framework of Chapter 2 and the empirical results of Chapters 5 and 6 establish the components and demonstrate their individual effectiveness. The integration represents the most immediate direction for future work.

7.3 Future Directions

The contributions of this dissertation open several avenues for future research. We organize these directions around three themes: extending the technical contributions, pursuing their integration, and connecting to the broader challenges of open-world autonomy.

Scaling structured abstractions to richer domains. The benchmark environments and adaptation methods developed in this dissertation were evaluated in gridworld and game-based domains that, while supporting diverse classes of novelty, operate at a level of complexity below real-world deployment settings. Extending the hybrid planning–learning framework to domains with continuous state spaces, partial observability, and richer task structure would test the scalability of the action abstraction approach. The force-space methods in Chapter 6 operate in continuous robotic manipulation settings, but on a restricted class of tasks (articulated objects and planar pushing). Extending to multi-step manipulation sequences—where force-based skills

must be composed into longer-horizon plans—would exercise both the action and interaction abstractions simultaneously.

Integrating perception with adaptation and generalization. As discussed in Section 3.1, this dissertation assumes access to task-relevant state information and does not address the perception problem directly. In real-world deployment, the agent must also perceive the environment, detect perceptual novelties, and determine their relevance to the current task. Integrating open-world perception systems—such as open-vocabulary detectors and vision-language models—with the adaptation and generalization mechanisms developed here would close an important loop in the agent architecture. A particularly interesting question is whether the novelty taxonomy of Section 2.2.2 can be extended to perceptual novelties, and whether the structured abstraction framework can be applied to the perception pipeline itself.

Bridging structured abstractions and foundation models. Section 3.4 argued that the abstraction-based approach developed in this dissertation and the foundation model paradigm are complementary. Realizing this complementarity in practice requires developing concrete architectures that combine the two. One promising direction is using large language models to initialize or refine symbolic domain models, with the hybrid framework providing the machinery for detecting when these models are incorrect and adapting accordingly. Another is using VLA models as initial executors within the operator–executor framework, with the targeted learning mechanisms of Chapter 5 enabling efficient fine-tuning when the initial executor fails. A third is augmenting foundation model-based agents with force-space representations for contact-rich manipulation, grounding their behavior in physical interaction dynamics.

Formal connections between novelty type and recovery strategy. The novelty taxonomy of Section 2.2.2 classifies novelties by their impact on task performance (prohibitive, obstructive, beneficial, nuisance), and the adaptation framework of Chapter 5 provides a cascading recovery pipeline. However, the connection between novelty type and the appropriate recovery strategy is currently heuristic: the agent tries lightweight strategies first and escalates as needed. Developing formal principles for selecting recovery strategies based on diagnosed novelty type—for example, proving that nuisance novelties never require executor relearning,

or that prohibitive novelties always require model revision—would provide stronger theoretical foundations for adaptation under novelty.

Multi-agent and collaborative open-world settings. The framework developed in this dissertation considers a single agent operating in an environment. In many real-world settings, multiple agents must coordinate in the presence of novelty—for example, a team of robots in a warehouse where new objects or procedures are introduced. Extending the novelty taxonomy, benchmark environments, and adaptation mechanisms to multi-agent settings raises new challenges: novelty may affect agents asymmetrically, coordination strategies may need to be revised, and the interaction between agents’ adaptation processes must be managed. The force-space representations in Chapter 6 provide a natural starting point for multi-agent manipulation, where multiple robots must coordinate their forces on a shared object.

Lifelong learning and knowledge accumulation. The adaptation methods in Chapter 5 enable an agent to recover from individual novelty events, but they do not address the broader problem of lifelong learning: accumulating knowledge over many encounters with novelty and leveraging this accumulated experience to handle future changes more efficiently. An agent that has recovered from several different types of novelty should be better prepared for the next one—for example, by recognizing patterns in how novelties manifest or by maintaining a library of previously learned executors that can be reused or adapted. Developing mechanisms for such cumulative learning within the structured abstraction framework is an important direction for building agents that improve over extended deployment.

7.4 Closing Remarks

The real world does not respect the boundaries of an agent’s training data. Objects appear that were never labeled, dynamics change in ways that were never simulated, and tasks evolve beyond what was anticipated during system design. The question is not whether agents will encounter novelty, but whether they can respond to it effectively.

This dissertation has argued that the answer depends on how agents represent their interaction with the world. Monolithic representations that tightly couple perception, reasoning, and

action provide no natural interfaces for diagnosing failures or localizing adaptation. Structured abstractions—representations that decompose behavior into compositional sequences and organize interaction around task-relevant invariants—create precisely these interfaces. They enable agents to identify what has changed, restrict learning to what must change, and preserve what still works.

The formal frameworks, benchmark environments, adaptation methods, and generalization techniques developed in this dissertation provide concrete instantiations of this principle. They demonstrate that structured abstractions over actions enable efficient recovery from execution failures caused by novelty, and that structured abstractions over interaction enable learned skills to generalize across objects, physical parameters, and robotic platforms. Beyond the specific algorithms and systems, the formal vocabulary itself—novelty as agent-relative mismatch, the taxonomy by task impact and domain element, the integrated planning task, the distinction between action and interaction abstractions—constitutes a contribution that we hope will be useful to others working on open-world autonomy, independent of the particular methods developed here. Together, these contributions offer a perspective on open-world autonomy in which the structure of an agent’s representations is not merely a design choice but a fundamental determinant of its capacity to operate in a world that is always, in some respect, new.

Bibliography

- [1] Davide Abati, Angelo Porrello, Simone Calderara, and Rita Cucchiara. Latent space autoregression for novelty detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 481–490, 2019.
- [2] Ben Abbatematteo, Eric Rosen, Skye Thompson, Tuluhan Akbulut, Sreehari Rammohan, and George Konidaris. Composable interaction primitives: A structured policy class for efficiently learning sustained-contact manipulation skills. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [3] David Abel. *A Theory of Abstraction in Reinforcement Learning*. PhD thesis, Brown University, 2020.
- [4] David Abel, Dilip Arumugam, Lucas Lehnert, and Michael Littman. State abstractions for lifelong reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, pages 10–19. PMLR, 2018.
- [5] David W. Aha. Goal reasoning: Foundations, emerging applications, and prospects. *AI Magazine*, 39(2):3–24, 2018.
- [6] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [7] Peter Auer. Using confidence bounds for exploitation-exploration trade-offs. *Journal of Machine Learning Research*, 3:397–422, 2002.
- [8] Jonathan Balloch, Zhiyu Lin, Robert Wright, Xiangyu Peng, Mustafa Hussain, Aarun Srinivas, Julia Kim, and Mark O Riedl. Neuro-symbolic world models for adapting to open world novelty. *arXiv preprint arXiv:2301.06294*, 2023.
- [9] Jonathan C Balloch, Zhiyu Lin, Robert Wright, Xiangyu Peng, and Mark O Riedl. Nov-Grid: A flexible grid world environment for studying novelty. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 2022.
- [10] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.
- [11] Abhijit Bendale and Terrance E Boulton. Towards open set deep networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1563–1572, 2016.

- [12] Lennart Bergmann, David Leins, Robert Haschke, and Klaus Neumann. Precision-focused reinforcement learning model for robotic object pushing. In *International Conference on Advanced Robotics and Mechatronics (ICARM)*, pages 758–765, 2025.
- [13] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A Vision-Language-Action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [14] Terrance E Boulton, Henry Grabowecy, Daniel Prijatel, Roberto Fernandez, Lawrence Holder, and Joshua Alspector. Towards a unifying framework for formal theories of novelty. *arXiv preprint arXiv:2103.02529*, 2021.
- [15] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [16] Hung Bui, Yifan Gao, Haoyu Yang, Elvin Cui, Sagar Mody, Brian Acosta, Tomas S Felix, Bruno Bianchini, and Michael Posa. Push anything: Single- and multi-object pushing from first sight with contact-implicit MPC. *arXiv preprint arXiv:2510.19974*, 2025.
- [17] Jiefeng Cheng and Nuno Vasconcelos. Learning deep classifiers consistent with fine-grained novelty detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1664–1673, 2021.
- [18] Alexander Clegg, Wenhao Yu, Zackory Erickson, Jie Tan, C. Karen Liu, and Greg Turk. Learning to navigate cloth using haptics. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2799–2805. IEEE, 2017.
- [19] Lin Cong, Hongtao Liang, Philipp Ruppel, Yongzhuo Shi, Michael Gerner, Norman Hendrich, and Jianwei Zhang. Reinforcement learning with vision-proprioception model for robot planar pushing. *Frontiers in Neurobotics*, 16:829437, 2022.
- [20] Iason Dadiotis, Mayank Mittal, Nikos Tsagarakis, and Marco Hutter. Dynamic object goal pushing with mobile manipulators through model-free constrained reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 13363–13369, 2025.
- [21] Alexander Dietrich, Kristin Bussmann, Florian Petit, Paul Kotyczka, Christian Ott, Boris Lohmann, and Alin Albu-Schäffer. Whole-body impedance control of wheeled mobile manipulators: Stability analysis and experiments on the humanoid robot Rollin’ Justin. *Autonomous Robots*, 40(3):505–517, 2016.
- [22] Ben Eisner, Harry Zhang, and David Held. FlowBot3D: Learning 3D articulation flow to manipulate articulated objects. *arXiv preprint arXiv:2205.04382*, 2022.
- [23] Íñigo Elguea-Aguinaco, Antonio Serrano-Muñoz, Dimitrios Chrysostomou, Ibai Inziarte-Hidalgo, Simon Bøgh, and Nestor Arana-Arexolaleiba. A review on reinforcement learning for contact-rich robotic manipulation tasks. *Robotics and Computer-Integrated Manufacturing*, 81:102517, 2023.
- [24] Shijie Fang, Wenchang Gao, Shivam Goel, Christopher Thierauf, Matthias Scheutz, and Jivko Sinapov. FLEX: A framework for learning robot-agnostic force-based skills involving sustained contact object manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

- [25] Nima Fazeli, Samuel Zapolsky, Evan Drumwright, and Alberto Rodriguez. Learning data-efficient rigid-body contact models: Case study of planar impact. In *Conference on Robot Learning (CoRL)*, pages 388–397. PMLR, 2017.
- [26] Patrick Feeney, Sarah Schneider, Panagiotis Lymperopoulos, Li-Ping Liu, Matthias Scheutz, and Michael C. Hughes. NovelCraft: A dataset for novelty detection and discovery in open worlds. *arXiv preprint arXiv:2206.11736*, 2022.
- [27] Juan Del Aguila Ferrandis, João Moura, and Sethu Vijayakumar. Nonprehensile planar manipulation through reinforcement learning with multimodal categorical exploration. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5606–5613, 2023.
- [28] Richard E Fikes and Nils J Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4):189–208, 1971.
- [29] Maria Fox, Alfonso Gerevini, Derek Long, Ivan Serina, et al. Plan stability: Replanning versus plan repair. In *ICAPS*, volume 6, pages 212–221, 2006.
- [30] Christian Fritz and Sheila A McIlraith. Monitoring plan optimality during execution. In *ICAPS*, pages 144–151, 2007.
- [31] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- [32] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, pages 1587–1596. PMLR, 2018.
- [33] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2020.
- [34] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021.
- [35] Yiran Geng, Boshi An, Haoran Geng, Yuanpei Chen, Yaodong Yang, and Hao Dong. RLafford: End-to-end affordance learning for robotic manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 5880–5886. IEEE, 2023.
- [36] Evana Gizzi, Mateo Guaman Castro, and Jivko Sinapov. Creative problem solving by robots using action primitive discovery. In *IEEE International Conference on Development and Learning and Epigenetic Robotics (ICDL-EPIROB)*, pages 228–233, 2019.
- [37] Evana Gizzi, Hassan Amel, Wo Wei Lin, Keenan Rhea, and Jivko Sinapov. Toward creative problem solving agents: Action discovery through behavior babbling. In *IEEE International Conference on Development and Learning (ICDL)*, pages 1–7, 2021.
- [38] Evana Gizzi, Wo Wei Lin, Mateo Guaman Castro, Ethan Harvey, and Jivko Sinapov. Toward life-long creative problem solving: Using world models for increased performance in novelty resolution. In *International Conference on Computational Creativity (ICCC)*, pages 121–125, 2022.

- [39] Evana Gizzi, Lakshmi Nair, Sonia Chernova, and Jivko Sinapov. Creative problem solving in artificially intelligent agents: A survey and framework. *arXiv preprint arXiv:2204.10358*, 2022.
- [40] Rafael Glatt and Anna Helena Reali Costa. Policy reuse in deep reinforcement learning. In *AAAI Conference on Artificial Intelligence*, 2017.
- [41] Shivam Goel, Gyan Tatiya, Matthias Scheutz, and Jivko Sinapov. Novelgridworlds: A benchmark environment for detecting and adapting to novelties in open worlds. In *AA-MAS Workshop on Adaptive Learning Agents (ALA)*, 2021.
- [42] Shivam Goel, Yash Shukla, Vasanth Sarathy, Matthias Scheutz, and Jivko Sinapov. Rapid-learn: A framework for learning to recover for handling novelties in open-world environments. In *IEEE International Conference on Development and Learning (ICDL), London, UK, September 12-15, 2022*, pages 1–8. IEEE, 2022. URL <https://arxiv.org/pdf/2206.12493>.
- [43] Shivam Goel, Panagiotis Lymperopoulos, Ravenna Thielstrom, Evan Krause, Patrick Feeney, Pierrick Lorang, Sarah Schneider, Yichen Wei, Eric Kildebeck, Stephen Goss, et al. A neurosymbolic cognitive architecture framework for handling novelties in open worlds. *Artificial Intelligence*, 331:104111, 2024.
- [44] Shivam Goel, Yichen Wei, Panagiotis Lymperopoulos, Klára Churá, Matthias Scheutz, and Jivko Sinapov. Novelgym: A flexible ecosystem for hybrid planning and learning agents designed for open worlds. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, pages 688–696, 2024.
- [45] Shivam Goel, Timothy Duggan, Tomas Maranga, Matthias Scheutz, and Jivko Sinapov. Pushing heavy objects: Learning object-centric force policies for sustained-contact manipulation. In *Submitted to IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. Under review.
- [46] Stephen A. Goss, Robert J. Steininger, Deepayan Narayanan, Daniel V. Olivença, Yuan Sun, Paul Qiu, James Amato, Eberhard O. Voit, Walter E. Voit, and Eric J. Kildebeck. Polycraft world AI lab (PAL): An extensible platform for evaluating artificial intelligence agents. *arXiv preprint arXiv:2301.11891*, 2023.
- [47] Suresh Goyal, Andy Ruina, and Jim Papadopoulos. Planar sliding with dry friction part 1. limit surface and moment function. *Wear*, 143(2):307–330, 1991.
- [48] Shixiang Gu, Ethan Holly, Timothy Lillicrap, and Sergey Levine. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3389–3396. IEEE, 2017.
- [49] Xiuye Gu, Tsung-Yi Lin, Weicheng Kuo, and Yin Cui. Open-vocabulary object detection via vision and language knowledge distillation. In *International Conference on Learning Representations (ICLR)*, 2022.
- [50] Lin Guan, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging approximate symbolic models for reinforcement learning via skill diversity. *arXiv preprint arXiv:2202.02886*, 2022.

- [51] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [52] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1856–1865, 2018.
- [53] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [54] Malte Helmert. The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- [55] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [56] Jörg Hoffmann. The Metric-FF planning system: Translating “ignoring delete lists” to numeric state variables. *Journal of Artificial Intelligence Research*, 20:157–194, 2003.
- [57] Francois R Hogan and Alberto Rodriguez. Reactive planar non-prehensile manipulation with hybrid model predictive control. *The International Journal of Robotics Research*, 39(7):755–773, 2020.
- [58] Francois R Hogan, Eudald Romo Grau, and Alberto Rodriguez. Reactive planar manipulation with convex hybrid MPC. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 247–253. IEEE, 2018.
- [59] Neville Hogan. Impedance control: An approach to manipulation. part II—implementation. *Journal of Dynamic Systems, Measurement, and Control*, 107(1):8–16, 1985.
- [60] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.
- [61] León Illanes, Xi Yan, Rodrigo Toro Icarte, and Sheila A McIlraith. Symbolic plans as high-level instructions for reinforcement learning. In *International Conference on Automated Planning and Scheduling (ICAPS)*, volume 30, pages 540–550, 2020.
- [62] Steven James, Benjamin Rosman, and George Konidaris. Autonomous learning of object-centric abstractions for high-level planning. In *International Conference on Learning Representations*, 2022.
- [63] Minqi Jiang, Jelena Luketina, Nantas Nardelli, Pasquale Minervini, Philip HS Torr, Shimon Whiteson, and Tim Rocktäschel. WordCraft: An environment for benchmarking commonsense agents. *arXiv preprint arXiv:2007.09185*, 2020.
- [64] Mu Jin, Zhihao Ma, Kebin Jin, Hankz Hankui Zhuo, Chen Chen, and Chao Yu. Creativity of AI: Automatic symbolic option discovery for facilitating deep reinforcement learning. *arXiv preprint arXiv:2112.09836*, 2021.

- [65] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Hierarchical task and motion planning in the now. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1470–1477, 2011.
- [66] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning (CoRL)*, pages 651–673. PMLR, 2018.
- [67] Oussama Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1): 43–53, 1987.
- [68] Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75:1401–1476, 2022.
- [69] Tae Kyun Kim. T test as a parametric statistic. *Korean Journal of Anesthesiology*, 68(6):540–546, 2015.
- [70] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.
- [71] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- [72] Matthew Klenk, Wiktor Piotrowski, Roni Stern, Shiwali Mohan, and Johan de Kleer. Model-based novelty adaptation for open-world AI. In *International Workshop on Principles of Diagnosis (DX)*, 2020.
- [73] Harsha Kokel, Arjun Manoharan, Sriraam Natarajan, Balaraman Ravindran, and Prasad Tadepalli. RePReL: Integrating relational planning and reinforcement learning for effective abstraction. In *International Conference on Automated Planning and Scheduling (ICAPS)*, pages 533–541, 2021.
- [74] George Konidaris. Constructing abstraction hierarchies using a skill-symbol loop. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, pages 1648–1654, 2016.
- [75] George Konidaris. On the necessity of abstraction. *Current Opinion in Behavioral Sciences*, 29:1–7, 2019.
- [76] George Konidaris and Andrew G Barto. Building portable options: Skill transfer in reinforcement learning. In *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence*, pages 895–900, 2007.
- [77] George Konidaris, Leslie Pack Kaelbling, and Tomas Lozano-Perez. Constructing symbolic representations for high-level planning. In *AAAI Conference on Artificial Intelligence*, pages 1932–1940, 2014.

- [78] George Konidaris, Leslie Pack Kaelbling, and Tomas Lozano-Perez. From skills to symbols: Learning symbolic representations for abstract high-level planning. *Journal of Artificial Intelligence Research*, 61:215–289, 2018.
- [79] Oliver Kroemer, Scott Niekum, and George Konidaris. A review of robot learning for manipulation: Challenges, representations, and algorithms. *Journal of Machine Learning Research*, 22(30):1–82, 2021.
- [80] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- [81] Pat Langley. Open-world learning for radically autonomous agents. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(09):13539–13543, 2020.
- [82] Yunseung Lee and Pilsung Kang. AnoViT: Unsupervised anomaly detection and localization with vision transformer-based encoder-decoder. *arXiv preprint arXiv:2203.10808*, 2022.
- [83] Lihong Li, Thomas J Walsh, and Michael L Littman. Towards a unified theory of state abstraction for MDPs. In *Proceedings of the International Symposium on Artificial Intelligence and Mathematics*, pages 531–539, 2006.
- [84] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.
- [85] Minghuan Liu, Menghui Zhu, and Weinan Zhang. Goal-conditioned reinforcement learning: Problems and solutions. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 5502–5511, 2022.
- [86] Pierrick Lorang, Shivam Goel, Philipp Zips, Jivko Sinapov, and Matthias Scheutz. Speeding-up continual learning through information gains in novel experiences. In *4th Planning and Reinforcement Learning (PRL) Workshop at IJCAI-2022*, 2022.
- [87] Pierrick Lorang, Helmut Horvath, Tobias Kietreiber, Patrik Zips, Clemens Heitzinger, and Matthias Scheutz. Adapting to the “open world”: The utility of hybrid hierarchical reinforcement learning and symbolic planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [88] Pierrick Lorang, Hong Lu, and Matthias Scheutz. Curiosity-driven imagination: Discovering plan operators and learning associated policies for open-world adaptation. *arXiv preprint arXiv:2503.04931*, 2025.
- [89] Kendall Lowrey, Svetoslav Kolev, Jeremy Dao, Aravind Rajeswaran, and Emanuel Todorov. Reinforcement learning for non-prehensile manipulation: Transfer from simulation to physical system. In *IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN)*, pages 35–42. IEEE, 2018.
- [90] Kai Lu, Bing Yang, Bo Wang, and Andrew Markham. Decoupling skill learning from robotic control for generalizable object manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 864–870. IEEE, 2023.

- [91] Kevin M Lynch and Matthew T Mason. Stable pushing: Mechanics, controllability, and planning. *The International Journal of Robotics Research*, 15(6):533–556, 1996.
- [92] Daoming Lyu, Fangkai Yang, Bo Liu, and Steven Gustafson. SDRL: Interpretable and data-efficient deep reinforcement learning leveraging symbolic planning. In *AAAI Conference on Artificial Intelligence*, pages 2970–2977, 2019.
- [93] Matthew T Mason. Mechanics and planning of manipulator pushing operations. *The International Journal of Robotics Research*, 5(3):53–71, 1986.
- [94] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.
- [95] Matthias Minderer, Alexey Gritsenko, Austin Stone, Maxim Neumann, Dirk Weissenborn, Alexey Dosovitskiy, Aravindh Mahendran, Anurag Arnab, Mostafa Dehghani, Zhuoran Shen, et al. Simple open-vocabulary object detection. In *European conference on computer vision*, pages 728–755. Springer, 2022.
- [96] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [97] Kaichun Mo, Leonidas J Guibas, Mustafa Mukadam, Abhinav Gupta, and Shubham Tulsiani. Where2Act: From pixels to actions for articulated 3D objects. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6813–6823, 2021.
- [98] Carlota Parés Morlans, Changhao Chen, Yuxing Weng, Mykel Yi, Yuzhe Huang, Nick Heppert, Linqi Zhou, Leonidas Guibas, and Jeannette Bohg. AO-Grasp: Articulated object grasp generation. *arXiv preprint arXiv:2310.15928*, 2023.
- [99] Faizan Muhammad, Vasanth Sarathy, Gyan Tatiya, Shivam Goel, Suman Gyawali, Martha Guaman, Jivko Sinapov, and Matthias Scheutz. A novelty-centric agent architecture for changing worlds. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 925–933, 2021.
- [100] Lakshmi Nair, Evana Gizzi, and Jivko Sinapov. Creative problem solving in large language and vision models – what would it take? In *Findings of the Association for Computational Linguistics: EMNLP*, pages 11978–11994, 2024.
- [101] Sanmit Narvekar, Jivko Sinapov, Matteo Leonetti, and Peter Stone. Source task creation for curriculum learning. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 566–574, 2016.
- [102] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.

- [103] Soroush Nasiriany, Huihan Liu, and Yuke Zhu. Augmenting reinforcement learning with behavior primitives for diverse manipulation tasks. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7477–7484. IEEE, 2022.
- [104] Alex Nichol, Vicki Pfau, Christopher Hesse, Oleg Klimov, and John Schulman. Gotta learn fast: A new benchmark for generalization in RL. *arXiv preprint arXiv:1804.03720*, 2018.
- [105] Fabio Pardo. Tonic: A deep reinforcement learning library for fast prototyping and benchmarking. *arXiv preprint arXiv:2011.07537*, 2020.
- [106] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning (ICML)*, pages 2778–2787, 2017.
- [107] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018.
- [108] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.
- [109] Eric Rosen, Ben M. Abbatematteo, Skye Thompson, Tuluhan Akbulut, and George Konidaris. On the role of structure in manipulation skill learning. In *CoRL 2022 Workshop on Learning, Perception, and Abstraction for Long-Horizon Planning*, 2022.
- [110] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- [111] Vasanth Sarathy and Matthias Scheutz. MacGyver problems: AI challenges for testing resourcefulness and creativity. *Advances in Cognitive Systems*, 6, 2018.
- [112] Vasanth Sarathy, Daniel Kasenberg, Shivam Goel, Jivko Sinapov, and Matthias Scheutz. SPOTTER: Extending symbolic planning operators through targeted reinforcement learning. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 1118–1126, 2021.
- [113] Walter J Scheirer, Anderson de Rezende Rocha, Archana Sapkota, and Terrance E Boulton. Toward open set recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(7):1757–1772, 2013.
- [114] Paul Schermerhorn, James Kramer, Timothy Brick, David Anderson, Aaron Dingler, and Matthias Scheutz. DIARC: A testbed for natural human-robot interaction. In *AAAI Workshop on Mobile Robot Competition and Exhibition*, 2006.
- [115] Matthias Scheutz, Selmer Bringsjord, and Naveen Sundar Govindarajulu. An overview of the distributed integrated cognition affect and reflection DIARC architecture. In *Cognitive Architecture Integrations: Common Model of Cognition and Beyond*, pages 201–224. Springer, 2019.

- [116] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- [117] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [118] Ted Senator. Science of artificial intelligence and learning for open-world novelty (SAIL-ON). *DARPA Broad Agency Announcement HR001119S0038*, 2019.
- [119] Tom Silver and Rohan Chitnis. PDDL Gym: Gym environments from PDDL problems. In *International Conference on Automated Planning and Scheduling (ICAPS) PRL Workshop*, 2020.
- [120] Tom Silver, Rohan Chitnis, Joshua B Tenenbaum, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Learning symbolic operators for task and motion planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3182–3189, 2021.
- [121] Roni Stern, Wiktor Piotrowski, Matthew Klenk, Johan de Kleer, Alexandre Perez, Jacob Le, and Shiwali Mohan. Model-based adaptation to novelty for open-world AI. In *Proceedings of the ICAPS Workshop on Bridging the Gap Between AI Planning and Learning*, 2022.
- [122] Jochen Stüber, Claudio Zito, and Rustam Stolkin. Let’s push things forward: A survey on robot pushing. *Frontiers in Robotics and AI*, 7:8, 2020.
- [123] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [124] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- [125] Yujie Tang, Hai Zhu, Sjoerd Potters, Martijn Wisse, and Wei Pan. Unwieldy object delivery with nonholonomic mobile base: A stable pushing approach. *IEEE Robotics and Automation Letters*, 8(11):7727–7734, 2023.
- [126] Zehui Tang, Yuming Feng, and Meng Guo. Collaborative planar pushing of polytopic objects with multiple robots in complex scenes. *arXiv preprint arXiv:2405.07908*, 2024.
- [127] Matthew E Taylor and Peter Stone. Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research*, 10(56):1633–1685, 2009.
- [128] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [129] Jordan K. Terry, Benjamin Black, Nathaniel Grammel, Ananth Hari Jayakumar, Sanghyun Lee, Luis Santos, Claus Dieffendahl, Caroline Horsch, and Rodrigo Pérez-Vicente. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 15032–15043, 2021.

- [130] Tung Thai, Ming Shen, Neeraj Varshney, Sriram Gopalakrishnan, Utkarsh Soni, Chitta Baral, Matthias Scheutz, and Jivko Sinapov. An architecture for novelty handling in a multi-agent stochastic environment: Case study in open-world monopoly. In *Designing Artificial Intelligence for Open Worlds: Papers from the 2022 Spring Symposium, AAAI Press*, 2022.
- [131] Christopher Thierauf and Matthias Scheutz. Fixing symbolic plans with reinforcement learning in object-based action spaces. *Learning*, 21:22, 2024.
- [132] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.
- [133] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012.
- [134] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [135] Shuoguang Wang, Liang Sun, Fusheng Zha, Weihai Guo, and Pengshuai Wang. Learning adaptive reaching and pushing skills using contact information. *Frontiers in Neurorobotics*, 17:1271607, 2023.
- [136] Jiayi Weng, Min Chen, Yujie Shi, Minghao Li, Zhihan Zhang, Feiyang Li, Jun Zhu, and Yun Liu. Tianshou: A highly modularized deep reinforcement learning library. *Journal of Machine Learning Research*, 23(267):1–6, 2022.
- [137] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [138] Ruihai Wu, Yan Zhao, Kaichun Mo, Zizheng Guo, Yian Wang, Tianhao Wu, Qingnan Fan, Xuelin Chen, Leonidas Guibas, and Hao Dong. VAT-Mart: Learning visual action trajectory proposals for manipulating 3D articulated objects. *arXiv preprint arXiv:2106.14440*, 2021.
- [139] Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yinghao Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X Chang, Leonidas J Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [140] Zhenjia Xu, Zhanpeng He, and Shuran Song. Universal manipulation policy network for articulated objects. *IEEE Robotics and Automation Letters*, 7(2):2447–2454, 2022.
- [141] Fangkai Yang, Daoming Lyu, Bo Liu, and Steven Gustafson. PEORL: Integrating symbolic planning and hierarchical reinforcement learning for robust decision-making. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4860–4866, 2018.

- [142] Qiaojun Yu, Junbo Wang, Wenhai Liu, Ce Hao, Liu Liu, Lin Shao, Weiming Wang, and Cewu Lu. GAMMA: Generalizable articulation modeling and manipulation for articulated objects. *arXiv preprint arXiv:2309.16264*, 2024.
- [143] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 1094–1100. PMLR, 2020.
- [144] Qinhong Zhou, Hongxin Zhang, Xiangye Lin, Zheyuan Zhang, Yutian Chen, Wenjun Liu, Zunzhe Zhang, Sunli Chen, Lixing Fang, Qiushi Lyu, et al. Virtual community: An open world for humans, robots, and society. *arXiv preprint arXiv:2508.14893*, 2025.
- [145] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.
- [146] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.